



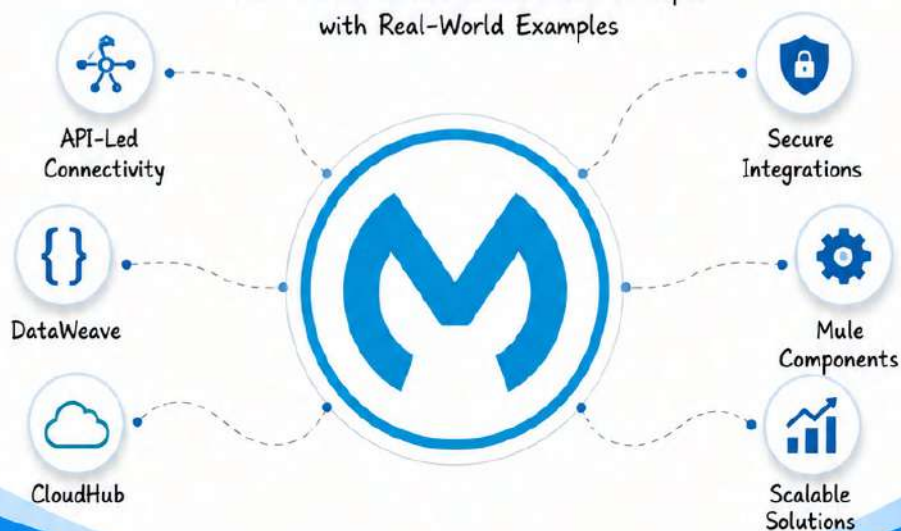
The Complete

# MULESOFT

## INTERVIEW PREPARATION & PRACTICAL GUIDE

A 7.5+ Years Developer's Companion

From Fundamentals to Advanced Concepts  
with Real-World Examples



40+ Modules  
Comprehensive  
Coverage

Real-World  
Examples  
Industry-Level  
Scenarios

Interview  
Questions  
For All Levels

Practical  
Approach  
Step-by-Step  
Learning

Career  
Focused  
Get Job-Ready  
with Confidence

Learn. Build. Integrate. Succeed.

\*\*\*  
YOUR JOURNEY  
TO BECOME A  
MULESOFT  
EXPERT

## 6.1 ERROR TYPES

### MODULE 6 ERROR HANDLING

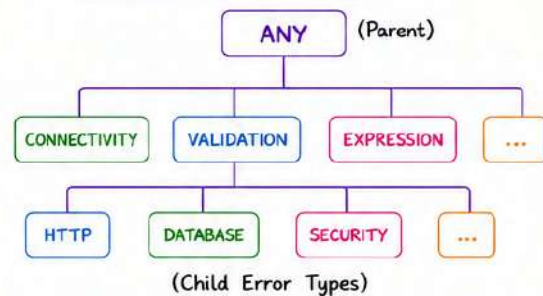


In MuleSoft, every error belongs to a specific category called an **Error Type**.  
Error Types help us understand the **nature of the error** and handle them properly using Error Handlers.

#### KEY POINTS

- Every error in MuleSoft has an **errorType**.
- Error Types are organized in a **hierarchy**.
- Mule provides **built-in** error types.
- We can create **custom error types** if required.
- We handle **specific errors** in **On Error Scope** based on their **error type**.
- **ANY** is the parent of all error types.

#### ERROR TYPE HIERARCHY



#### COMMON MULE ERROR TYPES

| #  | ERROR TYPE            | DESCRIPTION                                           | EXAMPLE SCENARIOS                                                 |
|----|-----------------------|-------------------------------------------------------|-------------------------------------------------------------------|
| 1  | <b>ANY</b>            | Parent error type of all errors.                      | Catches any type of error if no specific handler is defined.      |
| 2  | <b>CONNECTIVITY</b>   | Occurs when Mule cannot connect to a resource.        | Database down, API not reachable, Timeout, DNS issues.            |
| 3  | <b>VALIDATION</b>     | Occurs when data validation fails.                    | Schema validation failure, required field missing.                |
| 4  | <b>EXPRESSION</b>     | Occurs when DataWeave or MEL expression fails.        | Invalid MEL, DW script error, Null pointer in expression.         |
| 5  | <b>TRANSFORMATION</b> | Occurs when data transformation fails.                | DataWeave mapping error, invalid type conversion.                 |
| 6  | <b>SECURITY</b>       | Occurs when security validation/authentication fails. | Invalid credentials, Access token expired, Unauthorized access.   |
| 7  | <b>ROUTING</b>        | Occurs when event routing fails.                      | No matching route, VM queue not found.                            |
| 8  | <b>HTTP</b>           | Represents HTTP related errors.                       | 4xx, 5xx responses from API.                                      |
| 9  | <b>DATABASE</b>       | Occurs when database operation fails.                 | SQL exception, Connection failure, Constraint violation.          |
| 10 | <b>BUSINESS</b>       | Used for business rule violations (custom).           | Insufficient balance, Invalid order, Business validation failure. |

#### ✓ REAL-WORLD EXAMPLE

If an HTTP Request to a REST API returns 404, the error type will be **HTTP:NOT\_FOUND**.



#### 💡 INTERVIEW TIPS

- **ANY** is the parent of all error types.
- Always handle specific errors before **ANY**.
- Use **error.errorType** to identify the error.
- Custom error types should extend **BUSINESS**.

#### ★ BEST PRACTICES

- Always log the error with meaningful message.
- Return user-friendly error response.
- Use **correlationId** to track errors.
- Avoid using **ANY** unless it's the last handler.
- Notify support team for critical errors.

#### ⚠ COMMON MISTAKES

- Catching **ANY** error in the first handler.
- Not returning proper error response.
- Ignoring specific error types.
- Not logging error details.



Understanding **Error Types** is the first step towards building **robust, fault-tolerant** Mule applications.





## 6.2 GLOBAL ERROR HANDLER

### MODULE 6 ERROR HANDLING



MuleSoft

A Global Error Handler is a **centralized error handling** mechanism that catches errors thrown by **any flow** or any component in the application that are **not handled** by local error handlers.

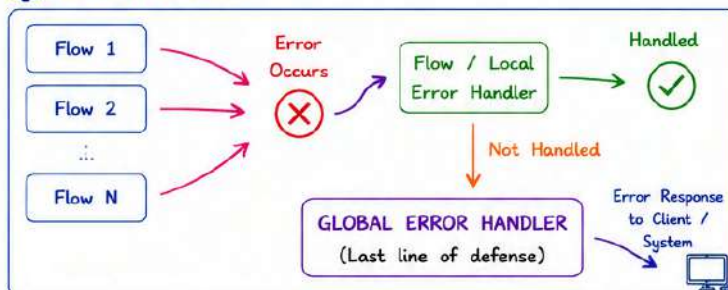
#### KEY POINTS

- Provides **centralized error** management for the entire application.
- Executes only if **no other error handler** handles the error.
- Ensures **consistent error response** to the client/system.
- Improves **maintainability** and reduces code duplication.
- Helps in **logging, alerting, monitoring** and notification.
- Defined once in **mule.xml** for the whole application.

#### WHY USE GLOBAL ERROR HANDLER?

- ✓ Centralized and **consistent** error handling
- ✓ Avoids **duplication** of error handling logic
- ✓ Handles **unexpected / unhandled** errors
- ✓ Improves **reliability** and user experience
- ✓ Makes application easier to **monitor**

#### HOW IT WORKS?



#### SCOPE

- Applies to the entire Mule application.
- Executed only if no other error handler handles the error.
- Defined in the **global-error-handler** element in **mule.xml**.

#### CONFIGURATION IN mule.xml

```
<mule>
  <global-error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR"
        message="Global Error Occurred: #[error.description]"/>
      <set-payload value='{"status":"ERROR",
        "message":#[error.description]}'
        mimeType="application/json"/>
    </on-error-continue>
  </global-error-handler>
</mule>
```

This will handle all unhandled errors of type ANY

#### REAL-WORLD EXAMPLE

If an HTTP Request fails due to a timeout and no On Error is defined in the flow, the error is caught by the **Global Error Handler** and a proper **JSON** response is returned to the client.



#### FLOW-LEVEL ERROR HANDLER VS GLOBAL ERROR HANDLER

| Aspect        | Flow-level / Local Error Handler             | Global Error Handler                                    |
|---------------|----------------------------------------------|---------------------------------------------------------|
| Scope         | Specific flow or scope                       | Entire application                                      |
| Execution     | Executed first                               | Executed only if no other handler handles the error.    |
| Use Case      | Handle expected / known errors inside a flow | Handle unexpected / unhandled errors across application |
| Configuration | Inside the flow or scope                     | In <b>mule.xml</b> ( <b>global-error-handler</b> )      |
| Frequency     | Multiple (as per requirement)                | Only one (last line of defense)                         |

#### BEST PRACTICES

- Always return user-friendly error response.
- Log the error with **correlationId**.
- Avoid exposing internal error details.
- Use proper HTTP status codes.
- Notify support team for critical errors.

#### INTERVIEW TIPS

- Global Error Handler is the **last line of defense**.
- It handles unhandled errors in the entire application.
- It is defined only once in **mule.xml**.
- Use **error.errorType** to identify the error.

#### COMMON MISTAKES

- Defining Global Error Handler multiple times.
- Returning technical stack trace to the client.
- Not logging error details.
- Using only **ANY** without specific handling.

#### KEY TAKEAWAY

Global Error Handler ensures **consistent, centralized and reliable** error handling for the entire Mule application.



## 6.3

## ON ERROR CONTINUE

MODULE 6  
ERROR HANDLING

MuleSoft

On Error Continue **handles an error** and allows the flow to **continue executing the next steps** instead of stopping the flow.

## KEY POINTS

- Used when we want the flow to **continue** even if an error occurs.
- The error is handled and the flow moves to the next processor.
- Useful for **non-critical operations**.
- Prevents the flow from failing.
- Can be used at **flow level** or inside a **Try Scope**.

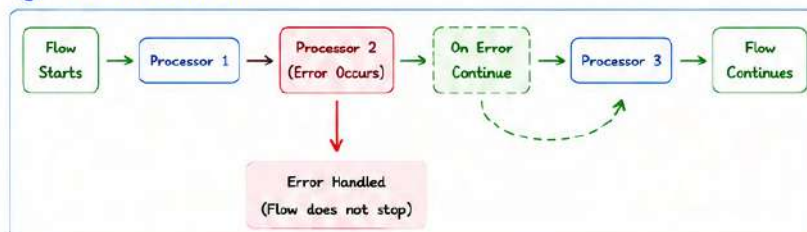


## WHEN TO USE?

- **Logging** errors and continuing the flow.
- **Optional** notifications (email, SMS).
- **Audit** operations.
- **Non-critical** API calls.
- **Data enrichment** steps.



## HOW IT WORKS?



## TYPES

- 1 **Flow Level**  
Handles errors for the entire flow.
- 2 **Try Scope Level**  
Handles errors only within the Try Scope.



## SYNTAX IN MULE.XML

```

<flow name="on-error-continue-example">
  <http:listener config-ref="HTTP Listener Config" path="/continue" />
  <set-payload value="#[payload]" />
  <try>
    <!-- Code that may throw an error -->
    <db:select config-ref="DB Config">
      <db:sql>SELECT * FROM EMP WHERE ID = :id</db:sql>
      <db:input-parameters><![CDATA[#{'id': payload.id}]]></db:input-parameters>
    </db:select>
    <logger level="INFO" message="DB operation successful" />
  </try>
  <error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR" message="Error occurred but continuing the flow" />
      <set-payload value="#['message': 'Default data used']" />
    </on-error-continue>
  </error-handler>
  <set-payload value="#[payload]" />
  <logger level="INFO" message="Flow continues after error" />
</flow>

```

Flow continues execution even if an error occurs!



## REAL-WORLD EXAMPLE

An eCommerce application calls a Promotional API to get discount information.

- If the API is down, we still want to show the product details.
- So we handle the error and continue the flow using default discount (0%).



Business is **not blocked** due to a **non-critical** failure.



## BEST PRACTICES

- ✓ Use On Error Continue only for **non-critical** errors.
- ✓ Log the error with proper details.
- ✓ Return a meaningful default response.
- ✓ Do not hide important errors.
- ✓ Notify support team for critical failures.
- ✓ Use **correlationId** for tracking.



## INTERVIEW TIPS

- On Error Continue handles the error and allows the flow to **continue**.
- It is used for **non-critical** operations.
- It prevents the flow from failing.
- It can be applied at flow level or inside Try Scope.



## COMMON MISTAKES

- ✗ Using it for critical errors.
- ✗ Ignoring the error completely.
- ✗ Not logging the error.
- ✗ Returning null or incomplete data.
- ✗ Hiding errors from monitoring and alerting.



## KEY TAKEAWAY

On Error Continue helps you handle an **error** and **continue** the flow for **non-critical** scenarios, ensuring better **resilience** and user experience.



Use On Error Continue wisely to keep your integrations **resilient** and **user-friendly** while ensuring important errors are still **captured and monitored**.





## 6.4 ON ERROR PROPAGATE

### MODULE 6 ERROR HANDLING



On Error Propagate **handles an error** and then **re-throws (propagates)** it to the **next error handler** in the chain. It is used when the current flow cannot fully handle the error and wants the **upper level** or **global** error handler to take control.

#### KEY POINTS

- Used when the current flow **cannot completely** handle the error.
- The error is handled partially (logging, cleanup) and then propagated.
- Control is passed to the **next applicable error handler**.
- Useful for **layered error handling** (Flow → Try → Global).
- Ensures **consistent error response** at higher level.
- Preserves **original error** information.
- Can be used at **flow level** or inside a **Try Scope**.

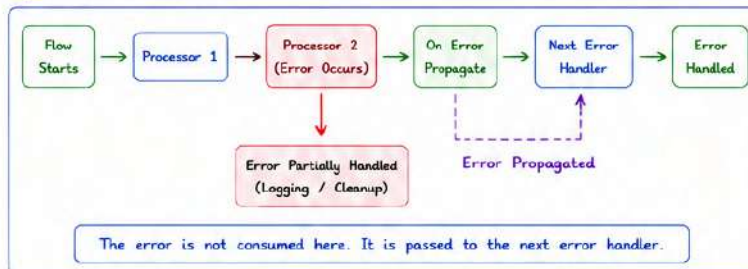


#### WHEN TO USE?

- When the flow **cannot recover** from the error.
- When you want a **common error response** from a higher level handler.
- When the error needs to be logged at **multiple levels**.
- When business or technical teams need **centralized** error handling.
- When a **Try Scope** should not consume the error.



#### HOW IT WORKS?



#### TYPES

- 1 **Flow Level**  
Propagates the error to the next applicable handler (Try Scope or Global Error Handler).
- 2 **Try Scope Level**  
Propagates the error outside the Try Scope to the next outer handler.



#### SYNTAX IN MULE.XML

```
<flow name="on-error-propagate-example">
  <http:listener config-ref="HTTP Listener Config" path="/propagate" />
  <set-payload value="#['start']" />
  <try>
    <!-- Code that may throw an error -->
    <db:select config-ref="DB Config">
      <db:sql>SELECT * FROM ORDERS WHERE ID = :id</db:sql>
      <db:input-parameters>#[[id: payload.id]]</db:input-parameters>
    </db:select>
    </try>
    <error-handler>
      <on-error-propagate type="ANY">
        <logger level="ERROR" message="Error occurred. Propagating to next handler" />
        <set-variable variableName="errorInfo" value="#[error.description]" />
      </on-error-propagate>
    </error-handler>
  </flow>
```

Error is logged and then propagated to the next handler.



#### REAL-WORLD EXAMPLE

In a banking application, if a transaction fails in the database:

- The flow logs the error.
- It propagates the error to the global handler.
- The global handler sends the standardized error response to the client.



#### BEST PRACTICES

- ✓ Use On Error Propagate when the error cannot be fully handled in the current scope.
- ✓ Always log meaningful details before propagating.
- ✓ Let the global error handler send the final response.
- ✓ Maintain a clear error handling hierarchy.
- ✓ Avoid using it for handled (non-critical) errors.



#### INTERVIEW TIPS

- On Error Propagate re-throws the error to the next handler.
- Use it when you want the upper level handler to take control.
- It maintains the original error information.
- Commonly used in layered error handling.
- Understand the difference between Continue and Propagate.



#### COMMON MISTAKES

- ✗ Using it for errors that can be handled locally.
- ✗ Not logging the error before propagating.
- ✗ Propagating without understanding the error hierarchy.
- ✗ Catching the error again at the same level.
- ✗ Returning a response in the same scope after propagating.



#### KEY TAKEAWAY

On Error Propagate allows you to **log / clean up** and then **pass the error** to the next handler, ensuring **centralized and consistent** error management across the application.



Use On Error Propagate to **pass the baton**, not to finish the race. It ensures the **right handler** handles the error at the **right level**.



## 6.5

## RAISE ERROR

MODULE 6  
ERROR HANDLING

MuleSoft

Raise Error is used to manually **create and throw** a **custom error** from your flow. It helps in handling **business validations**, custom conditions or unexpected scenarios.

## KEY POINTS

- Used to throw a **custom error** intentionally.
- Triggers error handling mechanism (**On Error Continue / On Error Propagate / Try Scope**).
- Helps in enforcing **business rules** and validations.
- Includes **error type**, **description** and optional attributes.
- Can be raised at **any point** in the flow.
- Useful for reusing **custom error types** across flows.

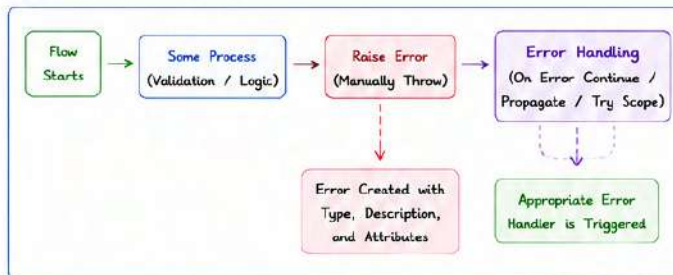


## WHEN TO USE?

- Business validation** fails.
- Required data is **missing** or **invalid**.
- External system returns **unexpected result**.
- You want to **stop the current process** and invoke error handling.
- To return **meaningful** and **standardized error response**.



## HOW IT WORKS?



## COMPONENT CONFIGURATION

Display Name:

Error Type:

Error Description:

Attributes:

| Name          | Value            |
|---------------|------------------|
| field         | email            |
| rejectedValue | #[payload.email] |
| errorCode     | E1001            |



## SYNTAX IN MULE.XML

```

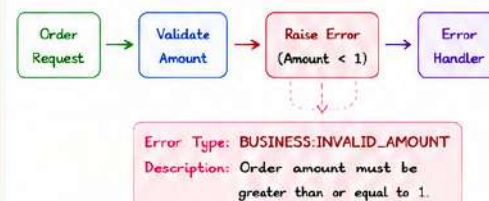
<raise-error type="VALIDATION:INVALID_INPUT"
  description="Invalid input provided. Please check
    and try again.">
  <error-variable>myError</error-variable>
  <set-variable variableName="myError">
    <set-payload value="#[[
      'field': 'email',
      'rejectedValue': payload.email,
      'errorCode': 'E1001'
    ]]" />
  </set-variable>
</raise-error>
  
```

This raises a **VALIDATION:INVALID\_INPUT** error with custom details.



## REAL-WORLD EXAMPLE

In an order processing flow, if the order amount is less than 1, we raise a **BUSINESS** error.



## BEST PRACTICES

- Use meaningful error types and descriptions.
- Include relevant attributes for better troubleshooting.
- Maintain consistency in error types across APIs.
- Raise error as early as possible when validation fails.
- Log the error before raising if needed.



## INTERVIEW TIPS

- Raise Error is used to manually throw an error.
- It helps trigger the configured error handling.
- Typically used for business validation failures.
- It contains error type, description and optional attributes.
- Handled by **On Error Continue**, **On Error Propagate** or **Try Scope**.



## COMMON MISTAKES

- Raising error without proper error type.
- Not providing useful error description.
- Raising error too late in the flow.
- Not handling the raised error properly.
- Hardcoding sensitive data in error attributes.



## KEY TAKEAWAY

Raise Error empowers you to **enforce business** rules and take **control** of error handling by throwing **meaningful** and **structured** errors in your Mule flows.



Use **Raise Error** to communicate **problems** clearly and let the **right** error handler take **appropriate** action.





## 6.7

## TRY SCOPE

MODULE 6  
ERROR HANDLING

MuleSoft

Try Scope is used to **group** a set of processors that **might** throw an error.

It provides a **dedicated error handling** mechanism inside the scope **without** affecting the main flow.

## KEY POINTS

- Captures errors that occur within a specific block of processors.
- Provides **isolated** error handling.
- Prevents the error from terminating the **main flow**.
- Supports **On Error Continue** and **On Error Propagate**.
- Useful for **non-critical** operations.
- Allows **custom error handling** at a granular level.
- Can be **nested** inside other scopes.

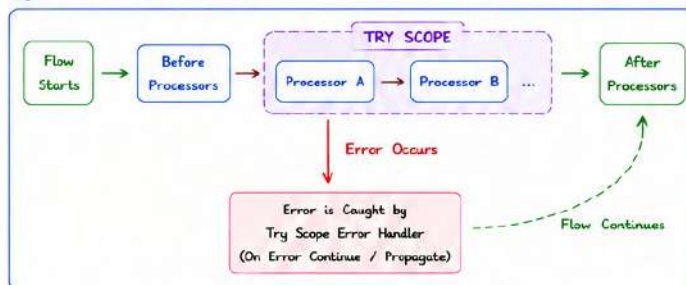


## WHEN TO USE?

- When only a **portion** of the flow may fail.
- To handle errors **locally** without affecting the rest of the flow.
- For **optional** or **non-critical** operations.
- When you want **custom error response** for a specific block.
- To prevent unnecessary **Global Error Handler** triggers.



## HOW IT WORKS?



## TYPES OF TRY SCOPE

## 1 On Error Continue (Default)

- ✓ Handles the error inside the Try Scope.
- ✓ Flow continues with next processor after the Try Scope.
- ✓ Best for **non-critical** operations.

## 2 On Error Propagate

- ✓ Handles the error inside the Try Scope.
- ✓ Error is re-thrown to the next error handler.
- ✓ Best when higher-level handling is needed.



## SYNTAX IN MULE.XML

```
<try>
  <!-- Processors that might throw an error -->
  <http:request config-ref="HTTP_Request_config" path="/data"/>
    <transform doc:name="Transform Data">
      <ee:message>
        <ee:set-payload><![CDATA[%dw 2.0
          output application/json
          ---
          payload
        ]]></ee:set-payload>
      </ee:message>
    </transform>
    <error-handler>
      <on-error-continue type="ANY">
        <logger level="ERROR" message="Error in Try Scope: #[error.description]"/>
        <set-payload value="{['status': 'partial_success', 'message': 'Some data failed']}"
          mediaType="application/json"/>
      </on-error-continue>
    </error-handler>
  </try>
```

Processors inside the Try Scope are protected and errors are handled locally.



## REAL-WORLD EXAMPLE

In an order flow, getting a customer's loyalty points may fail. We handle it inside Try Scope so that the order creation still succeeds.



- ✓ If API succeeds → actual points are used.
- ✓ If API fails → default points (0) are used.
- ✓ Flow continues and customer is not impacted.



## BEST PRACTICES

- ✓ Use Try Scope for non-critical operations only.
- ✓ Keep the Try Scope small and focused.
- ✓ Provide meaningful error messages.
- ✓ Log errors for monitoring and troubleshooting.
- ✓ Return safe default values when possible.



## INTERVIEW TIPS

- Try Scope provides local error handling.
- It prevents errors from affecting the main flow.
- Default behavior is On Error Continue.
- Use On Error Propagate when you want the error to bubble up.
- It is similar to try-catch block in programming.



## COMMON MISTAKES

- ✗ Using Try Scope for critical operations.
- ✗ Ignoring error details.
- ✗ Not returning a proper response.
- ✗ Making Try Scope too large.
- ✗ Not logging errors.



## KEY TAKEAWAY

Try Scope gives you **fine-grained** control over **error** handling. It **isolates** failures, prevents unwanted flow termination and improves **resilience** of your Mule applications.



Use Try Scope to **protect** a specific block, not the entire flow.

Handle locally, **continue gracefully** and keep your integrations **resilient**!



## 6.8

## RETRY STRATEGY

MODULE 6  
ERROR HANDLING

MuleSoft®

Retry Strategy is used to **automatically retry** a failed operation a **specified number of times** with a **defined delay**. It helps handle temporary issues like network glitches, service unavailability, or timeouts.

## KEY POINTS

- Automatically **retries** a failing operation.
- Helps in handling **temporary** and intermittent failures.
- **Configurable**: max attempts, delay, and backoff.
- Supports **exponential backoff** to avoid overloading systems.
- Prevents immediate failure and improves **resilience**.
- Works with **On Error Continue** and **On Error Propagate**.
- Useful for **idempotent** operations.

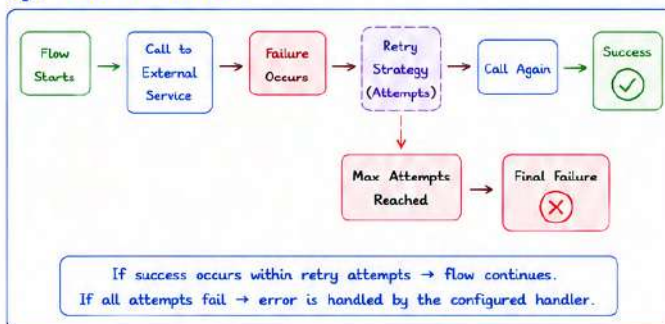


## WHEN TO USE?

- When failures are likely to be **temporary**.
- Network **timeouts** or connection issues.
- Third-party service **unavailability**.
- **Rate limit** exceeded scenarios.
- Database **deadlocks** or locks.
- When you want to reduce **false failures**.
- To improve integration **reliability**.



## HOW IT WORKS?



## RETRY STRATEGY PARAMETERS

| Parameter                | Description                                                     |
|--------------------------|-----------------------------------------------------------------|
| <b>maxAttempts</b>       | Maximum number of retry attempts (including the first attempt). |
| <b>delay</b>             | Time to wait before the next retry (in milliseconds).           |
| <b>backoffMultiplier</b> | Multiplier applied to delay after each failed attempt.          |
| <b>maxDelay</b>          | Maximum delay between retries.                                  |
| <b>whenSuccessful</b>    | Determines if the result is returned when any attempt succeeds. |



## SYNTAX IN MULE.XML

```

<retry-policy maxAttempts="3"
  delay="2000"
  backoffMultiplier="2.0"
  maxDelay="10000">
  <until-successful whenSuccessful="processAll">
    <try>
      <!-- Operation that may fail -->
      <http:request config-ref="HTTP_Request_config"
        path="/api/data" method="GET" />
    </try>
    <error-handler>
      <on-error-continue type="ANY">
        <logger level="WARN" message="Retry attempt failed:
          #[error.description]" />
      </on-error-continue>
    </error-handler>
  </until-successful>
</retry-policy>

```

This will retry up to 3 times with an initial delay of 2 sec and exponential backoff.



## REAL-WORLD EXAMPLE

An eCommerce application calls a payment gateway. Sometimes the gateway is slow or unavailable. We use Retry Strategy to automatically retry the payment request.



- ✓ Improves success rate for temporary issues.
- ✓ Prevents customers from facing false order failures.
- ✓ Enhances overall user experience.



## BEST PRACTICES

- ✓ Use Retry only for temporary and recoverable errors.
- ✓ Keep maxAttempts reasonable (avoid very high values).
- ✓ Use exponential backoff to avoid overwhelming services.
- ✓ Log each attempt with proper details.
- ✓ Combine with meaningful error handling.



## INTERVIEW TIPS

- Retry Policy automatically retries a failing operation.
- Key parameters: **maxAttempts**, **delay**, **backoffMultiplier**, **maxDelay**.
- Exponential backoff increases the delay after each failure.
- Use only for temporary failures.
- Ensure operations are idempotent.



## COMMON MISTAKES

- ✗ Using Retry for permanent errors.
- ✗ Setting too high **maxAttempts**.
- ✗ Not using **backoff** (can overload systems).
- ✗ Ignoring idempotency.
- ✗ Not adding proper logging and alerts.



## KEY TAKEAWAY

Retry Strategy gives your integrations the power to recover from **temporary** failures automatically. Use it wisely to improve **reliability** without **overloading** systems.



Use Retry Strategy to handle **temporary** failures smartly – retry, **recover** and deliver a **reliable** integration experience!





## 6.9 ERROR MAPPING

### MODULE 6 ERROR HANDLING



MuleSoft

Error Mapping is used to transform a technical error into a **meaningful business error**.

It ensures clients receive **clear** and **user-friendly** error messages instead of technical details.

#### KEY POINTS

- Transforms **technical errors** into meaningful **business errors**.
- Improves the **user experience**.
- Hides **internal system details** from clients.
- Maintains **security** and information hygiene.
- **Consistent** error responses across all integrations.
- Used inside an **error handler**.
- Helps in **standardizing** error responses.

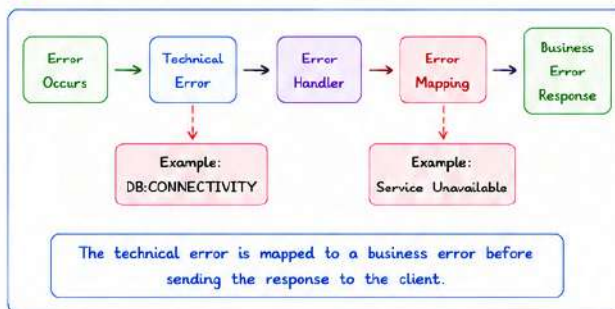


#### WHEN TO USE?

- When exposing **technical errors** to clients.
- When you want to send custom, **user-friendly** messages to clients.
- When you need to **hide internal system** or **infrastructure details**.
- When different technical errors should result in the **same business error**.
- To maintain **consistent** API error contracts.
- For **compliance** and **security** requirements.



#### HOW IT WORKS?



#### ERROR MAPPING STRUCTURE

| Element           | Description                                          |
|-------------------|------------------------------------------------------|
| Source Error      | The technical error type (e.g., DB:CONNECTIVITY).    |
| Target Error      | The business error type (e.g., SERVICE:UNAVAILABLE). |
| Error Type        | Defines the business error category.                 |
| Error Description | A user-friendly message for the client.              |
| Error Attributes  | Additional data (optional) to send with the error.   |



#### SYNTAX IN MULE.XML

```

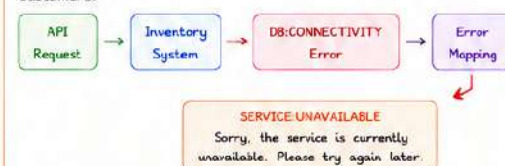
<error-mapping source="DB:CONNECTIVITY" target="SERVICE:UNAVAILABLE">
  <error-type> SERVICE:UNAVAILABLE </error-type>
  <error-description>
    Sorry, the service is currently unavailable.
    Please try again later.
  </error-description>
  <error-attributes>
    <add-attribute key="errorCode" value="SVC-503" />
    <add-attribute key="timestamp" value="#[now()]" />
  </error-attributes>
</error-mapping>
  
```

Maps technical DB connectivity error to a clean business error response.



#### REAL-WORLD EXAMPLE

An eCommerce API calls an inventory system. If the inventory service is down, we don't want to expose technical errors to customers.



- ✓ Client gets a friendly message.
- ✓ Internal details stay hidden.
- ✓ Better reliability and user trust.



#### BEST PRACTICES

- ✓ Create a clear error mapping strategy.
- ✓ Do not expose internal or sensitive information.
- ✓ Use consistent error codes and messages.
- ✓ Log the original technical error for debugging.
- ✓ Keep error responses simple and meaningful.



#### INTERVIEW TIPS

- Error Mapping is for converting technical errors to business errors.
- It improves the user experience and security.
- Used inside the error handler before sending the response.
- It does not fix the error, it only transforms it.
- Always log the original error before mapping.



#### COMMON MISTAKES

- ✗ Exposing technical details to clients.
- ✗ Not having a standard error mapping strategy.
- ✗ Mapping all errors to a single generic message.
- ✗ Not logging the original error.
- ✗ Sending inconsistent error responses.



#### KEY TAKEAWAY

Error Mapping helps you speak your **client's language**. Transform **technical noise** into **meaningful messages** that are **secure, consistent** and **user-friendly**.



Always map **technical errors** to meaningful **business errors**.  
Clear messages build **trust**, reduce **support tickets** and improve **user experience**!



## 6.10

## CUSTOM ERRORS

MODULE 6  
ERROR HANDLING

MuleSoft

Custom Errors are user-defined error types created to represent **business-specific error scenarios**. They help in returning **meaningful, consistent** and **reusable** error responses.

## KEY POINTS

- Created to represent **business or domain-specific** errors.
- Reusable** across multiple flows and applications.
- Provides **meaningful** error types, descriptions and attributes.
- Improves **consistency** in error handling and responses.
- Used in **Error Mapping** and **Error Handlers**.
- Helps **hide technical details** from the end user.
- Makes error management **scalable** and **maintainable**.
- Support **custom attributes** for more context.

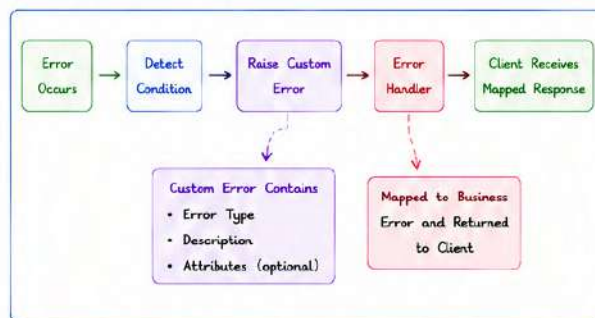


## WHEN TO USE?

- When **default Mule errors** do not fit your needs.
- When you want to represent **business rule violations**.
- When building reusable and **consistent APIs**.
- When you need to send **user-friendly** error messages.
- When you want **better classification** of errors.
- When you need **additional attributes** with errors.
- For **compliance, audit** and reporting needs.



## HOW IT WORKS?



## CUSTOM ERROR STRUCTURE

| Field                 | Description                                                           |
|-----------------------|-----------------------------------------------------------------------|
| Namespace             | Unique namespace for the custom error.                                |
| Identifier            | Unique name of the custom error.                                      |
| Description           | Human readable description of the error.                              |
| Type                  | Defines the category (e.g., BUSINESS, VALIDATION).                    |
| Attributes (Optional) | Additional key-value pairs for more context (e.g., field, errorCode). |



## SYNTAX IN MULE.XML

```

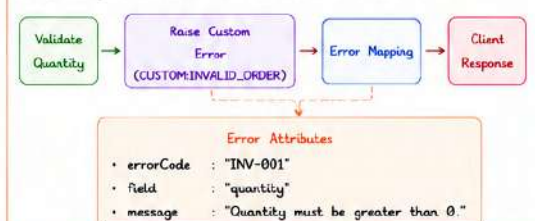
<error-types>
  <error-type namespace="http://acme.com/errors"
    name="CUSTOM.INVALID_ORDER">
    <type>BUSINESS</type>
    <description>Order is invalid due to business rule violation.</description>
    <attributes>
      <attribute key="errorCode" type="STRING"/>
      <attribute key="field" type="STRING"/>
    </attributes>
  </error-type>
</error-types>
  
```

This defines a custom error with type, description and attributes.



## REAL-WORLD EXAMPLE

An eCommerce API validates an order quantity. If quantity is less than 1, we raise a custom error.



## BEST PRACTICES

- ✓ Create clear and meaningful error names.
- ✓ Use proper namespaces to avoid conflicts.
- ✓ Provide useful descriptions.
- ✓ Include relevant attributes for debugging.
- ✓ Reuse custom errors instead of creating new ones.
- ✓ Map custom errors to business errors before sending to clients.



## INTERVIEW TIPS

- Custom Errors are user-defined error types for business scenarios.
- They provide consistency and reusability.
- They are defined in error-types section.
- They can carry attributes for more context.
- They are used with Raise Error component.



## COMMON MISTAKES

- ✗ Using generic errors instead of custom errors.
- ✗ Not providing a proper namespace.
- ✗ Missing meaningful descriptions or attributes.
- ✗ Exposing technical error details to clients.
- ✗ Creating duplicate custom errors.



## KEY TAKEAWAY

Custom Errors allow you to model **business-specific error scenarios** and return **clear, consistent** and **user-friendly** responses. They improve maintainability, reusability and user experience.



Use Custom Errors to speak your **client's language**, not the system's. Meaningful errors build **trust** and **better experiences**!





## 7.1

## HTTP CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

HTTP Connector is used to communicate with external systems over **HTTP** or **HTTPS** protocol. It supports **REST**, **SOAP** and other HTTP-based services.

## KEY POINTS

- Used to send HTTP requests and receive responses.
- Supports all HTTP methods: **GET**, **POST**, **PUT**, **PATCH**, **DELETE**.
- Supports **query parameters**, **headers**, and **payload**.
- Handles **authentication** (Basic, OAuth 2.0, etc.).
- Supports **TLS/SSL** for secure communication.
- Can consume **REST APIs** and **SOAP** web services.
- Supports connection pooling for performance.
- Returns response status, headers, and payload.
- Configurable **timeouts**, **reconnection** and **retry policies**.
- Widely used to integrate with external systems and APIs.

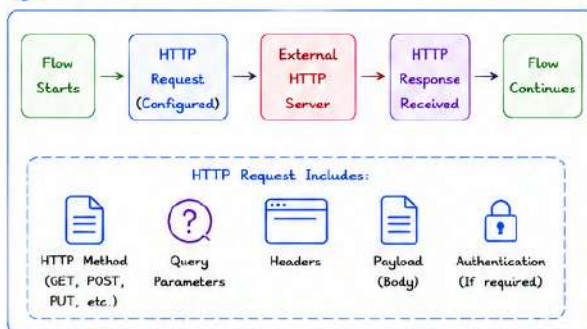


## WHEN TO USE?

- When you need to call **REST** or **SOAP** services.
- When integrating with third-party systems exposed over **HTTP/HTTPS**.
- When you need fine control over request methods, **headers** and **parameters**.
- When you need to handle **authentication** and **secure communication**.
- When consuming public or private APIs.
- When you need to send or receive data in **JSON**, **XML**, or other formats.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                                             |
|-------------------------|-------------------------------------------------------------------------|
| Connector Configuration | Base path, host, port, protocol, timeouts, connection and TLS settings. |
| Request Path            | Relative path of the resource.                                          |
| HTTP Method             | GET, POST, PUT, PATCH, DELETE, etc.                                     |
| Headers                 | Key-value pairs sent with the request.                                  |
| Query Parameters        | Parameters appended to the URL.                                         |
| Body (Payload)          | Request body for POST, PUT, PATCH, etc.                                 |
| Response                | Status code, headers and payload.                                       |
| Error Mapping           | Map HTTP errors to custom errors.                                       |



## SYNTAX IN MULE.XML

```

<http:request config-ref="HTTP_Config" method="GET" path="/api/customers">
  <http:query-params>
    <http:param-param key="page" value="1"/>
    <http:param-param key="size" value="10"/>
  </http:query-params>
  <http:headers>
    <http:header key="Accept" value="application/json"/>
  </http:headers>
</http:request>
  
```

This sends a GET request to /api/customers?page=1 &size=10 with Accept header.



## REAL-WORLD EXAMPLE

An order management system calls a payment gateway API to process payments.



## Request:

```

• Method : POST
• Path : /pay
• Body : { "amount": 100, "currency": "USD", "card": "****" }
  
```

- ✓ Sends payment details securely to the payment gateway.
- ✓ Receives success or failure response.
- ✓ Updates order status based on the response.



## BEST PRACTICES

- ✓ Use secure protocols (HTTPS).
- ✓ Use connection pooling for better performance.
- ✓ Set appropriate timeouts.
- ✓ Use reusable configurations.
- ✓ Handle errors using proper error mapping.
- ✓ Log request and response wisely (avoid sensitive data).
- ✓ Use query params instead of hardcoded values.



## INTERVIEW TIPS

- HTTP Connector is used for external communication over HTTP/HTTPS.
- It supports all HTTP methods and custom headers.
- It can consume REST and SOAP services.
- It supports authentication, TLS, timeouts and connection pooling.
- It returns response status, headers and payload.
- Errors are handled using error mapping and retry strategies.



## COMMON MISTAKES

- ✗ Using HTTP instead of HTTPS.
- ✗ Hardcoding endpoints and parameters.
- ✗ Not setting timeouts and retries.
- ✗ Not handling error responses.
- ✗ Logging sensitive data in clear text.
- ✗ Ignoring status codes and headers.
- ✗ Not using connection pooling.



## KEY TAKEAWAY

HTTP Connector is the most commonly used connector in MuleSoft for integrating with external services. It provides **flexibility**, **security** and **performance** when calling **REST** or **SOAP** APIs over **HTTP/HTTPS**.



Use HTTP Connector to **integrate**, **communicate** and collaborate with external systems **securely** and **efficiently**!





## 7.2

## DATABASE CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

Database Connector is used to connect to **relational databases** and perform **CRUD operations**.

It allows you to execute **SQL queries**, **stored procedures** and manage **transactions**.

## KEY POINTS

- Used to connect to **relational databases** (e.g., MySQL, Oracle, SQL Server, PostgreSQL, DB2, etc.).
- Supports operations: **SELECT, INSERT, UPDATE, DELETE, CALL**.
- Execute **SQL queries** and **stored procedures**.
- Supports **input and output parameters**.
- Supports **transactions** (begin, commit, rollback).
- Supports **connection pooling** for better performance.
- Parameterized queries help prevent **SQL Injection**.
- Returns **result sets**, **update counts** and **output parameters**.
- Works with **on-premises** and **cloud databases**.
- Can be used within **global transactions**.
- Supports **bulk operations** for improved efficiency.
- Used widely for **data integration**, **reporting** and **data management**.

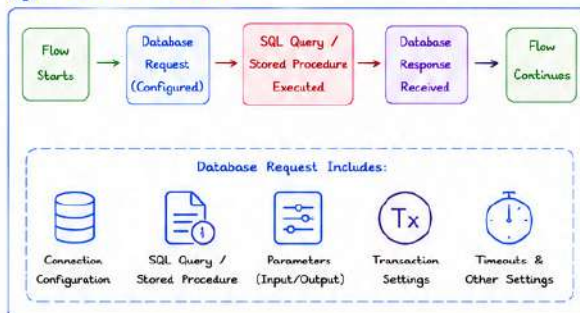


## WHEN TO USE?

- When your application needs to read from or write to a **relational database**.
- When you need to execute complex **SQL queries** or **stored procedures**.
- When performing **CRUD operations** on database tables.
- When you need to work with **transactions**.
- When you need to fetch **large datasets** or update multiple rows.
- When integrating Mule applications with **enterprise databases**.
- When using database data in **APIs** or business processes.
- When you need **secure** and **reliable** database access.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                        | Description                                            |
|------------------------------|--------------------------------------------------------|
| Connector Configuration      | Database connection details, pooling and TLS settings. |
| Operation                    | SELECT, INSERT, UPDATE, DELETE, CALL                   |
| SQL Query / Stored Procedure | SQL statement or stored procedure to execute.          |
| Input Parameters             | Values for query parameters.                           |
| Output Parameters            | Variables to capture output params.                    |
| Result Set                   | Result mapping for query responses.                    |
| Transaction Action           | None / Begin / Commit / Rollback                       |
| Fetch Size                   | Number of rows to fetch per request.                   |
| Timeout                      | Time to wait for database response.                    |
| Validate Connection          | Validate connection at startup.                        |



## SYNTAX IN MULE.XML

```

<db:select config-ref="DB_Config" doc:name="Get Customer"
  target="customers">
  <db:sql><![CDATA[
    SELECT id, name, email, status
    FROM customers
    WHERE status = :status
  ]]></db:sql>
  <db:input-parameters>
    <db:input-parameter key="status" value="#[vars.status]"/>
  </db:input-parameters>
</db:select>
  
```

Executes a SQL query with a parameter and returns a result set.



## REAL-WORLD EXAMPLE

An eCommerce application retrieves order details from the database when a customer requests order history.



SQL Query:

```

SELECT order_id, order_date, amount, status
FROM orders
WHERE customer_id = :customerId
  
```

- Fetches orders for the given customer.
- Returns order history as JSON response.
- Uses parameterized query for security.



## BEST PRACTICES

- Use connection pooling for better performance.
- Always use parameterized queries.
- Validate connections at startup.
- Use appropriate fetch size for large result sets.
- Use transactions for multiple DB operations.
- Handle exceptions and roll back on failure.



## INTERVIEW TIPS

- Database Connector connects Mule applications to relational databases.
- It supports CRUD operations and stored procedures.
- It uses connection pooling for efficiency.
- Transactions ensure data consistency.
- Parameterized queries prevent SQL injection.
- It returns result sets, update counts and output parameters.



## COMMON MISTAKES

- Hardcoding SQL queries and values.
- Not using parameterized queries.
- Not handling database timeouts.
- Forgetting to close transactions.
- Fetching too many rows at once.
- Ignoring connection pooling.
- Not validating connections.



## KEY TAKEAWAY

Database Connector helps you interact with relational databases securely and efficiently in MuleSoft.

It supports **SQL queries**, **stored procedures**, **transactions** and **connection pooling** for reliable data operations.



Use Database Connector to **access**, **manage** and integrate your business data with **reliability**, **performance** and **security**!





## 7.3

## SALESFORCE CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

Salesforce Connector is used to integrate Mule applications with **Salesforce CRM**.

It allows you to perform **CRUD** operations, query data (**SOQL**), manage **sObjects** and handle **Bulk APIs**.

## KEY POINTS

- Used to connect to **Salesforce CRM**.
- Supports operations: **CREATE, READ, UPDATE, DELETE, UPSERT, QUERY, SEARCH**.
- Supports **SOQL queries** to fetch data.
- Supports **Bulk API** for large data operations.
- Handles authentication using **OAuth 2.0/JWT/Username-Password**.
- Supports **sObjects** like **Account, Contact, Opportunity, Lead**, etc.
- Supports **batch operations** for large datasets.
- Can perform **metadata** operations.
- Provides automatic pagination for query results.
- Returns Salesforce response in **JSON** format.
- Widely used in **CRM integrations** and cloud applications.

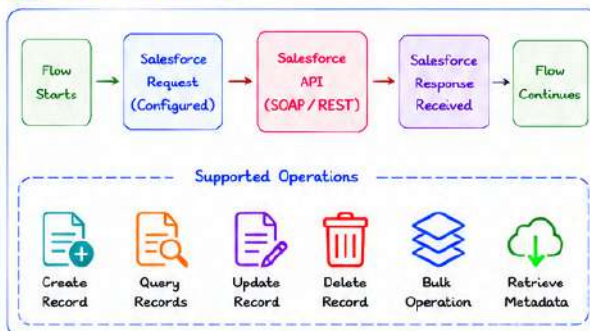


## WHEN TO USE?

- When integrating Mule applications with **Salesforce**.
- When you need to perform **CRUD** on Salesforce objects.
- When you need to fetch data using **SOQL queries**.
- When **synchronizing data** between systems and Salesforce.
- When you need to use **Bulk API** for large data volumes.
- When you need to manage **Leads, Accounts, Opportunities, Contacts**, etc.
- When **validating or updating** Salesforce records.
- When you need to automate **business processes** in Salesforce.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                         |
|-------------------------|-----------------------------------------------------|
| Connector Configuration | Salesforce connection, host, and security settings. |
| Operation               | CREATE, QUERY, UPDATE, DELETE, UPSERT, etc.         |
| Object (sObject)        | Salesforce object (e.g., Account, Contact, Lead).   |
| SOQL Query              | Query string to fetch data.                         |
| Input Payload           | Data to be sent (JSON) for create/update.           |
| Output                  | Salesforce API response in JSON.                    |
| Batch Size              | Number of records to process in bulk.               |
| Timeout                 | Time to wait for Salesforce response.               |



## SYNTAX IN MULE.XML

```

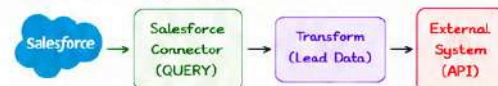
<salesforce:query config-ref="SF_Config">
  <salesforce:salesforce:query>
    <salesforce:query-string>
      SELECT Id, Name, Email FROM Contact WHERE Status = 'Active'
    </salesforce:query-string>
  </salesforce:salesforce:query>
</salesforce:query>
  
```

This sends a **SOQL query** to Salesforce and returns the result set.



## REAL-WORLD EXAMPLE

An organization wants to sync new Leads from Salesforce to an external analytics system.



## SOQL Query Example:

```

SELECT Id, Name, Email, CreatedDate
FROM Lead
WHERE CreatedDate = LAST_N_DAYS:1
  
```

- Fetches new leads created in last 1 day.
- Transforms the data as required.
- Sends data to external analytics system.



## BEST PRACTICES

- Use connection pooling for better performance.
- Always use **SOQL** instead of retrieving all records.
- Use pagination for large result sets.
- Use **Bulk API** for large data operations.
- Handle Salesforce API limits and exceptions.
- Log **SOQL queries** for debugging.



## INTERVIEW TIPS

- Salesforce Connector supports **REST and SOAP APIs**.
- It supports **CRUD operations and SOQL queries**.
- It uses **OAuth 2.0/JWT** for authentication.
- Bulk API** is used for large data operations.
- It returns response in **JSON**.



## COMMON MISTAKES

- Querying too many records without pagination.
- Not handling Salesforce API rate limits.
- Using **Bulk API** for small datasets.
- Hardcoding credentials instead of using secure properties.
- Ignoring null fields in responses.



## KEY TAKEAWAY

Salesforce Connector helps you seamlessly integrate Mule applications with Salesforce. Use **SOQL**, **Bulk API** and proper authentication to build efficient, reliable and scalable CRM integrations.



Use **Salesforce Connector** to connect, **query**, **update** and manage data in Salesforce efficiently and **securely**!





## 7.4

## SAP CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

SAP Connector is used to integrate Mule applications with SAP systems such as **SAP ECC**, **SAP S/4HANA**, and **SAP BTP**. It enables you to perform **RFC**, **IDoc**, **BAPI**, and **OData** operations securely and efficiently.

## KEY POINTS

- Used to connect Mule applications with **SAP** systems.
- Supports **SAP ECC**, **S/4HANA**, and **SAP BTP** (Cloud).
- Supports **RFC**, **IDoc**, **BAPI**, and **OData** protocols.
- Supports **synchronous** and **asynchronous** communication.
- Handles **connection pooling** for better performance.
- Supports transactional RFC (**tRFC**) and **qRFC**.
- Supports secure communication via **SNC**, **HTTPS**, and **Basic Auth**.
- Supports **pagination**, **filtering**, **sorting**, and **field selection**.
- Supports **IDoc** and **BAPI** operations for data exchange.
- Supports **OData** services exposed by **SAP** systems.
- Supports **error handling** and **detailed SAP** fault responses.
- Widely used for **enterprise integrations** with **SAP** landscapes.

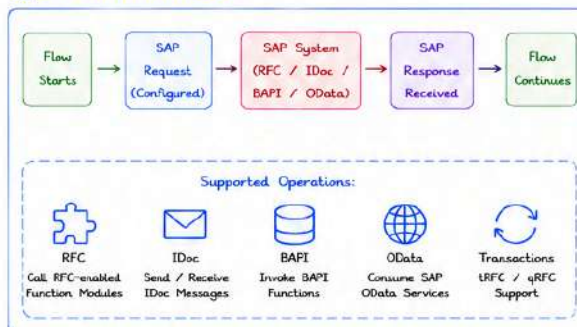


## WHEN TO USE?

- When your application needs to connect to **SAP** systems.
- When you need to execute **RFC-enabled** function modules.
- When you need to send or receive **IDoc** messages.
- When you need to call **BAPIs** for business operations.
- When you need to consume **SAP OData** services.
- When you need transactional updates with **tRFC/qRFC**.
- When you need to fetch or update master or transactional data from **SAP**.
- When integrating Mule applications with **SAP S/4HANA**.
- When you need **secure** and **reliable** communication.
- When building end-to-end **enterprise** process integrations.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                    | Description                                          |
|--------------------------|------------------------------------------------------|
| Connection Configuration | SAP system connection details and authentication.    |
| SAP System               | Application server host, system number, client.      |
| Client                   | SAP client (e.g., 100, 200).                         |
| User / Password          | Credentials for SAP authentication.                  |
| Language                 | SAP login language (e.g., EN, DE).                   |
| Pool Configuration       | Max pool size, idle timeout, validation query.       |
| Operation                | RFC, IDoc, BAPI or OData.                            |
| Function / Service       | RFC function, BAPI name, IDoc type or OData service. |
| Parameters / Payload     | Input parameters or request payload.                 |
| Timeout                  | Time to wait for SAP response.                       |
| TLS / Security           | SNC, HTTPS, or other security settings.              |



## SYNTAX IN MULE.XML

```
<saprequest config-ref="SAP_Config" docname="Get Material Details">
  <sapoperation>BAPI</sapoperation>
  <sapbapiname>BAPI_MATERIAL_GET_DETAIL</sapbapiname>
  <sapparameters>
    <sapparameter name="MATERIAL" value="#[vars.materialNo]"/>
  </sapparameters>
</saprequest>
```

Calls a **BAPI** function to get material details from **SAP**.



You can also use `<sapoperation>RFC</sapoperation>`, `<sapoperation>IDOC</sapoperation>` or `<sapoperation>ODATA</sapoperation>` based on your requirement.



## REAL-WORLD EXAMPLE

An order management system needs to create a sales order in **SAP**.



## BAPI Request Parameters:

```
• SALES_ORG      : "1000"
• DISTRIBUTION_CHAN : "10"
• CUSTOMER       : "CUST123"
• ORDER_ITEMS    : [ { Material: "M-100", Qty: 10 } ]
```

- Creates sales order in **SAP**.
- Returns **SAP** document number as response.
- Ensures data consistency and reliability.



## BEST PRACTICES

- Use connection pooling for better performance.
- Reuse connections.
- Use appropriate operation type (RFC, IDoc, BAPI, OData).
- Validate connections at startup.
- Use pagination for large result sets.
- Enable secure communication (SNC/HTTPS).
- Handle **SAP** errors and roll back when needed.



## INTERVIEW TIPS

- SAP** Connector supports **RFC**, **IDoc**, **BAPI** and **OData**.
- RFC** is for real-time calls to function modules.
- IDoc** is for asynchronous message exchange.
- BAPI** is for business object operations.
- OData** is for RESTful access to **SAP** data.
- Supports **tRFC** and **qRFC** for reliable transactional updates.



## COMMON MISTAKES

- Using wrong operation type for the requirement.
- Not handling **SAP** timeouts.
- Ignoring **LUW** (Logical Unit of Work) in **tRFC/qRFC**.
- Incorrect function module or **BAPI** name.
- Missing mandatory parameters.
- Not enabling connection pooling.



## KEY TAKEAWAY

**SAP** Connector enables secure and high-performance integration with **SAP** systems. It supports **RFC**, **IDoc**, **BAPI** and **OData** operations to exchange data reliably across enterprise applications.



Use **SAP** Connector to **integrate** your Mule applications with **SAP** systems seamlessly, securely and efficiently!





## 7.5

## FILE CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

File Connector is used to read from and write to files on the local file system or on remote servers using **FTP**, **FTPS**, **SFTP**.

## KEY POINTS

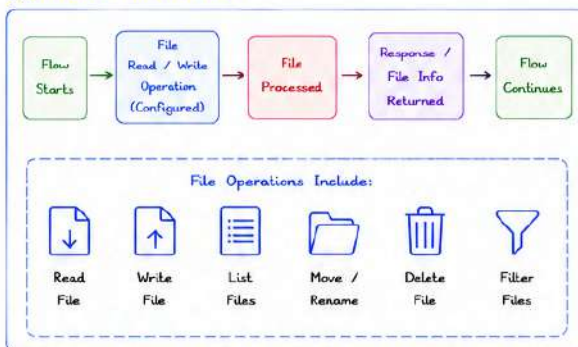
- Used to **read** from and **write** to files.
- Supports **local file system** and remote file systems (**FTP**, **FTPS**, **SFTP**).
- Supports **read**, **write**, **create**, **delete**, **move**, and **list** operations.
- Supports **binary** and **text** file handling.
- Supports **recursive directory** operations.
- Supports **streaming** large files for better performance.
- Supports file **filters** (name, type, regex, age, size, etc.).
- Supports secure file transfer using **FTPS** and **SFTP**.
- Supports file **locking** to prevent concurrent access issues.
- Configurable **polling** for periodic file operations.
- Used widely for **batch processing**, **reports**, and **file integration**.



## WHEN TO USE?

- When you need to **read** files from local or remote locations.
- When you need to **write** files to local or remote locations.
- When processing **flat files**, **reports**, or **data extracts**.
- When you need to **move**, **delete**, or **rename** files.
- When you need to **list** files in a directory.
- When handling large files using **streaming**.
- When integrating with external systems via **file exchange**.
- When automating file transfers using **FTP**, **FTPS** or **SFTP**.
- When you need to **filter** files based on name, type, size or last modified date.
- When implementing **batch** or **scheduled** file processes.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                       |
|-------------------------|---------------------------------------------------|
| Connector Configuration | File system or remote connection details.         |
| Connection Type         | LOCAL, FTP, FTPS, or SFTP.                        |
| Host                    | Remote host (for FTP/FTPS/SFTP).                  |
| Port                    | Port number (e.g., 21 for FTP, 22 for SFTP).      |
| Username                | User name for authentication (remote).            |
| Password / Private Key  | Password or private key for SFTP authentication.  |
| Base Directory / Path   | Base path or directory to operate on.             |
| Operation               | READ, WRITE, LIST, DELETE, MOVE, etc.             |
| File Name / Pattern     | File name or pattern (wildcards/regex supported). |
| File Filter             | Filter by name, type, size, age, etc.             |
| Streaming               | Enable streaming for large files.                 |
| Polling Frequency       | Frequency for file polling (if using listener).   |
| Locking Strategy        | File locking to avoid concurrent access.          |

## SYNTAX IN MULE.XML

```

<file:read config-ref="File_Config" path="/input/data.csv"
  outputMimeType="text/csv">
  <file:success-strategy>
    <file:move path="/processed/archive" />
  </file:success-strategy>
  <file:error-strategy>
    <file:move path="/error" />
  </file:error-strategy>
</file:read>

```

Reads a file and moves it to archive on success, or to error folder on failure.

Similar tags: <file:write>, <file:list>, <file:delete>, <file:move>

## REAL-WORLD EXAMPLE

A daily batch job reads order files from an FTP server, processes them and moves the files to archive or error folders.



Read File from FTP:

- Path : /inbound/orders/\*.csv
- Operation : READ
- Streaming : True
- Success : Move to /archive
- Error : Move to /error

- Ensures reliable file processing.
- Automatically archives processed files.
- Handles errors by moving files to error folder.

## BEST PRACTICES

- Use streaming for large files.
- Use appropriate file filters.
- Handle file exceptions and move to error folder.
- Use polling with optimal frequency.
- Secure connections (FTPS/SFTP) for remote files.
- Use file locking to prevent concurrent access.
- Keep directory structure organized.

## INTERVIEW TIPS

- File Connector supports operations on local and remote file systems.
- It supports FTP, FTPS, SFTP for remote file access.
- Common operations include read, write, list, delete, move.
- Use streaming for handling large files efficiently.
- Use filters to select specific files.
- Handle errors by moving files to error directory.

## COMMON MISTAKES

- Not handling file exceptions.
- Using polling frequency too low or too high.
- Not enabling streaming for large files.
- Hardcoding file paths.
- Not using secure protocols for remote file transfer.
- Ignoring file locking.

## KEY TAKEAWAY

File Connector in MuleSoft enables seamless file operations on local and remote systems. It supports reading, writing, and managing files securely and efficiently using **FTP**, **FTPS**, and **SFTP**. It is ideal for batch processing, file exchange, and data integration.



Use File Connector to automate file operations **securely**, **efficiently** and **reliably** in your integrations!





## 7.6

## SFTP CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

SFTP Connector is used to **securely** transfer files to and from remote servers using the **SSH File Transfer Protocol (SFTP)**. It enables secure file operations over an **encrypted channel**.

## KEY POINTS

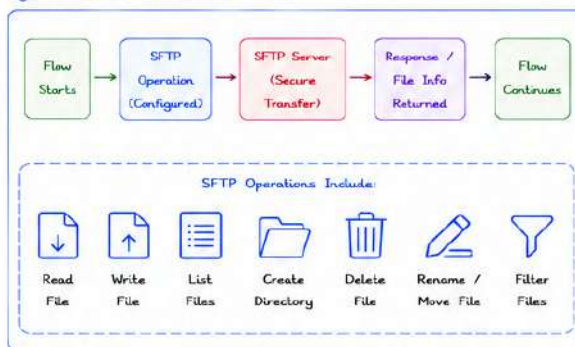
- Used to perform **secure** file transfers over **SSH** using **SFTP**.
- Supports **read, write, create, delete, rename, move** and **list** operations.
- Supports recursive directory operations.
- Supports file filters (name, regex, size, age, type, etc.).
- Supports binary and text file handling.
- Supports streaming large files for better performance.
- Supports **secure authentication** using username/password or private key.
- Supports **connection pooling** and configurable timeouts.
- Supports **error handling** and detailed error responses.
- Used widely for **secure file exchange** and **data integration**.



## WHEN TO USE?

- When you need to transfer files **securely** to/from a remote server.
- When your business partners provide files via **SFTP**.
- When you need to **read, write** or archive files on an **SFTP** server.
- When handling large files using **streaming**.
- When you need to **automate** file exchanges.
- When integrating with systems that expose **SFTP** endpoints.
- When you need **secure authentication** using private keys.
- When you need to process **flat files, reports** or **data extracts**.
- When you need to **schedule** file transfers.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                        |
|-------------------------|----------------------------------------------------|
| Connector Configuration | SFTP server connection details and authentication. |
| Host                    | SFTP server host.                                  |
| Port                    | Port number (default: 22).                         |
| Username                | User name for authentication.                      |
| Password / Private Key  | Password or private key for authentication.        |
| Private Key Passphrase  | Passphrase for encrypted private key (if any).     |
| Remote Directory        | Base path on the SFTP server.                      |
| Timeout                 | Time to wait for SFTP response.                    |
| Operation               | READ, WRITE, LIST, DELETE, MOVE, etc.              |
| File Name / Pattern     | File name or pattern (wildcards/regex supported).  |
| File Filter             | Filter by name, type, size, age, etc.              |
| Streaming               | Enable streaming for large files.                  |
| Polling Frequency       | Frequency for file polling (if using listener).    |
| Secure Protocol         | SFTP over SSH (Secure by default).                 |

## SYNTAX IN MULE.XML

```

<sftp:read config-ref="SFTP_Config" path="/unbound/orders/*.csv"
  outputMimeType="text/csv">
  <sftp:success-strategy>
    <sftp:move path="/archive/success/" />
  </sftp:success-strategy>
  <sftp:error-strategy>
    <sftp:move path="/archive/error/" />
  </sftp:error-strategy>
</sftp:read>

```

Reads CSV files from SFTP and moves them to success or error folders.

Other operations: <sftp:write>, <sftp:list>, <sftp:create-directory>, <sftp:delete>, <sftp:move>, <sftp:rename>

## REAL-WORLD EXAMPLE

A system receives daily sales files from partners via SFTP, processes them and archives the files.



## Read File from SFTP:

- Path: /unbound/sales/\*.csv
- Operation: READ
- Streaming: True
- Success: Move to /archive/success
- Error: Move to /archive/error

- Securely downloads files from the partner.
- Processes data and generates reports.
- Archives files in success or error folders.
- Ensures reliability and data consistency.

## BEST PRACTICES

- Use connection pooling for better performance.
- Use streaming for large files.
- Use file filters to process only required files.
- Secure credentials using Mule Secure Configuration Properties.
- Use private key authentication instead of passwords.
- Handle errors and move files to error directory.
- Keep remote directories clean and organized.

## INTERVIEW TIPS

- SFTP Connector uses SSH File Transfer Protocol.
- It provides secure file transfer over an encrypted channel.
- Supports read, write, list, delete, move, rename, create directory.
- Streaming is used for large files.
- Connection pooling improves performance.
- Used for secure file exchange with partners and systems.

## COMMON MISTAKES

- Using plain FTP instead of SFTP.
- Hardcoding credentials in flows.
- Not using streaming for large files.
- Not handling file exceptions.
- Incorrect remote directory paths.
- Not using file filters, leading to unnecessary processing.
- Ignoring file locking or concurrency.

## KEY TAKEAWAY

SFTP Connector enables secure and reliable file transfer operations in MuleSoft. It supports **read, write, list, delete, move** and other file operations using SFTP. It is ideal for **secure file exchange, automation** and **enterprise integrations**.



Use SFTP Connector to **securely** transfer files, automate exchanges and integrate your systems with **reliability** and **performance**!





## 7.7

## FTP CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft®

FTP Connector is used to transfer files to and from remote servers using the File Transfer Protocol (FTP). It enables file operations over an **unencrypted** connection.

## KEY POINTS

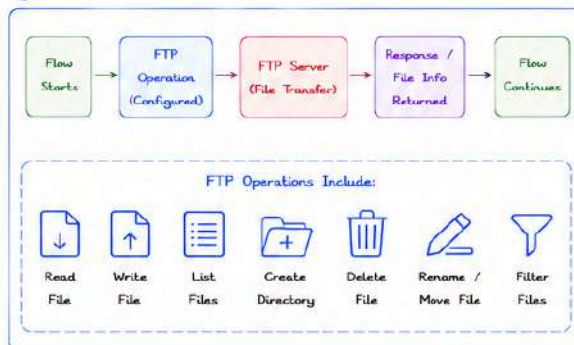
- Used to perform **file transfers** over **FTP**.
- Supports **read**, **write**, **create**, **delete**, **rename**, **move** and **list** operations.
- Supports recursive directory operations.
- Supports **file filters** (name, regex, size, age, type, etc.).
- Supports **binary** and **text** file handling.
- Supports **streaming** large files for better performance.
- Supports **connection pooling** and configurable timeouts.
- Supports configurable **passive mode** for firewall compatibility.
- Supports **error handling** and detailed error responses.
- Used widely for **file exchange** and **data integration**.



## WHEN TO USE?

- When you need to **transfer files** to/from an **FTP server**.
- When your business partners provide files via **FTP**.
- When you need to **read**, **write** or archive files on an **FTP server**.
- When handling large files using **streaming**.
- When you need to **automate** file exchanges.
- When integrating with systems that expose **FTP endpoints**.
- When **passive mode** is required due to firewall restrictions.
- When you need to process **flat files**, **reports** or **data extracts**.
- When you need to **schedule** file transfers.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                       |
|-------------------------|---------------------------------------------------|
| Connector Configuration | FTP server connection details and authentication. |
| Host                    | FTP server host.                                  |
| Port                    | Port number (default: 21).                        |
| Username                | User name for authentication.                     |
| Password                | Password for authentication.                      |
| Remote Directory        | Base path on the FTP server.                      |
| Passive Mode            | Enable passive mode (recommended for firewalls).  |
| Timeout                 | Time to wait for FTP response.                    |
| Operation               | READ, WRITE, LIST, DELETE, MOVE, etc.             |
| File Name / Pattern     | File name or pattern (wildcards/regex supported). |
| File Filter             | Filter by name, type, size, age, etc.             |
| Streaming               | Enable streaming for large files.                 |
| Polling Frequency       | Frequency for file polling (if using listener).   |
| FTP Mode                | TRANSFER_MODE (BINARY or ASCII).                  |

## SYNTAX IN MULE.XML

```

<ftp:read config-ref="FTP_Config" path="/inbound/orders/*.csv"
  outputMimeType="text/csv">
  <ftp:success-strategy>
    <ftp:move path="/archive/success/" />
  </ftp:success-strategy>
  <ftp:error-strategy>
    <ftp:move path="/archive/error/" />
  </ftp:error-strategy>
</ftp:read>
  
```

Reads CSV files from FTP and moves them to success or error folders.

Other operations: <ftp:write>, <ftp:list>, <ftp:delete>, <ftp:create-directory>, <ftp:move>, <ftp:rename>

## REAL-WORLD EXAMPLE

A daily batch process downloads report files from an FTP server, processes them and archives the files.



## Read File from FTP:

- Path : /reports/daily/\*.csv
- Operation : READ
- Streaming : True
- Passive Mode : True
- Success : Move to /archive/success
- Error : Move to /archive/error

- Downloads report files from FTP.
- Processes and generates insights.
- Archives files based on success or failure.
- Ensures reliable and automated file exchange.

## BEST PRACTICES

- Use connection pooling for better performance.
- Use streaming for large files.
- Enable passive mode for better firewall compatibility.
- Use file filters to process only required files.
- Secure credentials using Mule Secure Configuration Properties.
- Handle errors and move files to error directory.
- Keep remote directories clean and organized.

## INTERVIEW TIPS

- FTP Connector uses File Transfer Protocol for file operations.
- It provides read, write, list, delete, move, rename, create directory.
- Passive mode is recommended for firewall environments.
- Streaming is used for handling large files efficiently.
- Use polling for scheduled file processing.
- Used for secure file exchange and batch integrations.

## COMMON MISTAKES

- Not enabling passive mode behind firewall.
- Hardcoding credentials in flows.
- Not using streaming for large files.
- Incorrect remote directory paths.
- Not handling file exceptions.
- Not using file filters, leading to unnecessary processing.
- Ignoring file locking or concurrency.

## KEY TAKEAWAY

FTP Connector enables reliable file transfer operations using FTP. It supports **read**, **write**, **list**, **delete**, **move** and other file operations. It is ideal for file exchange, automation and enterprise integrations.



Use FTP Connector to **transfer files**, automate exchanges and integrate your systems **efficiently** and **reliably**!





## 7.8

## EMAIL CONNECTOR

MODULE 7  
CONNECTORS

Email Connector is used to **send**, **receive** and **manage** emails using **SMTP**, **IMAP** and **POP3** protocols. It enables email communication with secure authentication and configurable operations.

## KEY POINTS

- Used to **send**, **receive** and **manage** emails.
- Supports **SMTP** for sending emails.
- Supports **IMAP** and **POP3** for receiving emails.
- Supports operations: **send**, **receive**, **list**, **delete**.
- Supports **secure authentication** using username/password.
- Supports **SSL/TLS** encryption for secure email communication.
- Supports **attachments** in email messages.
- Supports **filtering** emails by subject, sender, date, etc.
- Configurable **polling** for periodic email retrieval.
- Used widely for **notifications**, **alerts** and **business communications**.



## WHEN TO USE?

- When you need to **send** emails via **SMTP**.
- When you need to **receive** emails from a mailbox.
- When your business requires automated email **notifications**.
- When you need to **process** incoming emails.
- When you need to read email **attachments**.
- When you need to **delete** or archive emails.
- When you need to **filter** emails based on criteria.
- When integrating systems that **communicate** via email.
- When you need **secure** email communication using **SSL/TLS**.
- When you need to **schedule** or **poll** emails periodically.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                                            |
|-------------------------|------------------------------------------------------------------------|
| Connector Configuration | Email server connection details and authentication.                    |
| Protocol                | SMTP (send), IMAP (receive) or POP3 (receive).                         |
| Host                    | Email server host (e.g., smtp.gmail.com).                              |
| Port                    | Port number (e.g., 25, 465, 587 for SMTP; 993 for IMAP; 995 for POP3). |
| Username                | User email address.                                                    |
| Password                | Password for authentication.                                           |
| Security                | None, SSL or TLS.                                                      |
| Folder / Mailbox        | Mailbox or folder to access (INBOX, Sent, etc.).                       |
| Operation               | SEND, RECEIVE, LIST, DELETE, etc.                                      |
| Email From              | Sender email address (used for sending).                               |
| Email To / Cc / Bcc     | Recipient email address(es) (used for sending).                        |
| Subject                 | Email subject (used for sending).                                      |
| Polling Frequency       | Frequency for polling emails (if using listener).                      |
| Timeout                 | Time to wait for server response.                                      |



## SYNTAX IN MULE.XML

```

<email:send config-ref="Email_Config" >
  <email:from>noreply@company.com</email:from>
  <email:to>customer@company.com</email:to>
  <email:subject>Order Confirmation</email:subject>
  <email:body><![CDATA[
    Dear Customer,
    Your order has been confirmed.
    Thank you!
  ]]></email:body>
  <email:attachment filePath="/data/invoice.pdf"
    contentType="application/pdf"/>
</email:send>
  
```

Sends an email with subject, body and attachment using SMTP.

For receiving emails, use: <email:receive>, <email:list>, <email:delete> etc.



## REAL-WORLD EXAMPLE

A support system sends ticket notifications and processes incoming customer replies.



## Send Notification (SMTP)

- From : support@company.com
- To : customer@example.com
- Subject : Ticket #12345 Created
- Body : Ticket details and instructions

## Receive Reply (IMAP/POP3)

- Connect to mailbox
- Read customer reply
- Extract subject, body, attachments
- Update ticket in system

- Automatically notifies customers about tickets.
- Retrieves customer replies and updates the system.
- Supports attachments for invoices, reports, and documents.
- Ensures secure and reliable email communication.



## BEST PRACTICES

- Use **SSL/TLS** for secure email communication.
- Store credentials in **Secure Configuration Properties**.
- Use appropriate protocol (**SMTP** for send, **IMAP/POP3** for receive).
- Use **filters** to fetch only relevant emails.
- Delete** or **archive** processed emails to avoid duplicates.
- Handle errors and timeouts gracefully.
- Avoid **hardcoding** email addresses and passwords.



## INTERVIEW TIPS

- Email Connector uses **SMTP**, **IMAP** and **POP3** protocols.
- SMTP** is used to send emails.
- IMAP** and **POP3** are used to receive emails.
- Supports operations: **send**, **receive**, **list**, **delete**.
- SSL/TLS** provides secure email communication.
- Polling** is used to check for new emails periodically.
- Supports **attachments** and email **filters**.



## COMMON MISTAKES

- Using wrong protocol for sending/receiving emails.
- Hardcoding credentials in flows.
- Not enabling **SSL/TLS**.
- Not handling empty inbox scenarios.
- Not deleting processed emails.
- Ignoring email size limits and attachment restrictions.
- Not handling authentication failures.



## KEY TAKEAWAY

Email Connector in MuleSoft enables seamless email communication by supporting **send**, **receive**, **list**, **delete** and other email operations over **SMTP**, **IMAP** and **POP3**. It is ideal for notifications, alerts, automated processes and enterprise integrations with secure and reliable email handling.



Use Email Connector to **send**, **receive** and **manage** emails **securely**, automatically and efficiently in your integrations!





## 7.9

## JMS CONNECTOR

MODULE 7  
CONNECTORS

MuleSoft

JMS Connector is used to **send** and **receive** messages using Java Message Service (JMS). It enables reliable, **asynchronous** messaging between Mule applications and JMS providers.

## KEY POINTS

- Used to **send** and **receive** messages using JMS.
- Supports JMS providers: ActiveMQ, IBM MQ, TIBCO EMS, Oracle AQ, etc.
- Supports operations: **send**, **receive**, browse, request-response.
- Supports **Queue** and **Topic** destinations.
- Supports TextMessage, MapMessage, ObjectMessage, BytesMessage, StreamMessage.
- Supports **message selectors** for filtered consumption.
- Supports client **acknowledgement** and **transactions**.
- Configurable connection **pooling** and **reconnection** strategy.
- Supports **durable subscriptions** for topic listeners.
- Used widely for **asynchronous** communication and event-driven integration.



## WHEN TO USE?

- When you need to **send** messages to a JMS Queue or Topic.
- When you need to **receive** messages **asynchronously**.
- When systems need to communicate in a **decoupled** manner.
- When you need **reliable messaging** between applications.
- When you need to implement **request-response** over JMS.
- When you need to handle **high-volume** message traffic.
- When you need to **filter** messages using **message selectors**.
- When you need **transactional** or **acknowledged** messaging.
- When you need **durable subscriptions** for topic listeners.
- When integrating with legacy systems using JMS.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                | Description                                                |
|----------------------|------------------------------------------------------------|
| Connection Factory   | Reference to the JMS connection factory.                   |
| JMS Provider         | JMS provider implementation (e.g., ActiveMQ).              |
| Broker URL           | URL of the JMS provider (e.g., tcp://localhost:61616).     |
| Destination          | Queue or Topic name.                                       |
| Destination Type     | QUEUE or TOPIC.                                            |
| Username             | User name for authentication (if required).                |
| Password             | Password for authentication (if required).                 |
| Client ID            | Client identifier (required for durable subscriptions).    |
| Message Selector     | Expression to filter messages.                             |
| Acknowledgement Mode | AUTO_ACKNOWLEDGE, CLIENT_ACKNOWLEDGE, DUPS_OK_ACKNOWLEDGE. |
| Transacted           | Enable/disable transactions.                               |
| Durable Subscription | Enable durable subscription for topics (listener only).    |
| Connection Pooling   | Pool configuration for connection factory.                 |
| Reconnection         | Reconnection strategy for resilience.                      |



## SYNTAX IN MULE.XML

```

<jms:config name="JMS_Config" docname="JMS Config">
  <jms:connection-factory>
    <jms:provider class="org.apache.activemq.ActiveMQConnectionFactory">
      <jms:property name="brokerURL" value="tcp://localhost:61616"/>
    </jms:provider>
  </jms:connection-factory>
</jms:config>

<jms:send config-ref="JMS_Config" destination="orders.queue"
docname="Send Order Message">
  <jms:message>
    <jms:text-message>#{payload}</jms:text-message>
  </jms:message>
</jms:send>
  
```

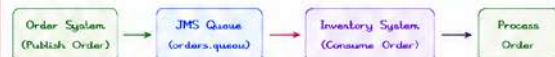
Example shows sending a text message to a JMS Queue.

Use <jms:receive>, <jms:browse>, <jms:request>, <jms:reply> or <jms:listener> based on your requirement.



## REAL-WORLD EXAMPLE

An order processing system publishes orders to a JMS Queue. Another system consumes and processes the orders.



## Message Flow

- Order System sends order messages to the JMS Queue.
- Messages are stored in the queue by the JMS provider.
- Inventory System listens on the queue and receives messages.
- Orders are processed asynchronously and reliably.

- ✓ Decouples systems for better scalability and reliability.
- ✓ Ensures message delivery even if consumer is temporarily unavailable.
- ✓ Supports high-throughput and fault-tolerant messaging.
- ✓ Ideal for event-driven and enterprise messaging scenarios.



## BEST PRACTICES

- ✓ Use connection pooling for better performance.
- ✓ Use appropriate acknowledgement mode based on use case.
- ✓ Use transactions for reliable message processing.
- ✓ Use message selectors to consume only relevant messages.
- ✓ Use durable subscriptions for topic listeners.
- ✓ Handle exceptions and implement retry logic.
- ✓ Monitor queue/topic sizes and message flow.



## INTERVIEW TIPS

- JMS provides asynchronous and reliable messaging.
- Two types of destinations: Queue (point-to-point) and Topic (publish-subscribe).
- Messages are persisted by JMS provider until consumed.
- Acknowledgement modes control message delivery guarantee.
- Durable subscriptions ensure messages are not lost for offline subscribers.



## COMMON MISTAKES

- ✗ Not closing connections or sessions.
- ✗ Using wrong destination type (Queue vs Topic).
- ✗ Not enabling transactions when required.
- ✗ Ignoring message selectors and fetching all messages.
- ✗ Not setting client ID for durable subscriptions.
- ✗ Not handling timeouts and exceptions properly.



## KEY TAKEAWAY

JMS Connector in MuleSoft enables asynchronous and reliable messaging using Java Message Service. It supports **send**, **receive**, **browse**, **request-response** and listener operations on Queues and Topics. It is ideal for decoupled, event-driven architectures and enterprise message integration.



Use JMS Connector to exchange messages **reliably**, **asynchronously** and **securely** across systems in your integrations!





## 7.10

## VM CONNECTOR

MODULE 7  
CONNECTORS

VM Connector is used to publish to and consume messages from VM (In-Memory) queues. It enables fast, lightweight messaging between flows within the same Mule application (same JVM).

## KEY POINTS

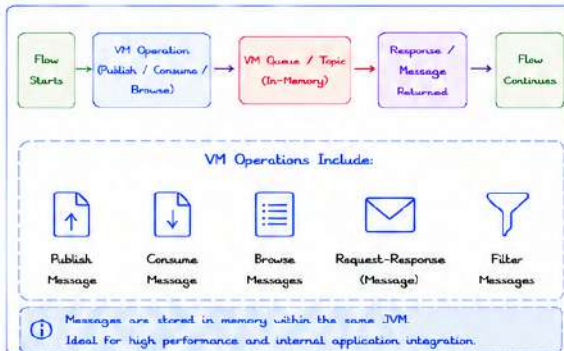
- Used to **publish** to and **consume** messages from VM queues.
- Enables messaging between flows running in the **same Mule application**.
- Messages do not leave the JVM memory (**in-memory** transport).
- Very **high performance** and low latency.
- Supports operations: **publish**, **consume**, **browse**.
- Supports **Queue** and **Topic** destinations.
- Supports **persistent** queues to survive application restarts.
- Supports **transactions** and **client acknowledgement**.
- Supports **message selectors** for filtered message consumption.
- Used widely for **internal application integration** and decoupled flows.



## WHEN TO USE?

- When you need to exchange messages between flows in the **same Mule application**.
- When you need **fast, lightweight** in-memory messaging.
- When you want to **decouple** producers and consumers.
- When you need reliable messaging with **transactions**.
- When you need **persistent** queues to handle restarts.
- When you need **topic-based** **publish/subscribe** communication.
- When you need **request-response** over in-memory queues.
- When integrating **multiple flows** within the same application.
- When you need to implement **event-driven** architectures.
- When using **external brokers** is not required or overkill.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                                                  |
|-------------------------|------------------------------------------------------------------------------|
| Connector Configuration | Reference to the VM Connector configuration.                                 |
| Destination             | Queue or Topic name.                                                         |
| Destination Type        | QUEUE or TOPIC                                                               |
| Persistent              | Enable persistent queue (survives restarts).                                 |
| Expiry Policy           | Action when persistent queue is full (NONE, DROP_OLDEST, DROP_NEWEST, FAIL). |
| Max Queue Size          | Maximum number of messages in persistent queue.                              |
| Username                | User name for authentication (if required).                                  |
| Password                | Password for authentication (if required).                                   |
| Ack Mode                | AUTO_ACK, CLIENT_ACK or TRANSACTED.                                          |
| Message Selector        | Expression to filter messages (e.g., JMS selector).                          |
| Receive Timeout         | Time to wait for a message (in milliseconds).                                |
| Redelivery Policy       | Configuration for redelivery attempts and back-off.                          |
| Transaction Manager     | Reference to transaction manager (if using transactions).                    |

## SYNTAX IN MULE.XML

```

<!-- Publish to VM Queue -->
<vm:publish config-ref="VM_Config" destination="orders.queue">
  <vm:message>
    <vm:text-message><[CDATA[Order Created]]</vm:text-message>
  </vm:message>
</vm:publish>

<!-- Consume from VM Queue -->
<vm:listener config-ref="VM_Config" destination="orders.queue"
  ackMode="AUTO_ACK">
  <vm:message>
    <!-- Process message -->
  </vm:message>
</vm:listener>

```

Use publish to send messages and listener to consume messages from VM queues or topics.

Use <vm:request> and <vm:reply> for request-response patterns.

## REAL-WORLD EXAMPLE

An order system publishes orders to a VM queue. Another flow processes the orders asynchronously.



## How it works:

- Order API Flow publishes order messages to orders.queue.
- Order Processing Flow listens to orders.queue and processes each order.
- VM messaging ensures high performance and decoupling within the same application.

- High performance in-memory messaging.
- Decouples flows for scalability and reliability.
- Persistent queues ensure no message loss on restart.
- Ideal for internal/application-level integration.

## BEST PRACTICES

- Use VM queues for point-to-point communication.
- Use VM topics for publish/subscribe scenarios.
- Keep queue sizes reasonable to avoid memory pressure.
- Enable persistent queues for critical time-dependent flows for end-to-end message reliability.
- Use message selectors to consume only relevant messages.
- Monitor queue sizes and consumer health.
- Avoid long-running processing in listeners.

## INTERVIEW TIPS

- VM Connector enables in-memory messaging within the same application.
- Supports Queue and Topic patterns.
- Extremely fast since no network communication.
- Persistent queues survive restarts.
- Supports transactions and acknowledgements.
- Ideal for decoupling flows and implementing event-driven architectures.
- Not suitable for communication across different applications or servers.

## COMMON MISTAKES

- Using VM queues for cross-application communication.
- Ignoring queue size limits.
- Not enabling persistence for critical messages.
- Forgetting to handle exceptions in listeners.
- Not using transactions when required.
- Processing messages synchronously in producer flow.
- Ignoring queue monitoring.

## KEY TAKEAWAY

VM Connector in MuleSoft enables fast, reliable, in-memory messaging using VM queues and topics. It supports **publish**, **consume**, **browse** and **request-response** operations. It is perfect for internal integration, decoupling flows, and building high-performance, event-driven applications.



Use VM Connector to exchange messages within your application **fast, reliably and efficiently!**





## 7.11

## SOAP CONSUMER

MODULE 7  
CONNECTORS

SOAP Consumer is used to consume SOAP web services by invoking operations on a SOAP endpoint. It supports both SOAP 1.1 and SOAP 1.2 and enables secure, reliable web service integration.

## KEY POINTS

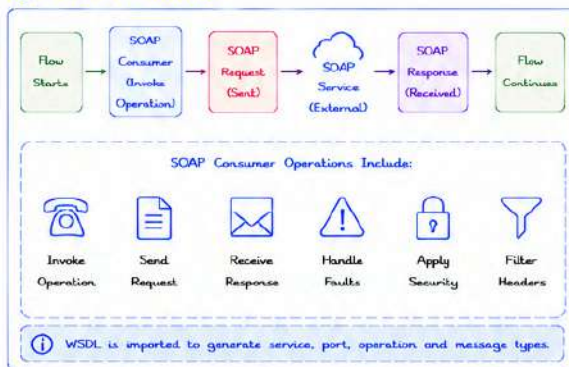
- Used to consume SOAP web services.
- Supports SOAP 1.1 and SOAP 1.2.
- Imports WSDL to generate service and operation metadata.
- Supports dynamic value for endpoint, operation and payload.
- Supports WS-Security (Username Token, X.509, etc.).
- Supports MTOM for file/attachment transfer.
- Supports custom SOAP headers.
- Supports connection pooling and configurable timeouts.
- Supports error handling and detailed fault responses.
- Supports request-response and one-way (in-out / in-only) patterns.
- Used widely for system-to-system integration via SOAP services.



## WHEN TO USE?

- When you need to consume (invoke) a SOAP web service.
- When your business partners expose services via SOAP.
- When you need to call operations defined in a WSDL.
- When you need secure communication using WS-Security.
- When you need to send or receive file attachments (MTOM).
- When you need to add custom SOAP headers.
- When you need reliable request-response interactions.
- When integrating with enterprise or legacy SOAP services.
- When you need to handle SOAP faults and error codes.
- When you need to use dynamic endpoint or operation names.
- When you prefer standards-based web service integration.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                     | Description                                                |
|---------------------------|------------------------------------------------------------|
| Connector Configuration   | Reference to the SOAP Consumer connector configuration.    |
| WSDL Location             | URL or path to the WSDL file.                              |
| Service                   | SOAP service name from the WSDL.                           |
| Port                      | SOAP port (endpoint) from the WSDL.                        |
| Operation                 | Operation to be invoked.                                   |
| Endpoint Address          | Service endpoint URL (overrides WSDL port address if set). |
| SOAP Version              | SOAP 1.1 or SOAP 1.2.                                      |
| Security                  | WS-Security configuration (if required).                   |
| MTOM                      | Enable MTOM for attachments.                               |
| Connection Timeout        | Timeout for establishing connection.                       |
| Response Timeout          | Timeout to wait for the response.                          |
| Request Encoding          | Encoding used for the SOAP request (e.g., UTF-8).          |
| Disable Transport Headers | Do not send transport headers (if required).               |
| Stream Attachments        | Stream attachments to optimize memory usage.               |
| Doc/Lit/Bare Support      | Supports Document/Literal (Bare) style web services.       |

## SYNTAX IN MULE.XML

```

<soap:consumer config-ref="SOAP_Config"
  docname="SOAP_Consumer">
  <soap:request operation="getCustomer">
    <soap:body>
      <soap:customerId>#{vars.customerId}</soap:customerId>
    </soap:body>
    <soap:headers>
      <!-- Optional Custom Headers -->
      <soap:header
        namespace="Security"
        <!-- WS-Security elements -->
        </soap:header>
      </soap:headers>
    </soap:request>
  </soap:consumer>

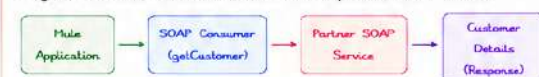
```

Invokes the operation defined in the WSDL and returns the response.

Use DataWeave to build request payloads and transform responses.

## REAL-WORLD EXAMPLE

A system retrieves customer details from a partner SOAP service.



## Example Flow:

- Mule application sends SOAP request with customerId.
- Partner SOAP service processes the request.
- SOAP response with customer details is returned.
- Mule transforms and stores or forwards the response.

- Standards-based and interoperable communication.
- Secure communication using WS-Security.
- Reliable request-response and fault handling.
- Ideal for enterprise and legacy system integration.

## BEST PRACTICES

- Use connection pooling for better performance.
- Import WSDL and use metadata instead of hardcoding payloads.
- Use WS-Security for authentication and message integrity.
- Use MTOM for large attachments.
- Handle SOAP faults and business errors gracefully.
- Set appropriate timeouts to avoid hanging requests.
- Reuse connector configuration across flows.

## INTERVIEW TIPS

- SOAP Consumer invokes operations on a SOAP web service.
- WSDL defines service, port, operation and message structure.
- It supports SOAP 1.1 and SOAP 1.2.
- Supports WS-Security, MTOM and custom headers.
- Returns SOAP response or SOAP fault.
- Uses request-response (in-out) or one-way (in-only) patterns.

## COMMON MISTAKES

- Using wrong endpoint, service or operation.
- Not handling SOAP faults.
- Ignoring timeouts and connection pooling.
- Hardcoding payloads instead of using WSDL types.
- Not enabling MTOM for large attachments.
- Incorrect WS-Security configuration.

## KEY TAKEAWAY

SOAP Consumer in MuleSoft enables reliable and secure consumption of SOAP web services using WSDL. It supports standards-based communication, security, attachments and robust error handling—ideal for enterprise and legacy integrations.



Use SOAP Consumer to securely, reliably and efficiently consume SOAP web services and integrate your systems seamlessly!





## 7.12

## WEB SERVICE CONSUMER

MODULE 7  
CONNECTORS

MuleSoft

Web Service Consumer is used to consume web services by invoking operations on a service endpoint. It supports SOAP and REST web services and enables secure, reliable service integration.

## KEY POINTS

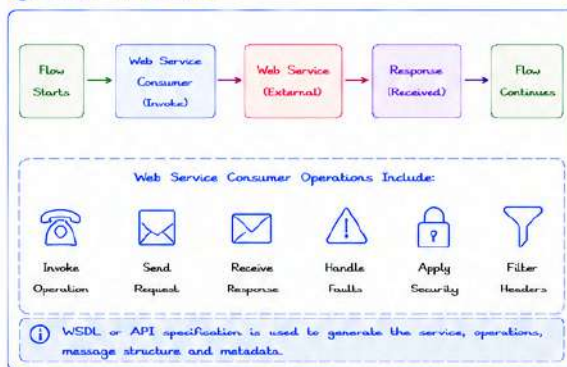
- Used to consume SOAP and REST web services.
- Supports SOAP 1.1, SOAP 1.2 and REST.
- Imports WSDL (for SOAP) or API specifications (for REST).
- Supports dynamic values for endpoint, operation, headers and payload.
- Supports WS-Security for authentication and message security.
- Supports MTOM for file/attachment transfer (SOAP).
- Supports custom headers for both SOAP and REST.
- Supports connection pooling and configurable timeouts.
- Supports error handling and detailed fault responses.
- Supports request-response and one-way (in-out / in-only) patterns.
- Used widely for system-to-system integration and API consumption.



## WHEN TO USE?

- When you need to consume a SOAP or REST web service.
- When your business partners expose services via APIs.
- When you need to call operations defined in a WSDL (SOAP).
- When you need to interact with RESTful resources.
- When you need secure communication using WS-Security or OAuth.
- When you need to send or receive file attachments (MTOM).
- When you need custom headers, query params or path params.
- When you need reliable request-response interactions.
- When integrating with enterprise or legacy web services.
- When you need to handle web service faults and errors.
- When you prefer standards-based web service integration.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                     | Description                                                    |
|---------------------------|----------------------------------------------------------------|
| Connector Configuration   | Reference to the Web Service Consumer connector configuration. |
| Service                   | Web service name (from WSDL or API spec).                      |
| Port / Endpoint           | Service endpoint URL (overrides WSDL port if set).             |
| Operation                 | Operation to be invoked.                                       |
| WSDL Location / API Spec  | URL or path to the WSDL file (SOAP) or OpenAPI/Swagger (REST). |
| Protocol                  | SOAP 1.1, SOAP 1.2 or REST.                                    |
| Security                  | WS-Security configuration, OAuth or Basic Auth.                |
| MTOM                      | Enable MTOM for attachments (SOAP).                            |
| Response Timeout          | Timeout to wait for the response.                              |
| Connection Timeout        | Timeout for establishing connection.                           |
| Request Encoding          | Encoding used for the request (e.g., UTF-8).                   |
| Disable Transport Headers | Do not send transport headers (if required).                   |
| Stream Attachments        | Stream attachments to optimize memory usage.                   |
| Doc/Lib/Bare Support      | Supports Document/Literal (Bare) style web services.           |

## SYNTAX IN MULE.XML

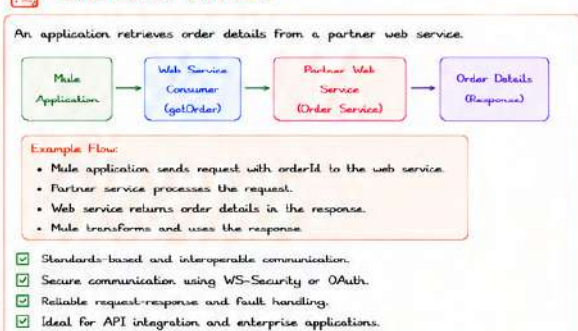
```

<ws:consumer config-ref="WS_Config"
  service="CustomerService"
  port="CustomerPort"
  operation="getCustomer">
  <ws:request><[CDATA[ <ns:getCustomerRequest>
    <ns:customerId>#{vars.customerId}</ns:customerId>
  </ns:getCustomerRequest> ]]></ws:request>
</ws:consumer>
  
```

Invokes the specified operation on the web service and returns the response.

Use <ws:header> to add custom headers and <ws:attachment> for files.

## REAL-WORLD EXAMPLE



## BEST PRACTICES

- Use connection pooling for better performance.
- Reuse Web Service Consumer configurations across Flows.
- Set appropriate timeouts to avoid hanging requests.
- Use WS-Security for secure communication.
- Use MTOM for large attachments.
- Handle SOAP faults and business errors gracefully.
- Log troubleshooting.
- Monitor service availability and performance.

## INTERVIEW TIPS

- Web Service Consumer is used to invoke SOAP or REST web services.
- It supports SOAP 1.1, SOAP 1.2 and REST.
- WSDL defines service, port, operations and message structure (SOAP).
- For REST, API specification (e.g., OpenAPI) is used.
- Supports WS-Security, MTOM, custom headers and dynamic values.
- It handles request-response and one-way patterns.
- Returns response or faults from the web service.

## COMMON MISTAKES

- Using wrong endpoint, service or operation.
- Incorrect WSDL location or API spec.
- Ignoring timeouts and connection pooling.
- Not handling SOAP faults or error codes.
- Hardcoding request payloads instead of using dynamic data.
- Not using WS-Security when required.
- Overloading headers instead of using proper security.

## KEY TAKEAWAY

Web Service Consumer in MuleSoft enables secure, reliable and efficient consumption of SOAP and REST web services. It supports standards-based communication, dynamic configuration, security and robust error handling—ideal for enterprise API and system integration.



Use Web Service Consumer to securely, reliably and efficiently consume web services and integrate your applications seamlessly!





## 7.13

## REST API

MODULE 7  
CONNECTORS

MuleSoft

REST API Connector is used to interact with RESTful web services using HTTP methods. MuleSoft supports **creating** and **consuming** REST APIs for scalable and stateless integrations.

## KEY POINTS

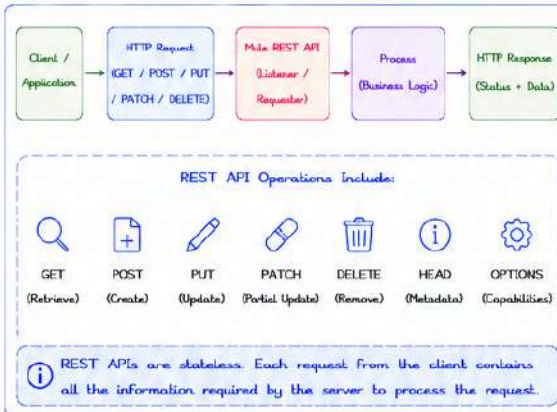
- Based on REST architectural style and HTTP protocol.
- Supports HTTP methods: GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS.
- Supports common data formats: JSON, XML, CSV, Text, etc.
- Uses URIs for resource identification.
- Stateless communication using request-response model.
- Supports query parameters, path parameters and headers.
- Supports content negotiation and status codes.
- Supports authentication using Basic, OAuth 2.0, API Key, etc.
- Supports error handling and exception mapping.
- Used widely for system, application and API integrations.



## WHEN TO USE?

- When you need to expose your systems as RESTful APIs.
- When you need to consume REST APIs from external systems.
- When you need lightweight and scalable integrations.
- When you need platform and language independent APIs.
- When you need to integrate with cloud and SaaS services.
- When you need fine-grained control using HTTP methods.
- When you need to pass data using URL, headers or body.
- When you need support for modern formats like JSON.
- When you need to implement stateless integrations.
- When you need to follow REST best practices.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                   | Description                                        |
|-------------------------|----------------------------------------------------|
| Connector Configuration | Reference to the REST API Connector configuration. |
| Protocol                | HTTP or HTTPS.                                     |
| Host                    | Host name of the REST API.                         |
| Port                    | Port number (default: 80 for HTTP, 443 for HTTPS). |
| Base Path               | Base path of the API (e.g. /api/v1).               |
| Method                  | HTTP method (GET, POST, PUT, PATCH, DELETE, etc.). |
| Resource Path           | Relative resource path (e.g. /orders/{id}).        |
| Query Parameters        | Map of query parameters.                           |
| Path Parameters         | Map of path parameters.                            |
| Headers                 | Custom request/response headers.                   |
| Body                    | Request payload (JSON, XML, Text, etc.).           |
| Response                | Response payload and status code.                  |
| Authentication          | Basic, OAuth 2.0, API Key, etc.                    |
| Timeout                 | Connection and response timeout settings.          |
| Error Handling          | Define error types and response mapping.           |

## SYNTAX IN MULE.XML

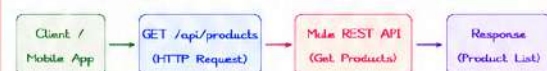
```
<http:listener-config name="HTTP_Listener_Config"
  host="0.0.0.0" port="8081" basePath="/api"/>
<http:listener-config-ref="HTTP_Listener_Config" path="/users">
  <http:method>
    <http:response statusCode="200" reasonPhrase="OK"
      doc:name="Response"/>
  </http:method>
</http:listener>
```

Use DataWeave to transform request/response payloads.



## REAL-WORLD EXAMPLE

An e-commerce system exposes REST API to manage products.



## Example Use Cases:

- Expose CRUD operations for products.
- Integrate with third-party REST services (payment, shipping, etc.).
- Build mobile and web applications.
- Enable microservices communication.

## BEST PRACTICES

- Use meaningful URIs and HTTP methods.
- Use appropriate status codes.
- Validate and sanitize inputs.
- Use pagination for large data.
- Implement security and rate limiting.
- Version your APIs.
- Document your APIs.

## INTERVIEW TIPS

- REST is an architectural style.
- It uses HTTP methods and URIs.
- REST is stateless and scalable.
- Common methods: GET, POST, PUT, DELETE.
- Supports formats like JSON and XML.
- Authentication: Basic, OAuth 2.0, API Key.
- MuleSoft supports creating and consuming REST APIs.

## COMMON MISTAKES

- Using wrong HTTP method.
- Ignoring status codes.
- Hardcoding URIs and endpoints.
- Not handling timeouts.
- Exposing sensitive information.
- Ignoring input validation.
- Not versioning APIs.
- Overfetching or underfetching data.

## KEY TAKEAWAY

REST APIs in MuleSoft enable you to **create** and **consume** web services using **HTTP** methods and **URIs**. They provide **scalable**, **stateless**, **platform-independent** and **standards-compliant** integrations ideal for modern applications.



Use REST APIs to build **scalable**, **stateless** and **secure** integrations and expose your systems to the world!





## 7.14

## ACTIVE MQ

MODULE 7  
CONNECTORS

MuleSoft

ActiveMQ Connector is used to **publish** and **consume** messages from ActiveMQ message broker. It enables reliable, asynchronous messaging using **Queues** and **Topics**.

## KEY POINTS

- Used to publish and consume messages from ActiveMQ broker.
- Supports **Queues** (point-to-point) and **Topics** (publish-subscribe).
- Supports operations: **send**, **receive**, **request-response**.
- Supports message acknowledgment modes: **AUTO**, **CLIENT**, **DUPS\_OK**.
- Supports transactions for guaranteed delivery.
- Supports durable subscriptions for Topics.
- Supports message selectors for filtered consumption.
- Supports **persistent** messages to survive broker restarts.
- Supports **SSL/TLS** for secure communication.
- Used widely for reliable, asynchronous and decoupled integrations.

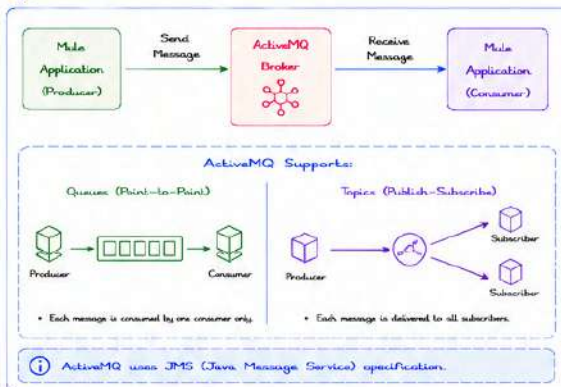


## WHEN TO USE?

- When you need **asynchronous** and **reliable** messaging.
- When systems must be communicated in a decoupled manner.
- When you need to handle high message volumes.
- When you need to implement **publish-subscribe** communication.
- When you need guaranteed message **delivery**.
- When you need to buffer messages during downtime.
- When you need message filtering using **selectors**.
- When integrating with legacy systems using **JMS**.
- When building event-driven and enterprise applications.
- When you prefer a mature, open-source message broker.



## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                | Description                                              |
|----------------------|----------------------------------------------------------|
| Connection Factory   | ActiveMQ connection factory.                             |
| Broker URL           | URL of the ActiveMQ broker (e.g. tcp://localhost:61616). |
| Username             | Username for authentication (if required).               |
| Password             | Password for authentication (if required).               |
| Destination Type     | QUEUE or TOPIC.                                          |
| Destination          | Queue or Topic name.                                     |
| Operation            | SEND, RECEIVE or REQUEST-RESPONSE.                       |
| Acknowledgment Mode  | AUTO_ACK, CLIENT_ACK, DUPS_OK_ACK.                       |
| Message Selector     | Expression to filter messages.                           |
| Persistent Delivery  | Enable to send persistent messages.                      |
| Durable Subscription | Enable durable subscription for topics.                  |
| Client ID            | Unique client ID for durable subscriptions.              |
| Transaction          | Enable transactions (if required).                       |
| Redelivery Policy    | Configuration for redelivery attempts and delays.        |
| Use SSL              | Enable SSL/TLS for secure communication.                 |



## SYNTAX IN MULE.XML

```

<activemq:activemq-connection-factory name="ActiveMQ-Connection"
  brokerURL="tcp://localhost:61616"
  userName="admin" password="admin"
  clientId="mule-client"
  acknowledgementMode="AUTO_ACK" />

<!-- Send message to Queue -->
<activemq:send config-ref="ActiveMQ-Connection"
  destinationType="QUEUE"
  destination="orders.queue"
  messagePersistent="true">
  <activemq:message>#{payload}</activemq:message>
</activemq:send>

<!-- Receive message from Queue -->
<activemq:receive config-ref="ActiveMQ-Connection"
  destinationType="QUEUE"
  destination="orders.queue"
  acknowledgementMode="AUTO_ACK" />

```

① Use DataWeave to transform messages as needed.



## REAL-WORLD EXAMPLE

An order processing system uses ActiveMQ for asynchronous order handling.



## Example Flow:

- Order Service publishes order messages to orders.queue.
- Order Processing Service consumes messages asynchronously.
- Each order is processed and updated in the database/ERP.
- Ensures reliable delivery even during system downtime.

- ✓ Improves system scalability and performance.
- ✓ Decouples services for flexibility and reliability.
- ✓ Ensures guaranteed message delivery.
- ✓ Ideal for batch processing and event-driven architectures.



## BEST PRACTICES

- ✓ Use **Queues** for point-to-point communication.
- ✓ Use **Topics** for publish-subscribe scenarios.
- ✓ Choose appropriate acknowledgment mode.
- ✓ Use persistent messages for reliability.
- ✓ Use message selectors to filter unwanted messages.
- ✓ Use transactions for critical integrations.
- ✓ Enable **SSL/TLS** for secure communication.
- ✓ Monitor broker and queues using ActiveMQ console.



## INTERVIEW TIPS

- ActiveMQ supports JMS specification.
- It uses **Queues** (P2P) and **Topics** (Pub-Sub).
- Common operations: **send**, **receive**, **request-response**.
- Acknowledgment modes: **AUTO**, **CLIENT**, **DUPS\_OK**.
- Supports durable subscriptions for Topics.
- Supports transactions and persistent messages.
- Used for reliable, asynchronous and decoupled messaging.



## COMMON MISTAKES

- ✗ Using wrong destination type (Queue/Topic).
- ✗ Not setting appropriate ack mode.
- ✗ Ignoring message persistence.
- ✗ Not using durable subscriptions for topics.
- ✗ Hardcoding broker URL and credentials.
- ✗ Not handling redelivery and failed messages.
- ✗ Not monitoring broker queues and performance.



## KEY TAKEAWAY

ActiveMQ Connector in MuleSoft enables **reliable**, **asynchronous** messaging using **Queues** and **Topics**. It supports acknowledgment modes, persistent messages, transactions, selectors, **SSL/TLS** and durable subscriptions for robust and scalable integrations.



Use ActiveMQ to build **reliable**, **scalable** and **decoupled** messaging solutions in your applications!





## 7.15

## ANYPOINT MQ

MODULE 7  
CONNECTORS

MuleSoft

Anypoint MQ Connector is used to **publish** and **consume** messages from Anypoint MQ message broker. It enables reliable, secure and asynchronous messaging using **Queues** and **Topics**.

## KEY POINTS

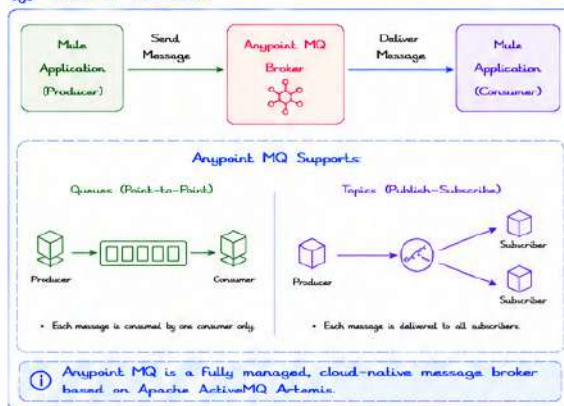
- Used to **publish** and **consume** messages from Anypoint MQ.
- Supports **Queues** (point-to-point) and **Topics** (publish-subscribe).
- Supports operations: **send**, **receive**, **request-response**.
- Supports message acknowledgment modes: **AUTO**, **CLIENT**, **DUPS\_OK**.
- Supports transactions for guaranteed delivery.
- Supports durable subscriptions for Topics.
- Supports message selectors for filtered consumption.
- Supports persistent messages to survive broker restarts.
- Supports **SSL/TLS** for secure communication.
- Used widely for reliable, asynchronous and decoupled integrations.



## WHEN TO USE?

- When you need **asynchronous** and **reliable messaging**.
- When systems must be decoupled in time and space.
- When you need to handle high message volumes.
- When you need guaranteed message delivery.
- When you need to implement **request-response** over MQ.
- When you need message filtering using **selectors**.
- When integrating with legacy systems using **JMS**.
- When building event-driven and microservices architectures.
- When you prefer a fully managed, cloud-native MQ service.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                | Description                                                                                     |
|----------------------|-------------------------------------------------------------------------------------------------|
| Connection Factory   | Anypoint MQ connection factory                                                                  |
| Broker URL           | URL of the Anypoint MQ broker (e.g. <code>sl://mq.us-east-1.anypoint.mulesoft.com:5671</code> ) |
| Username             | Username for authentication                                                                     |
| Password             | Password for authentication                                                                     |
| Destination Type     | QUEUE or TOPIC                                                                                  |
| Destination          | Queue or Topic name                                                                             |
| Operation            | SEND, RECEIVE or REQUEST-RESPONSE                                                               |
| Acknowledge Mode     | AUTO_ACK, CLIENT_ACK, DUPS_OK_ACK                                                               |
| Message Selector     | Expressions to filter messages                                                                  |
| Persistent Delivery  | Enable to send persistent messages                                                              |
| Durable Subscription | Enable durable subscription for Topics                                                          |
| Client ID            | Unique client ID for durable subscriptions                                                      |
| Transaction          | Enable transactions (if required)                                                               |
| Redelivery Policy    | Configuration for redelivery attempts and delays                                                |
| Use SSL/TLS          | Enable SSL/TLS for secure communication                                                         |

## SYNTAX IN MULE.XML

```

<anypoint-mq:config name="AnypointMQ_Config"
  connectionFactory="AnypointMQ_CF"
  brokerUrl="sl://mq.us-east-1.anypoint.mulesoft.com:5671"
  username="mule-client"
  password="mule-client-secret"
  acknowledgeMode="AUTO_ACK" />

<anypoint-mq:send config-ref="AnypointMQ_Config"
  destinationType="QUEUE"
  destination="orders.queue"
  messagePersistent="true"
  <anypoint-mq:message>[payload]</anypoint-mq:message>
</anypoint-mq:send>

<anypoint-mq:receive config-ref="AnypointMQ_Config"
  destinationType="QUEUE"
  destination="orders.queue"
  acknowledgeMode="AUTO_ACK" />
<-- Or use <anypoint-mq:listener> for event-driven consumption -->
  
```

① Use DataWeave to transform messages as needed.

## REAL-WORLD EXAMPLE



## BEST PRACTICES

- Use Queues for point-to-point communication.
- Use Topics for publish-subscribe scenarios.
- Choose appropriate acknowledgment mode.
- Use persistent messages for reliability.
- Use message selectors to filter unwanted messages.
- Use transactions for critical integrations.
- Enable SSL/TLS for secure communication.
- Monitor broker and queues using Anypoint MQ console.

## INTERVIEW TIPS

- Anypoint MQ is MuleSoft's fully managed message broker based on Apache ActiveMQ Artemis.
- It supports Queues (P2P) and Topics (Pub-Sub).
- Common operations: `send`, `receive`, `request-response`.
- Acknowledgment modes: `AUTO`, `CLIENT`, `DUPS_OK`.
- Supports durable subscriptions for Topics.
- Supports transactions and persistent messages.
- Used for reliable, asynchronous and decoupled messaging.

## COMMON MISTAKES

- Using wrong destination type (Queue/Topic).
- Not setting appropriate ack mode.
- Ignoring message persistence.
- Not using durable subscriptions for Topics.
- Hardcoding broker URL and credentials.
- Not handling redelivery and failed messages.
- Not monitoring broker queues and performance.

## KEY TAKEAWAY

Anypoint MQ Connector in MuleSoft enables **reliable**, **secure** and **asynchronous** messaging using **Queues** and **Topics**. It supports acknowledgment modes, transactions, selectors, **SSL/TLS** and durable subscriptions for robust and scalable integrations in modern applications.



Use Anypoint MQ to build **reliable**, **scalable** and **decoupled** messaging solutions and drive asynchronous integrations in your applications!





## 7.16

## AZURE SERVICE BUS

MODULE 7  
CONNECTORS

Azure Service Bus Connector is used to connect to Microsoft Azure Service Bus to send and receive messages. It provides reliable, secure and scalable messaging using **Queues**, **Topics** and **Subscriptions**.

## KEY POINTS

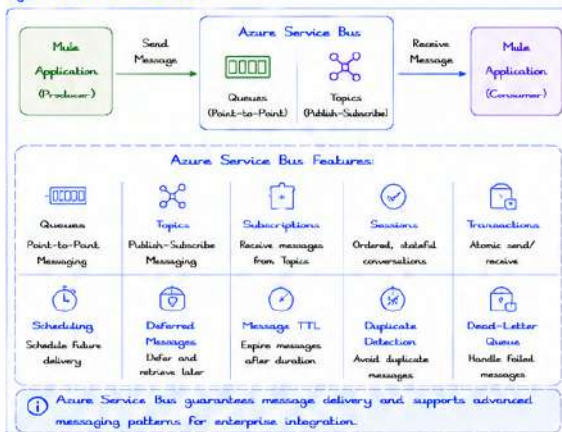
- Used to send and receive messages using Azure Service Bus.
- Supports **Queues** (point-to-point) and **Topics/Subscriptions** (publish-subscribe).
- Supports operations: **send**, **receive**, **peek-lock receive**, **complete**, **abandon**, **dead-letter**.
- Supports **sessions** for message ordering and stateful conversations.
- Supports message **scheduling** and **deferred** messages.
- Supports duplicate detection and message TTL (Time-To-Live).
- Supports **transactions** for atomic send/receive.
- Supports message **filters** using SQL-like syntax.
- Supports managed identity, SAS Key and Azure AD authentication.
- Used widely for cloud-native, scalable and secure integrations.



## WHEN TO USE?

- When you need **reliable** and **secure** cloud messaging.
- When systems must be decoupled in time and space.
- When you need to handle **high message volumes**.
- When you need **guaranteed** message delivery.
- When you need **sessions** for ordered message processing.
- When you need message filtering using **SQL** rules.
- When you need to **schedule** or **defer** messages.
- When integrating with **Azure services** and cloud apps.
- When you prefer a **fully managed**, enterprise-grade service.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field              | Description                                                                              |
|--------------------|------------------------------------------------------------------------------------------|
| Connection String  | Connection string for the Azure Service Bus namespace.                                   |
| Namespace          | Azure Service Bus namespace (e.g. mynamespace.servicebus.windows.net).                   |
| Authentication     | SAS Key, SAS Token, Managed Identity or Azure AD.                                        |
| Transport Type     | HTTPS (recommended).                                                                     |
| Entity Type        | QUEUE or TOPIC.                                                                          |
| Entity Name        | Name of the Queue or Topic.                                                              |
| Subscription Name  | Required for Topics. Name of the Subscription.                                           |
| Operation          | SEND, RECEIVE, PEEK-LOCK RECEIVE, COMPLETE, ABANDON, DEAD-LETTER, SCHEDULE MESSAGE, etc. |
| Message Selector   | SQL expression to filter messages (for receive operations).                              |
| Session Enabled    | Enable if the Queue/Topic has sessions.                                                  |
| Lock Duration      | Duration for peek-lock (e.g. PT1M for 1 minute).                                         |
| Max Delivery Count | Max delivery attempts before moving to Dead-Letter Queue.                                |
| Receive Mode       | PEEK_LOCK or RECEIVE_AND_DELETE.                                                         |
| Client ID          | Client identifier for application (optional).                                            |

## SYNTAX IN MULE.XML

```

<azure-servicebus:config name="Azure_ServiceBus_Config"
  connectionString="Endpoint=sb://mynamespace.servicebus.windows.net/;
  SharedAccessKeyName=RootManageSharedAccessKey;
  SharedAccessKey=key1234567890123456789012345678901234567890" />

<!-- Send Message to Queue -->
<azure-servicebus:send config-ref="Azure_ServiceBus_Config"
  entityName="orders-queue"
  message="#[payload]" />

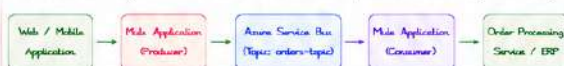
<!-- Receive Message from Subscription -->
<azure-servicebus:receive config-ref="Azure_ServiceBus_Config"
  entityName="orders-topic"
  subscriptionName="orders-subscription"
  receiveMode="PEEK_LOCK"
  messageSelector="priority = 'high'" />

```

① Use DataWeave to transform messages as needed.

## REAL-WORLD EXAMPLE

An e-commerce system uses Azure Service Bus to process orders asynchronously.



## Example Use Cases:

- Process customer orders asynchronously.
- Decouple order submission from downstream systems.
- Send notifications to multiple services using Topics and Subscriptions.
- Ensure reliable delivery even during system failures.

- ✓ Improves scalability and fault tolerance.
- ✓ Ensures guaranteed message delivery.
- ✓ Supports ordered and stateful processing with sessions.
- ✓ Ideal for cloud-native and event-driven architectures.

## BEST PRACTICES

- ✓ Use Queues for point-to-point communication.
- ✓ Use Topics and Subscriptions for publish-subscribe scenarios.
- ✓ Enable Sessions when message ordering is required.
- ✓ Use message selectors to filter unwanted messages.
- ✓ Set appropriate Lock Duration and Max Delivery Count.
- ✓ Monitor Dead-Letter Queue for failed messages.
- ✓ Use Managed Identity or Azure AD for secure authentication.
- ✓ Enable SSL/TLS for all communications.

## INTERVIEW TIPS

- Azure Service Bus is a managed messaging service by Microsoft.
- Supports Queues (P2P) and Topics/Subscriptions (Pub-Sub).
- Common operations: send, receive, complete, abandon.
- Provides sessions, transactions, scheduling, defer, TTL and duplicate detection.
- Ensures reliable and secure cloud messaging.
- Used for decoupling, load leveling and scalable integrations.
- Supports multiple authentication mechanisms.

## COMMON MISTAKES

- ✗ Using Queues when Pub-Sub is required.
- ✗ Not enabling Sessions when ordering is needed.
- ✗ Ignoring Dead-Letter Queue monitoring.
- ✗ Setting Lock Duration too short.
- ✗ Not handling message expiration (TTL).
- ✗ Using inappropriate Receive Mode.
- ✗ Not securing connection string or authentication.

## KEY TAKEAWAY

Azure Service Bus Connector in MuleSoft enables **reliable**, **secure** and **scalable** messaging using **Queues**, **Topics** and **Subscriptions**. It supports **sessions**, **transactions**, **scheduling**, **filtering** and **dead-letter handling** for robust cloud integrations and event-driven architectures.



Use Azure Service Bus to build **reliable**, **scalable** and **secure** messaging solutions and power your cloud integrations!





## 7.18

## AWS SQS

MODULE 7  
CONNECTORS

MuleSoft

AWS SQS Connector is used to send and receive messages to and from Amazon Simple Queue Service (SQS). It enables reliable, scalable and decoupled messaging using **Queues**.

## KEY POINTS

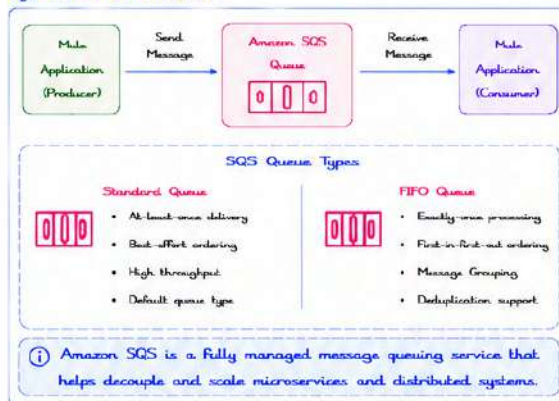
- Used to send and receive messages using Amazon SQS.
- Supports **Queues** (Standard & FIFO).
- Supports operations: **send**, **receive**, **delete**, change message visibility, get attributes, purge.
- Supports **long polling** to reduce empty responses and increase efficiency.
- Supports message attributes and message system attributes.
- Supports message batching for better performance.
- Supports FIFO queue features: **Message Group ID** and **Deduplication ID**.
- Uses AWS Signature V4 for secure authentication.
- Used widely for decoupling, buffering and scalable integrations.



## WHEN TO USE?

- When you need **reliable** and **durable** message delivery.
- When systems must be decoupled in time and space.
- When you need to handle **high message volumes**.
- When you need to **buffer** messages during downtime.
- When you need to implement **asynchronous** processing.
- When you need to **scale** consumers independently.
- When ordering of messages is required (FIFO queues).
- When you need to integrate **cloud** and **on-prem** systems.
- When you prefer a managed, **serverless** message service.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                              | Description                                            |
|------------------------------------|--------------------------------------------------------|
| Connection Name                    | Name of the AWS SQS connection.                        |
| Region                             | AWS Region of the SQS service (e.g., us-east-1).       |
| Access Key ID                      | AWS Access Key ID with required permissions.           |
| Secret Access Key                  | AWS Secret Access Key.                                 |
| Session Token (Optional)           | AWS Session Token (if using temporary credentials).    |
| Queue Name / URL                   | Name or URL of the SQS Queue.                          |
| Queue Type                         | Standard or FIFO.                                      |
| Long Polling                       | Enable to wait for messages (1 to 20 seconds).         |
| Max Number of Messages             | Number of messages to receive (1 to 10).               |
| Wait Time (Seconds)                | Long polling wait time (1 to 20 seconds).              |
| Visibility Timeout (Seconds)       | Time during which a message is invisible (0 to 43200). |
| Message Attribute Names            | Message attributes to retrieve.                        |
| Message System Attribute Names     | System attributes to retrieve.                         |
| Content Based Deduplication (FIFO) | Enable content-based deduplication (FIFO queues).      |
| Client ID (Optional)               | Identifier for AWS SDK client.                         |

## SYNTAX IN MULE.XML

```

<!-- Send Message -->
<aws-sqs:send-message config-ref="AWS_SQS_Config"
  queueName="orders-queue"
  message="#[payload]"
  messageAttributes="#[attributes]"/>

<!-- Receive Message -->
<aws-sqs:receive-message config-ref="AWS_SQS_Config"
  queueName="orders-queue"
  maxNumberOfMessages="5"
  waitTimeSeconds="10"/>

<!-- Delete Message -->
<aws-sqs:delete-message config-ref="AWS_SQS_Config"
  queueUrl="#[attributes.queueUrl]"
  receiptHandle="#[attributes.receiptHandle]"/>

```

Use DataWeave to transform messages and handle payloads.

## REAL-WORLD EXAMPLE

An order processing system uses AWS SQS for asynchronous order handling.



## Example Use Cases:

- Decouple order submission from downstream processing.
- Buffer messages during peak traffic.
- Send notifications to multiple services.
- Ensure reliable processing even during system failures.

- Improves scalability and resiliency.
- Ensures guaranteed message delivery.
- Reduces load on backend systems.
- Ideal for microservices and event-driven architectures.

## BEST PRACTICES

- Use Long Polling to reduce empty responses.
- Use message batching to improve performance.
- Set appropriate Visibility Timeout based on processing time.
- Use Dead-Letter Queues (DLQ) for failed messages.
- Use FIFO queues when ordering and exactly-once processing are required.
- Use IAM roles and least-privilege policies for security.
- Monitor queues using CloudWatch metrics and alarms.

## INTERVIEW TIPS

- AWS SQS is a fully managed message queue service.
- It supports Standard and FIFO queues.
- Standard: at-least-once, best-effort ordering.
- FIFO: exactly-once, ordered, dedup.
- Common operations: send, receive, delete, change visibility, purge.
- Long polling reduces cost and improves efficiency.
- Visibility timeout controls message re-delivery.

## COMMON MISTAKES

- Not setting appropriate visibility timeout.
- Not deleting messages after successful processing.
- Using short polling instead of long polling.
- Ignoring Dead-Letter Queues.
- Not enabling deduplication in FIFO when needed.
- Hardcoding AWS credentials.
- Not monitoring queue metrics.

## KEY TAKEAWAY

AWS SQS Connector in MuleSoft enables reliable, scalable and secure messaging using Amazon SQS **Queues**. It supports Standard and FIFO queues with features like long polling, message attributes, visibility timeout and DLQ for robust integrations.



Use AWS SQS to build **reliable**, **scalable** and **decoupled** messaging solutions and **accelerate** your cloud integrations!





## 7.19

## AWS S3

MODULE 7  
CONNECTORS

Amazon S3 (Simple Storage Service) Connector is used to **store, retrieve and manage** objects in Amazon S3. It provides highly scalable, durable, secure and cost-effective **object storage** in the cloud.

## KEY POINTS

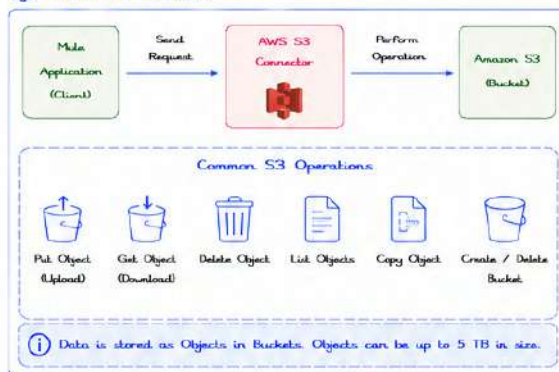
- Used to perform operations on Amazon S3.
- Supports operations: **put** (upload), **get** (download), **delete**, **copy**, **list**, **head**, **create bucket**, **delete bucket**, and more.
- Supports S3 Object and Bucket level operations.
- Supports **metadata**, **ACLs** (Access Control Lists) and storage classes.
- Supports multipart upload for large files.
- Supports **pre-signed URLs** for secure, time-limited access.
- Supports **encryption** (SSE-S3, SSE-KMS, SSE-C).
- Uses AWS Signature Version 4 for secure authentication.
- Ideal for storing files, documents, images, backups, logs, and more.



## WHEN TO USE?

- When you need durable and highly available object storage.
- When systems must store and retrieve large files.
- When you need to share files using pre-signed URLs.
- When you need to serve static content (images, documents, etc.).
- When you need to backup and archive data.
- When you need to integrate with data lakes or analytics.
- When you need cost-effective storage with lifecycle policies.
- When you prefer a fully managed, scalable cloud storage service.

## HOW IT WORKS?



## COMMON CONFIGURATION

| Field                | Description                                              |
|----------------------|----------------------------------------------------------|
| Connection Name      | Name of the AWS S3 connection.                           |
| Region               | AWS Region of the S3 service (e.g., us-east-1).          |
| Access Key ID        | AWS Access Key ID with required permissions.             |
| Secret Access Key    | AWS Secret Access Key.                                   |
| Endpoint (Optional)  | Custom endpoint (for S3 compatible services).            |
| Bucket Name          | Name of the S3 bucket.                                   |
| Operation            | Operation to perform (e.g., putObject, getObject).       |
| Key (Object Key)     | Path and name of the object in the bucket.               |
| Storage Class        | Storage class (STANDARD, IA, GLACIER, etc.).             |
| Content Type         | MIME type of the object.                                 |
| ACL (Optional)       | Access Control List (e.g., private, public-read).        |
| Encryption           | Server-side encryption: SSE-S3, SSE-KMS or SSE-C.        |
| Multipart Upload     | Enable for uploading large files (recommended > 100 MB). |
| Client ID (Optional) | Identifier for AWS SDK client.                           |

## SYNTAX IN MULE.XML

```

<aws-s3:config name="AWS_S3_Config" docname="AWS S3 Config">
  <aws-s3:basic-connection region="us-east-1"
    accessKeyId="${aws.accessKeyId}"
    secretAccessKey="${aws.secretAccessKey}" />
</aws-s3:config>

<!-- Upload Object -->
<aws-s3:put-object config-ref="AWS_S3_Config"
  bucketName="my-bucket"
  key="Folder/data.txt"
  contentType="text/plain"
  sseAlgorithm="AES256">
  <aws-s3:object-content>{[DATA[#{payload}]]}</aws-s3:object-content>
</aws-s3:put-object>

<!-- Download Object -->
<aws-s3:get-object config-ref="AWS_S3_Config"
  bucketName="my-bucket"
  key="Folder/data.txt" />

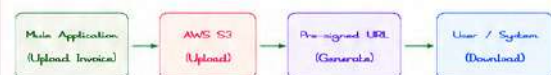
<!-- List Objects -->
<aws-s3:list-objects config-ref="AWS_S3_Config"
  bucketName="my-bucket"
  prefix="Folder/" />

```

*Use DW::InputStream and DW::File for large file handling.*

## REAL-WORLD EXAMPLE

An application uploads invoices to S3 and provides a pre-signed URL for secure download.



## Examples Use Cases:

- Store and retrieve documents, images, videos.
  - Backup and archival of application data.
  - Serve static content (websites, reports, assets).
  - Data exchange between systems.
  - Store logs, audit files and system exports.
- ✓ Scalable, durable and highly available storage.
  - ✓ Pay-as-you-go pricing.
  - ✓ Lifecycle policies to move data to cheaper storage.
  - ✓ Seamless integration with AWS services (Lambda, Glue, Athena, etc.).

## BEST PRACTICES

- ✓ Use appropriate storage class for your use case.
- ✓ Enable lifecycle policies to optimize cost.
- ✓ Use multipart upload for large files.
- ✓ Enable encryption for data at rest.
- ✓ Use pre-signed URLs instead of making objects public.
- ✓ Organize objects with a meaningful key structure (folders).
- ✓ Restrict access using IAM least privilege.
- ✓ Monitor using CloudWatch metrics and S3 server access logs.

## INTERVIEW TIPS

- S3 is an object storage service.
- Data is stored in Buckets and Objects.
- Objects are accessed using a key.
- Strong read-after-write consistency for PUTs and DELETes.
- Supports versioning, encryption and lifecycle policies.
- Max object size is 5 TB.
- Durable (11 9's) and highly available.
- Well-integrated with other AWS services.

## COMMON MISTAKES

- ✗ Hardcoding AWS credentials.
- ✗ Using wrong region or bucket name.
- ✗ Making objects public instead of using pre-signed URLs.
- ✗ Not handling large files with multipart upload.
- ✗ Ignoring lifecycle rules and storage cost.
- ✗ Not setting correct content type.
- ✗ Not handling access denied errors.

## KEY TAKEAWAY

AWS S3 Connector in MuleSoft enables you to **store, retrieve and manage** objects in Amazon S3 securely and efficiently. It supports a wide range of operations and features that help you build scalable, cost-effective and future-ready integrations.



Use AWS S3 to **store** data **securely**, **scale** effortlessly and **reduce costs** while integrating seamlessly with the AWS ecosystem!





**MODULE**  
**8**

# 8.1 API AUTODISCOVERY



API Autodiscovery is a feature in MuleSoft API Manager that **connects** a **deployed** Mule application with an **API instance** in API Manager.

Once connected, API Manager can **apply policies**, **monitor** and **manage** the API.

## WHAT IS API AUTODISCOVERY?

- It automatically links your deployed Mule API with API Manager using a unique API ID.
- After connection, API Manager can identify the API and manage its lifecycle.
- No changes are required in the application code for management.
- It works for APIs deployed on CloudHub or Runtime Fabric.

### Simple Definition:

API Autodiscovery = Linking your deployed Mule API with API Manager.

## CONFIGURATION EXAMPLE

```
<api-gateway:autodiscovery
  apiId="12345678" />
```

Here,

- apiId** = Unique API ID generated by API Manager.

## HOW API AUTODISCOVERY WORKS

- Design API**  
Create API specification (RAML).
- Publish API**  
Publish the API to Anypoint Exchange.
- Create API Instance**  
Create an API instance in API Manager.
- Copy API ID**  
Copy the unique API ID from API Manager.
- Add API Autodiscovery**  
Add the API ID in Mule application using api-autodiscovery element.
- Deploy Application**  
Deploy the Mule application to runtime.
- API Registered**  
API is automatically discovered and registered in API Manager.

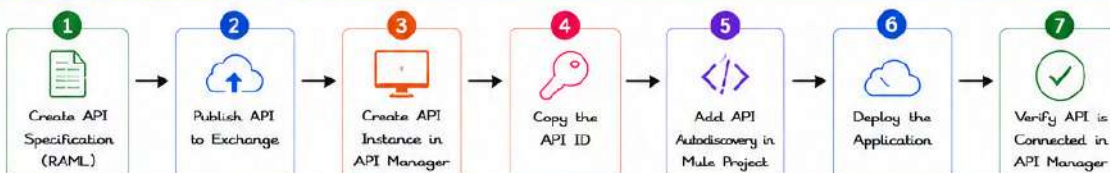
## WHY DO WE USE IT?

| Without Autodiscovery                               | With Autodiscovery                         |
|-----------------------------------------------------|--------------------------------------------|
| API runs but API Manager cannot identify it.        | API Manager recognizes the API.            |
| Cannot apply policies (Security, Rate limit, etc.). | Policies can be applied easily.            |
| No analytics and monitoring.                        | Analytics and monitoring become available. |
| Cannot enforce SLA and Client ID.                   | SLA and Client ID Enforcement work.        |
| Manual tracking is needed.                          | Centralized management and visibility.     |

## WHAT HAPPENS AFTER AUTODISCOVERY?

- ✓ API is visible in API Manager.
- ✓ Policies can be applied (OAuth, Rate Limiting, Spike Control, IP Whitelisting, etc.).
- ✓ Real-time analytics and monitoring.
- ✓ SLA and Client ID Enforcement.
- ✓ Manage API Lifecycle (versions, endpoints, etc.).

## STEPS TO CONFIGURE API AUTODISCOVERY



## REAL-WORLD EXAMPLE

Suppose an Order API is deployed on CloudHub.

**Without Autodiscovery:**

- API works.
- No security.
- No monitoring.

**After enabling Autodiscovery:**

- API appears in API Manager.
- Client ID policy is applied.
- Analytics dashboard shows traffic.
- API can be managed centrally.



## ADVANTAGES

- ✓ Easy API management.
- ✓ Centralized monitoring.
- ✓ Easy policy application.
- ✓ Better API security.
- ✓ Real-time analytics.
- ✓ No code changes for policies.
- ✓ Supports CloudHub and Runtime Fabric.

## INTERVIEW TIP

**Q: What is API Autodiscovery?**

**Answer:**

API Autodiscovery is a MuleSoft feature that links a deployed Mule application with an API instance in API Manager using an API ID. After the connection is established, API Manager can apply policies, monitor traffic, and manage the API without modifying the application code.

## COMMON MISTAKES

- ✗ Using wrong API ID.
- ✗ API not deployed or not running.
- ✗ API Manager instance not created.
- ✗ Incorrect environment selected.
- ✗ API Gateway module missing in project.

## USE CASES

- ◆ Discover existing APIs in legacy systems.
- ◆ Identify and govern shadow APIs.
- ◆ Migrate unmanaged APIs to API Manager.
- ◆ Maintain an up-to-date API inventory.
- ◆ Gain visibility for analytics and security.

## KEY TAKEAWAY

API Autodiscovery is simply the process of connecting a deployed Mule API with API Manager using an API ID. Once connected, API Manager can **secure**, **monitor**, and **manage** the API centrally without changing the application logic.



Use API Autodiscovery to **discover**, **govern** and unlock the full **value** of your APIs!



## MODULE 8

# 8.2 CLIENT ID ENFORCEMENT



Client ID Enforcement is a policy in MuleSoft API Manager that ensures every request to an API must contain a valid **Client ID**.

It helps to **identify and track** API consumers and **control access** to APIs.

### WHAT IS CLIENT ID ENFORCEMENT?

- It is a policy that requires a valid Client ID in every API request.
- The Client ID is issued to an application (consumer) by API Manager.
- It helps to identify the calling application.
- It is the first policy applied in the API policy chain.
- It works for both application (OAuth) and anonymous clients.

### HOW CLIENT ID ENFORCEMENT WORKS?



### HOW TO CONFIGURE?

- Create an API Instance in API Manager.
- Create or register an application (Client).
- In the application, note down the Client ID.
- Go to your API instance → Policies → Add Policy → Compliance → Client ID Enforcement.
- Select the application(s) allowed to access the API.
- Save and deploy the policy.

Once enabled, every request must contain the Client ID in the header.

Where to pass Client ID?

Add the Client ID in the HTTP header:

**Client-Id:** your\_client\_id

### EXAMPLE REQUEST

#### Request with Client ID

```
GET /api/orders HTTP/1.1
Host: apimycountry.com
Client-Id: 1256ab34-5678-90cd-ef12-34567890abcd
Accept: application/json
```

Response: 200 OK

#### Request without or invalid Client ID

```
GET /api/orders HTTP/1.1
Host: apimycountry.com
Accept: application/json
```

Response: 401 Unauthorized

```
{
  "message": "Client ID is missing or invalid",
  "errorCode": "CLIENT_ID_ENFORCEMENT"
}
```

### POLICY BEHAVIOR

#### If Client ID is Valid

- Request is allowed.
- API is executed.
- API Manager tracks the consumer.

#### If Client ID is Missing

- Request is rejected.
- Response: 401 Unauthorized.
- Error: CLIENT\_ID\_ENFORCEMENT

#### If Client ID is Invalid

- Request is rejected.
- Response: 401 Unauthorized.
- Error: INVALID\_CLIENT

### REAL-WORLD EXAMPLE

Company ABC has an Order API. They want to allow only their mobile app and website to access it.

#### Steps:

- Create API instance for Order API.
- Create two applications: Mobile App and Web Portal.
- Note down Client ID for both applications.
- Apply Client ID Enforcement policy and select these applications.
- Now, only requests from these applications with valid Client ID will be allowed.



### BEST PRACTICES

- Always apply Client ID Enforcement as the first policy.
- Use meaningful application names.
- Rotate Client IDs if an application is compromised.
- Use separate applications for dev, test and prod.
- Monitor analytics to track API usage by consumers.
- Combine with other policies for better security.

### COMMON MISTAKES

- Forgetting to include Client ID in the request header.
- Using wrong Client ID.
- Applying policy but not selecting any application.
- Not deploying the API after applying the policy.
- Testing with Postman without adding Client ID.

### USE CASES

- Identify and track API consumers.
- Control which applications can access the API.
- Generate accurate analytics and reports.
- Enforce SLAs and rate limits per client.
- Prevent misuse by unauthorized applications.

### INTERVIEW TIP

Q: What is Client ID Enforcement?

Ans: It is an API Manager policy that ensures every API request must contain a valid Client ID. It helps to identify the calling application and control access to the API. If the Client ID is missing or invalid, the request is rejected with 401 Unauthorized.



### KEY TAKEAWAY

Client ID Enforcement is the gatekeeper of your API. It ensures that only known and approved applications can access your API, helping you to secure, **monitor** and **manage** your APIs effectively.



Always enforce Client ID to **know your consumer**, **protect your API** and get **accurate analytics**!



## MODULE 8

## 8.3 OAUTH 2.0



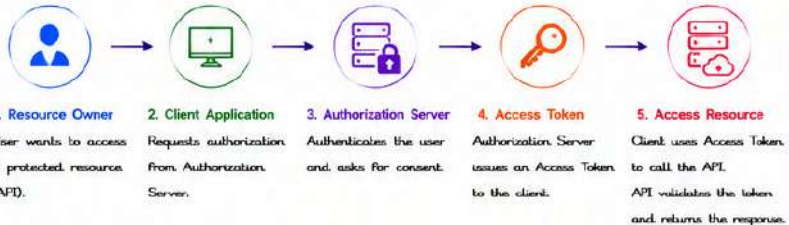
OAuth 2.0 is an **authorization** framework that enables applications to obtain limited access to user resources on an HTTP service, without sharing the user's credentials.

In MuleSoft API Manager, OAuth 2.0 policy **secures** APIs by issuing **access tokens** to valid clients and validating them on each request.

## WHAT IS OAUTH 2.0?

- OAuth 2.0 is an authorization protocol, not an authentication protocol.
- It allows third-party applications to access resources on behalf of a user.
- It uses Access Token instead of username and password.
- It is widely used by APIs and web services.
- In API Manager, it is implemented using OAuth 2.0 policy.

## HOW OAUTH 2.0 WORKS (AUTHORIZATION CODE FLOW)



## KEY COMPONENTS

-  **Resource Owner (User)**  
The end user who owns the data.
-  **Client Application**  
Application that requests access to the resource.
-  **Authorization Server**  
Server that authenticates the user and issues access tokens.
-  **Access Token**  
Token issued to the client to access the API.
-  **Resource Server (API)**  
API that hosts the protected resources and validates the token.

## OAUTH 2.0 FLOWS SUPPORTED IN API MANAGER

- Authorization Code Grant (Most Common)
- Client Credentials Grant
- Resource Owner Password Credentials Grant
- Implicit Grant
- Refresh Token

Most Used Flow: Authorization Code Grant (3-Legged)

- Secure and recommended for web & mobile apps.
- User authenticates and grants access.
- Access Token is used to call the API.

## OAUTH 2.0 IN API MANAGER

- Use OAuth 2.0 policy to protect APIs.
- API Manager acts as Authorization Server.
- Clients obtain Access Token from API Manager's Token Endpoint.
- All API requests must include a valid Access Token.
- Token can be in Authorization header or Query Parameter.

## OAUTH 2.0 POLICY IN API MANAGER

When enabled, the policy:

- Authenticates the client.
- Validates the access token.
- Checks scopes (if configured).
- Allows or denies the request.
- Provides token introspection.



### EXAMPLE REQUEST

### API call with Access Token

```
GET /api/orders HTTP/1.1
Host: apimycorpany.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Accept: application/json
```

Response: 200 OK

### TOKEN ENDPOINT (EXAMPLE)

```
POST /token HTTP/1.1
Host: your-org.auth.apimanager.io
Content-Type: application/x-www-form-urlencoded
grant_type=authorization_code
code=XXXXXXXX
client_id=your_client_id
client_secret=your_client_secret
redirect_uri=https://myapp.com/callback
Response: Access Token + Refresh Token
```

## OAUTH 2.0 CONFIGURATION STEPS (AUTHORIZATION CODE FLOW)



## USE CASES

- Mobile applications accessing enterprise APIs.
- Web applications accessing user data.
- Partner applications with delegated access.
- Secure access to cloud-based APIs.
- Third-party integrations.

## BENEFITS

- ✔ Secure delegated access.
- ✔ No need to share user credentials.
- ✔ Supports multiple client types.
- ✔ Granular access with scopes.
- ✔ Access tokens can be revoked.

## COMMON MISTAKES

- ✖ Exposing client secret in public apps.
- ✖ Not validating token on API.
- ✖ Using long token expiry unnecessarily.
- ✖ Not defining proper scopes.
- ✖ Allowing wildcard redirect URIs.

**INTERVIEW TIP**

Q: What is OAuth 2.0?

**Ans:** OAuth 2.0 is an authorization framework that allows applications to access resources on behalf of a user using access tokens instead of user credentials. It is used to grant limited access securely to APIs.

### EXAMPLE SCENARIO

A mobile app wants to access the "Order API".

1. User logs in to the app.
2. App redirects user to API Manager authorization page.
3. User authenticates and grants consent.
4. App receives Authorization Code.
5. App exchanges code for Access Token.
6. App calls API with Access Token.
7. API validates token and returns data.



 **KEY TAKEAWAY**

OAuth 2.0 policy in API Manager  
 secures your APIs by ensuring only  
 authorized clients with valid access  
 tokens can access the resources.  
 Always use proper flows, short-  
 lived tokens, and scopes for a  
 secure and controlled access.

- Use OAuth 2.0 to **authorize**, not **authenticate**. It keeps your APIs **secure**, **scalable** and **user-friendly**!



## MODULE 8

# 8.4 JWT (JSON WEB TOKEN)



JWT (JSON Web Token) is a compact, self-contained way to securely transmit information between parties as a JSON object.

In MuleSoft API Manager, JWT policy is used to **validate tokens** in API requests and **secure APIs** without maintaining server-side sessions.

### WHAT IS JWT?

- JWT is an open standard (RFC 7519) for securely transmitting information.
- It is a token-based authentication mechanism.
- It contains claims (information) about the user or client.
- Token is digitally signed by the issuer and verified by the API Manager.
- Stateless: No session is stored on the server.
- Widely used in modern APIs and microservices.

### HOW JWT WORKS (HIGH LEVEL FLOW)



### STRUCTURE OF JWT

JWT consists of three parts separated by dots (.)



1. **Header:** Contains token type and signing algorithm.
2. **Payload:** Contains claims (data).
3. **Signature:** Verifies the token is not tampered and is from a trusted issuer.

### TYPES OF CLAIMS IN JWT PAYLOAD

#### Registered Claims (Standard)

- **iss** : Issuer
- **sub** : Subject (user or client)
- **aud** : Audience (API / resource)
- **exp** : Expiration time
- **nbf** : Not before
- **iat** : Issued at
- **jti** : JWT ID

#### Custom Claims (Application specific)

- **role** : User role
- **scope** : Permissions / scopes
- **client\_id** : Client identifier
- **email** : User email
- **tenant** : Tenant ID
- any other custom information

### ADVANTAGES OF JWT

- ✓ Stateless and scalable.
- ✓ Compact and URL-safe.
- ✓ Can be used across domains.
- ✓ Supports fine-grained authorization.
- ✓ No server-side session storage.
- ✓ High performance.

### JWT POLICY IN API MANAGER

JWT policy in API Manager is used to:

- Validate incoming JWT tokens.
- Ensure token is issued by trusted issuer.
- Validate signature using public key or secret.
- Validate expiration and other claims.
- Allow or deny requests based on validation.



### WHERE TO SEND JWT?

Send JWT in the Authorization header:

**Authorization:** Bearer <JWT\_TOKEN>

Example:

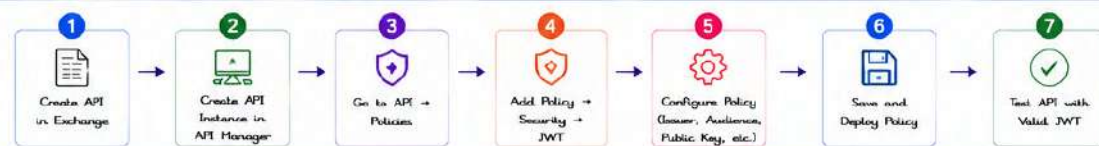
```
GET /api/orders HTTP/1.1
Host: apimycorpany.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...
```

### JWT VALIDATION IN API MANAGER

API Manager validates:

- Token signature (using public key or secret).
- Issuer (iss) claim.
- Audience (aud) claim.
- Expiration time (exp).
- Not before time (nbf).
- Any custom claims (if configured).

### CONFIGURING JWT POLICY IN API MANAGER



### SAMPLE JWT (DECODED PAYLOAD)

```
{
  "iss": "https://auth.mycompany.com",
  "sub": "user-123",
  "aud": "https://api.mycompany.com",
  "iat": 1715847000,
  "exp": 1715850600,
  "role": "admin",
  "scope": "read write"
}
```

### SAMPLE JWT POLICY CONFIGURATION

|                       |                            |
|-----------------------|----------------------------|
| Issuer (iss)          | https://auth.mycompany.com |
| Audience (aud)        | https://api.mycompany.com  |
| Token Signing Method  | RS256                      |
| Public Key            | (paste public key)         |
| Expiration Validation | Enabled                    |
| Required Claims       | sub, iss, aud, exp         |
| Additional Conditions | scope contains read        |

### REAL-WORLD EXAMPLE

1. A mobile app wants to access the Order API.
2. User logs in to the app.
3. Auth server validates user and returns JWT.
4. App stores JWT securely.
5. App calls Order API with JWT in header.
6. API Manager validates JWT.
7. If valid -> request is allowed.
8. If invalid -> request is rejected (401 Unauthorized).

### USE CASES

- Securing APIs for mobile and web applications.
- Microservices to microservices authentication.
- Single Sign-On (SSO) implementations.
- Partner and B2B API access.
- Serverless and cloud-native applications.

### COMMON MISTAKES

- ✗ Using wrong public key or secret.
- ✗ Not validating token expiration.
- ✗ Ignoring audience (aud) validation.
- ✗ Allowing tokens without proper claims.
- ✗ Sharing tokens between different APIs.

### INTERVIEW TIP

**Q:** What is JWT in MuleSoft API Manager?  
**A:** JWT (JSON Web Token) is a token format used to securely transmit claims between parties. In API Manager, JWT policy is used to validate the token's signature, issuer, audience, and expiry before allowing access to the API.

### ★ KEY TAKEAWAY

JWT policy in API Manager helps you secure your APIs by validating the identity and claims of the client through a signed token. It is stateless, scalable and widely used for modern API security.



Use JWT to **authenticate** and **authorize** your clients **securely** and **efficiently**!



## MODULE 8

# 8.5 BASIC AUTHENTICATION



Basic Authentication is a simple authentication method where the client sends a **username** and **password** with every request, encoded in **Base64**.  
In MuleSoft API Manager, Basic Authentication policy is used to **protect APIs** by validating users against an **internal user store** or **external identity provider**.

### WHAT IS BASIC AUTHENTICATION?

- It is an HTTP authentication method defined in RFC 7617.
- Client sends username and password in the Authorization header.
- The server validates the credentials.
- Simple and easy to implement.
- It must be used over HTTPS (SSL) for security.

### HOW BASIC AUTHENTICATION WORKS



### HOW CLIENT SENDS CREDENTIALS?

- Client sends credentials in HTTP header using Base64 encoding.
- Header format:

Authorization: Basic <Base64(username:password)>

- Example:

If username = john and password = 1234

Base64(john:1234) = am9objoxMjM0

Header:

Authorization: Basic am9objoxMjM0

### EXAMPLE REQUEST

```
GET /api/orders HTTP/1.1
Host: apimycorpany.com
Authorization: Basic am9objoxMjM0
Accept: application/json
```

If credentials are valid → 200 OK

If credentials are invalid → 401 Unauthorized

### BASIC AUTHENTICATION POLICY

In API Manager, Basic Authentication policy

- Extracts username and password from Authorization header.
- Validates against Internal or External User Store.
- Allows request if valid.
- Rejects request if invalid.
- Returns 401 Unauthorized for invalid credentials.

### CONFIGURING BASIC AUTHENTICATION

- Create API Instance in API Manager.
- Go to API → Policies → Add Policy.
- Select "Authentication" → "Basic Authentication".
- Select or create a User Store.
- Add the policy to the desired methods or API.
- Save and deploy the policy.

User Store can be: Internal, LDAP, Active Directory, or Custom (Database).

### USER STORE OPTIONS

- Internal User Store**  
Users are created and managed inside API Manager.
- LDAP / Active Directory**  
Users are validated from enterprise directory services.
- Custom Database**  
Users are validated from a custom database using connector.

### RESPONSE CODES

- 200 OK**  
Authentication successful and request allowed.
- 401 Unauthorized**  
Missing or invalid username/password.
- 403 Forbidden**  
Authenticated but not authorized. (if used with other policies).

### CONFIGURATION STEPS (VISUAL FLOW)



### REAL-WORLD EXAMPLE

A Company has an internal portal that calls an Order API. Only employees should access the API.

Steps:

- Admin creates users in Internal User Store.
- Basic Authentication policy is applied to Order API.
- Employee app sends username and password.
- API Manager validates and allows the request.
- API returns order details.



### ADVANTAGES

- Simple to implement.
- Supported by all HTTP clients.
- No tokens to manage.
- Works well for internal and trusted applications.
- Low overhead.

### LIMITATIONS

- Credentials are sent with every request.
- Not suitable if not used over HTTPS.
- Not ideal for public APIs.
- Less secure compared to OAuth 2.0 or JWT.

### USE CASES

- Internal and trusted applications.
- Server-to-server communication.
- Quick testing using tools like Postman.
- Legacy systems where token-based auth is not required.

### BEST PRACTICES

- Always use HTTPS.
- Use strong passwords.
- Limit access using IP restrictions.
- Use for internal or trusted clients only.
- Prefer OAuth 2.0 or JWT for public APIs.

### COMMON MISTAKES

- Using Basic Auth over HTTP (not HTTPS).
- Hardcoding credentials in applications.
- Using weak or default passwords.
- Enabling it for public APIs.

### INTERVIEW TIP

Q. What is Basic Authentication?

Ans: Basic Authentication is an HTTP authentication method where the client sends username and password in Base64 format in the Authorization header. API Manager validates the credentials using a user store and allows or rejects the request.



Basic Authentication is simple but must be used **securely**. Always prefer **HTTPS** and **strong credentials**!



## MODULE 8

# 8.6 RATE LIMITING

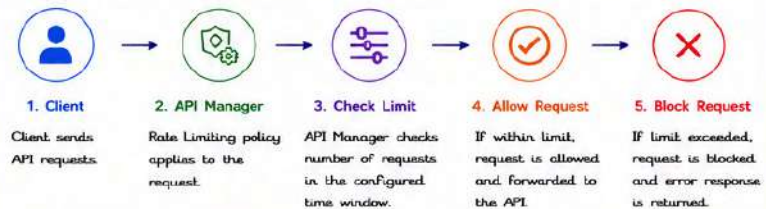


Rate Limiting is a policy in MuleSoft API Manager that controls the number of API requests a client can make within a specific time period. It helps protect APIs from abuse, ensures fairness, prevents server overload, and improves overall performance.

### WHAT IS RATE LIMITING?

- It restricts the number of requests a client can make in a given time window.
- If the limit is exceeded, further requests are blocked until the next time window.
- Configured at API, application or client level.
- Helps in better resource utilization and protects backend systems.

### HOW RATE LIMITING WORKS



### RATE LIMITING TYPES

- SLA Based:** Limit is based on SLA tiers (e.g., Gold, Silver, Bronze).
- Application Based:** Limit is applied to an application (all clients of that app).
- Client Based:** Limit is applied to individual clients.
- IP Based:** Limit is applied based on the IP address.
- Custom Header Based:** Limit using a custom header value.

### KEY CONCEPTS

- Time Window:** The duration for which the limit is applicable (e.g., 1 minute, 1 hour, 1 day).
- Request Limit:** Maximum number of requests allowed in the time window.
- Counter:** API Manager keeps track of requests made within the time window.
- Limit Exceeded:** If requests > limit, further requests are blocked until next window.
- Response:** A 429 (Too Many Requests) or 403 error is returned.

### EXAMPLE

- Limit: 100 requests per minute per client  
Time Window: 1 minute
- If a client sends 80 requests in 1 minute → **Allowed**
  - If the client sends 120th request in the same minute → **Blocked**
  - Response: **429 Too Many Requests**

### CONFIGURING RATE LIMITING IN API MANAGER

- Create or Select an API.
- Go to API → Policies → Add Policy.
- Select "Rate Limiting".
- Choose the Rate Limiting type (SLA / Application / Client / IP / Custom).
- Configure:
  - Time Window
  - Request Limit
- Apply the policy to the required methods or API.
- Save and deploy the policy.

### SAMPLE CONFIGURATION (SLA BASED)

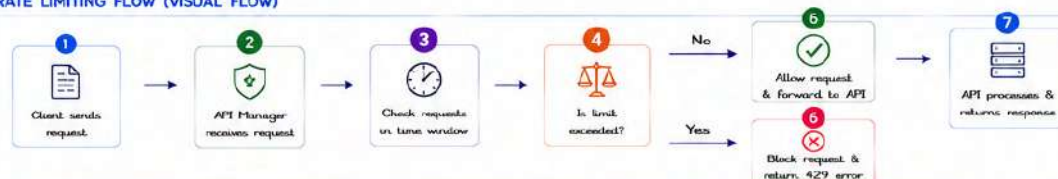
| SLA Tier | Time Window | Request Limit |
|----------|-------------|---------------|
| Gold     | 1 Minute    | 1000          |
| Silver   | 1 Minute    | 500           |
| Bronze   | 1 Minute    | 100           |

- Note:**
- Each time window has its own counter.
  - After window resets, the counter is set to 0.

### RESPONSE CODES

- 200 OK**  
Request is allowed.
- 429 Too Many Requests**  
Rate limit exceeded. Try again later.
- 403 Forbidden**  
Request blocked (if configured).

### RATE LIMITING FLOW (VISUAL FLOW)



### REAL-WORLD EXAMPLE

- An e-commerce API allows 500 requests per minute per application.
- Mobile App sends 450 requests in 1 minute → **Allowed**
  - Web App sends 550th request in same minute → **Blocked**
  - API Manager returns 429 Too Many Requests



### BEST PRACTICES

- Set limits based on API capacity.
- Use appropriate time windows.
- Apply limits at the right level (SLA/App/Client/IP).
- Monitor analytics to fine-tune limits.
- Communicate limits to API consumers.
- Combine with other policies (Spike Control, Throttling) for better protection.

### COMMON MISTAKES

- Setting very low limits (affects legitimate users).
- Setting very high limits (no protection).
- Wrong time window configuration.
- Not communicating limits to consumers.
- Applying on unwanted clients or methods.

### USE CASES

- Prevent API abuse and DDoS attacks.
- Ensure fair usage among consumers.
- Protect backend systems from overload.
- Enforce subscription or SLA limits.
- Control costs in multi-tenant systems.

### INTERVIEW TIP

- Q:** What is Rate Limiting in API Manager?
- A:** Rate Limiting is a policy that restricts the number of requests a client can make in a specific time window. If the limit is exceeded, API Manager blocks the request and returns a 429 error.

### KEY TAKEAWAY

Rate Limiting helps you control API usage by limiting the number of requests in a time window. It protects your APIs, ensures fair usage, and improves performance.



Use Rate Limiting to **protect** your APIs, ensure **fair access**, and deliver a **reliable experience** to all consumers.



## MODULE 8

# 8.7 SPIKE CONTROL



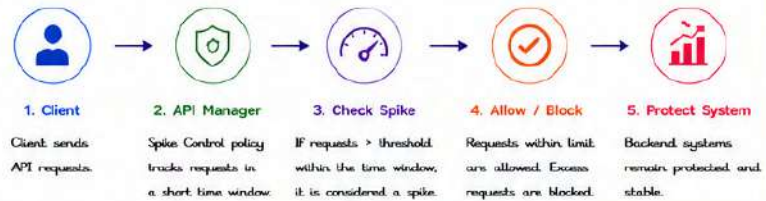
Spike Control is a policy in MuleSoft API Manager that protects APIs from sudden bursts of traffic (spikes) within a short period of time.

It helps **prevent system overload** and ensures **service availability** during traffic spikes.

### WHAT IS SPIKE CONTROL?

- Spike Control detects sudden increases in requests within a very short time window.
- If the number of requests in that window exceeds the configured threshold, the excess requests are blocked.
- It protects the backend systems from unexpected traffic bursts.
- It is applied at API, Application or Client level.

### HOW SPIKE CONTROL WORKS



### KEY CONCEPTS

- **Time Window:** Very short duration to detect spikes (e.g., 1 second, 5 seconds).
- **Request Limit:** Maximum number of requests allowed in the time window.
- **Counter:** API Manager counts requests in each time window.
- **Spike:** When request count > limit.
- **Action:** Block excess requests until window resets.

### SPIKE CONTROL TYPES

- **Fixed Window:** Requests are counted in fixed intervals.
- **Sliding Window:** More accurate. Window moves with each request.

### EXAMPLE

- Spike Limit: 50 requests per 2 seconds
- Client sends 40 requests in 2 seconds → **Allowed**
  - Client sends 70 requests in 2 seconds → First 50 allowed, remaining 20 blocked
  - Response for blocked requests:  
**429 Too Many Requests**

### CONFIGURING SPIKE CONTROL IN API MANAGER

1. Create or Select an API.
2. Go to API → Policies → Add Policy.
3. Select "Spike Control".
4. Choose the level: SLA / Application / Client / IP.
5. Configure:
  - Time Window (e.g., 2 seconds)
  - Request Limit (e.g., 50)
6. Apply this policy to the required methods or API.
7. Save and deploy the policy.

### SAMPLE CONFIGURATION

|               |                       |
|---------------|-----------------------|
| Time Window   | 2 Seconds             |
| Request Limit | 50 Requests           |
| Action        | Block Excess Requests |

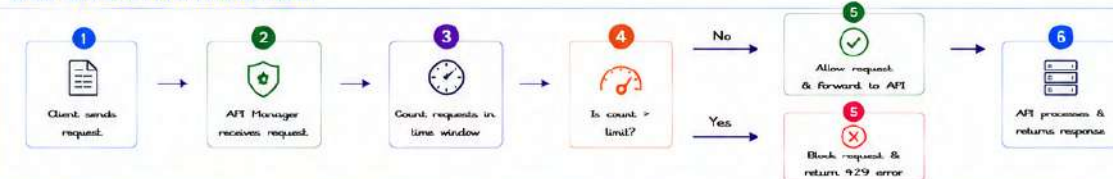
#### Note:

- Counter resets after every window.
- Only requests above the limit are blocked.

### RESPONSE CODES

- ✓ 200 OK  
Request is allowed.
- ✗ 429 Too Many Requests  
Spike limit exceeded.
- 403 Forbidden  
Request blocked (if configured).

### SPIKE CONTROL FLOW (VISUAL FLOW)



### REAL-WORLD EXAMPLE

- An e-commerce API allows 50 requests per 2 seconds.
- Normal traffic: 30 requests / 2 sec → **Allowed**
  - During flash sale: 120 requests / 2 sec → First 50 allowed, remaining 70 blocked
- Result: Backend system remains stable and available to all users.



### BEST PRACTICES

- ✓ Set appropriate time windows (very short).
- ✓ Configure limits based on API capacity.
- ✓ Test with high traffic scenarios.
- ✓ Use analytics to identify spike patterns.
- ✓ Combine with Rate Limiting for better control.

### COMMON MISTAKES

- ✗ Setting very high limits (no protection).
- ✗ Setting very low limits (legitimate users blocked).
- ✗ Wrong time window configuration.
- ✗ Not testing with real traffic.
- ✗ Applying on unwanted clients or methods.

### USE CASES

- Protect APIs during flash sales or promotions.
- Prevent abuse from bots or malicious users.
- Protect backend systems from sudden load.
- Ensure availability during unexpected spikes.
- Maintain performance and user experience.

### INTERVIEW TIP

Q: What is Spike Control in API Manager?

A: Spike Control is a policy that protects APIs from sudden bursts of traffic. It monitors the number of requests in a very short time window and blocks requests that exceed the configured threshold, returning a 429 error.

### KEY TAKEAWAY

Spike Control protects your APIs from sudden traffic bursts by limiting the number of requests allowed in a very short time window, ensuring stability and availability.



Use Spike Control to **protect** your APIs from **sudden** traffic bursts and keep your systems **stable** and **reliable**!

## MODULE 8

# 8.8 IP WHITE LISTING

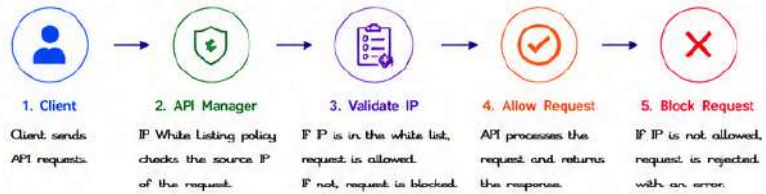


IP White Listing is a policy in MuleSoft API Manager that allows requests only from specific IP addresses or IP ranges. Requests from IPs not in the list are blocked. It helps protect APIs by restricting access to trusted sources and preventing unauthorized usage.

### WHAT IS IP WHITE LISTING?

- It allows access to an API only from predefined IP addresses or IP ranges.
- If a request comes from an IP not in the list, it will be blocked.
- It adds an extra layer of security by allowing only trusted systems.
- It is applied at API, Application or Client level.

### HOW IP WHITE LISTING WORKS



### KEY CONCEPTS

- IP Address:** Unique address of a client.
- IP Range (CIDR):** A range of IPs defined using CIDR notation. Example: 192.168.10/24
- Whitelist:** List of allowed IPs or IP ranges.
- Block:** Requests from IPs not in the whitelist.
- IPv4 and IPv6:** Both are supported.

### TYPES OF IP WHITE LISTING

- Single IP:** Allow one IP address.  
Example: 203.0.113.10
- IP Range:** Allow a range of IP addresses.  
Example: 203.0.113.0/24
- Multiple IPs:** Allow multiple IP addresses or ranges.
- IPv6 Address:** Allow IPv6 addresses or ranges.

### EXAMPLE

Whitelist contains:

- 203.0.113.10
- 198.51.100.0/24
- 2001:db8:10

If request comes from 203.0.113.10 → **Allowed**  
 If request comes from 198.51.100.25 → **Allowed**  
 If request comes from 10.1.1.5 → **Blocked**  
 Response for blocked requests: **403 Forbidden**

### CONFIGURING IP WHITE LISTING IN API MANAGER

- Create or Select an API.
- Go to API → Policies → Add Policy.
- Select "IP White Listing".
- Choose the level: SLA / Application / Client / IP.
- Add allowed IP addresses or IP ranges.
- Save and deploy the policy.
- Test the API with allowed and non-allowed IPs.

### SAMPLE CONFIGURATION

#### Allowed IPs / Ranges

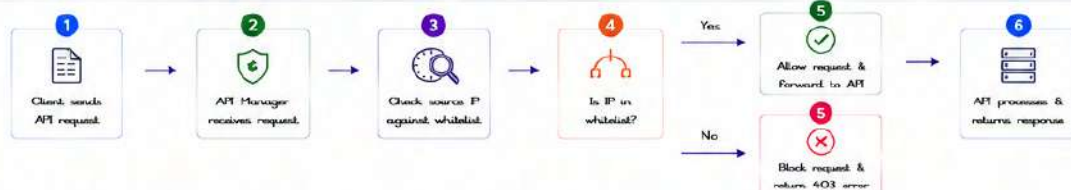
| Type            | Value              |
|-----------------|--------------------|
| Single IP       | 203.0.113.10       |
| IP Range        | 198.51.100.0/24    |
| IPv6 Address    | 2001:db8:10        |
| IP Range (IPv6) | 2001:db8:abcd::/48 |

**Note:**  
You can add multiple IPs and ranges.  
CIDR notation is required for ranges.

### RESPONSE CODES

- 200 OK**  
IP is allowed. Request is processed.
- 403 Forbidden**  
IP not allowed. Access is denied.

### IP WHITE LISTING FLOW (VISUAL FLOW)



### REAL-WORLD EXAMPLE

An enterprise exposes an Order API only for its internal systems.

- Allowed IPs: 10.10.1.5, 10.10.1.6
- IP Range: 10.10.2.0/24

Requests from these IPs → **Allowed**  
 Requests from any other IP → **Blocked (403 Forbidden)**



### BEST PRACTICES

- Use IP white listing for partner or internal system APIs.
- Keep the whitelist updated.
- Use IP ranges instead of multiple single IPs (where possible).
- Test thoroughly after adding or updating IPs.
- Combine with other policies (OAuth, Rate Limiting) for stronger security.

### COMMON MISTAKES

- ✗ Forgetting to add all required IPs.
- ✗ Using wrong IP or CIDR notation.
- ✗ Blocking your own IP while testing.
- ✗ Relying only on IP filtering without other security policies.
- ✗ Not considering dynamic IPs (use ranges if needed).

### USE CASES

- Restrict API access to corporate networks.
- Allow access only from partner IPs.
- Protect admin or critical APIs.
- Prevent unauthorized external access.
- Comply with security and compliance rules.

### INTERVIEW TIP

**Q:** What is IP White Listing in API Manager?  
**A:** IP White Listing is a policy that allows requests only from specific IP addresses or IP ranges. Requests from IPs not in the whitelist are blocked with 403 Forbidden.

### KEY TAKEAWAY

IP White Listing ensures that only trusted IP addresses can access your APIs. It is simple, effective, and an important layer of security in MuleSoft API Manager.



Use IP White Listing to allow only **trusted sources** and keep your APIs **secure**!



## MODULE 8

# 8.9 SLA TIERS

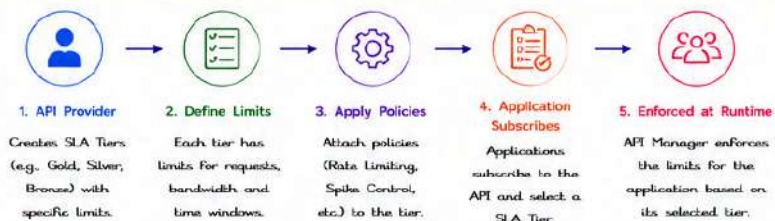


SLA Tiers in MuleSoft API Manager allow you to create different service levels for your APIs. Each tier defines limits (requests, bandwidth, time windows) and policies that apply to API consumers (applications / clients). It helps in offering **differentiated access** and ensuring **fair usage** of APIs.

### WHAT IS SLA TIERS?

- SLA (Service Level Agreement) is a contract between API provider and consumer.
- An SLA Tier is a set of limits and policies that define how an API can be used.
- Different tiers can have different limits and policies.
- SLA Tiers are applied to Applications when they subscribe to an API.

### HOW SLA TIERS WORK



### KEY CONCEPTS

- **Tier:** A plan that defines usage limits and policies.
- **Limit:** Maximum allowed usage (requests, bandwidth, etc.).
- **Policy:** Rules applied to the tier (Rate Limiting, Spike Control...).
- **Application:** The consumer app that subscribes to the API and selects a tier.

### TYPES OF LIMITS IN SLA TIERS

- **Rate Limit:** Number of requests allowed in a time window.
- **Bandwidth Limit:** Amount of data allowed in a time window.
- **Concurrent Connections:** Number of simultaneous connections allowed.
- **Quota Limit:** Total number of requests allowed in a period (daily/monthly).
- **Custom Limits:** Based on business needs.

### EXAMPLE

An API has 3 SLA Tiers:

| Tier   | Requests Limit | Time Window | Bandwidth Limit |
|--------|----------------|-------------|-----------------|
| Gold   | 5000           | 1 Minute    | 1 GB            |
| Silver | 1000           | 1 Minute    | 200 MB          |
| Bronze | 100            | 1 Minute    | 50 MB           |

If an application subscribed to Silver tier, it can make max 1000 requests per minute and 200 MB bandwidth per minute.

### SLA TIERS IN API MANAGER – CONFIGURATION STEPS

1. Create or select an API.
2. Go to API Manager → Policies → Add Policy.
3. Select "SLA Tier".
4. Define the SLA Tier (name, description, limits).
5. Add required policies to the tier (Rate Limiting, etc.).
6. Save the policy.
7. Deploy the policy.

### SAMPLE SLA TIER CONFIGURATION

| Field           | Example Value                      |
|-----------------|------------------------------------|
| Tier Name       | Silver                             |
| Description     | Standard access for regular users. |
| Rate Limit      | 1000 requests                      |
| Time Window     | 1 Minute                           |
| Bandwidth Limit | 200 MB                             |
| Spike Control   | 100 requests per 2 sec             |
| Quota Limit     | 10000 requests per day             |

### RESPONSE CODES

- ✓ **200 OK**  
Request is within the SLA Tier limits.
- ! **429 Too Many Requests**  
Rate limit exceeded.
- ✗ **403 Forbidden**  
SLA Tier limit (quota/bandwidth) exceeded. Access denied.

### SLA TIERS FLOW (VISUAL FLOW)



### REAL-WORLD EXAMPLE

A Video Streaming API offers 3 tiers to its partners.

- Gold Tier → For premium partners (high limits)
- Silver Tier → For regular partners (medium limits)
- Bronze Tier → For trial users (low limits)

This ensures fair usage and protects the backend from overload.



### BEST PRACTICES

- ✓ Design tiers based on business needs.
- ✓ Keep limits realistic and fair.
- ✓ Use proper time windows.
- ✓ Combine SLA Tiers with Rate Limiting, Spike Control, and Quota policies.
- ✓ Monitor usage and adjust limits when required.

### COMMON MISTAKES

- ✗ Setting limits too high or too low.
- ✗ Not considering backend capacity.
- ✗ Not applying proper policies to the tier.
- ✗ Not monitoring tier usage.
- ✗ Not communicating limits to consumers.

### USE CASES

- Different plans for different customers.
- Protect backend from heavy users.
- Offer premium plans with higher limits.
- Trial or free tier with limited access.

### INTERVIEW TIP

**Q:** What are SLA Tiers in API Manager?  
**A:** SLA Tiers are predefined sets of limits and policies that define how an API can be used. Applications subscribe to an API and select a tier, and API Manager enforces the limits defined in that tier.

### KEY TAKEAWAY

SLA Tiers help you offer different levels of API access with defined limits and policies. They ensure fairness, protect your systems, and allow you to monetize your APIs effectively.

💡 Use SLA Tiers to deliver better API experiences and ensure fair usage for all your consumers!

## MODULE 8

# 8.10 CORS (CROSS-ORIGIN RESOURCE SHARING)

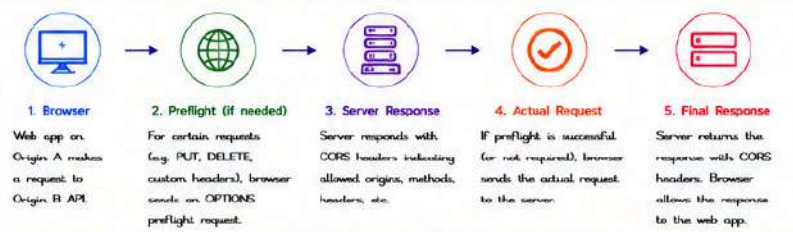


CORS (Cross-Origin Resource Sharing) is a security mechanism that allows a web application running on one origin (domain, protocol, or port) to access resources from another origin. It helps **securely enable** cross-origin requests while **preventing unauthorized access**.

### WHAT IS CORS?

- Browsers restrict cross-origin requests by default using the Same-Origin Policy.
- CORS allows the server to relax this restriction in a controlled way.
- It uses HTTP headers to inform the browser what is allowed.
- It is configured on the server (API) side, not on the client.

### HOW CORS WORKS



### SIMPLE vs NON-SIMPLE REQUESTS

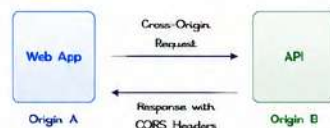
- **Simple Requests (No Preflight)**  
GET, HEAD, POST with:
  - Headers: Accept, Accept-Language, Content-Language, Content-Type (application/x-www-form-urlencoded, multipart/form-data, text/plain)
- **Non-Simple Requests (Preflight Required)**  
Any request that uses:
  - Methods: PUT, DELETE, PATCH, etc.
  - Headers: Custom headers
  - Content-Type: application/json, etc.

### IMPORTANT CORS HEADERS

- **Access-Control-Allow-Origin:** Specifies which origins are allowed.
- **Access-Control-Allow-Methods:** Specifies allowed HTTP methods.
- **Access-Control-Allow-Headers:** Specifies allowed request headers.
- **Access-Control-Allow-Credentials:** Indicates if credentials (cookies, auth headers) are allowed.
- **Access-Control-Max-Age:** Specifies how long the preflight response can be cached.
- **Access-Control-Expose-Headers:** Specifies headers exposed to the browser.

### EXAMPLE SCENARIO

Web app at <https://app.example.com> calling API at <https://api.example.com>.



### CORS REQUEST FLOW (WITH PREFLIGHT)



### CONFIGURING CORS IN MULESOFT

Using `<cors:config>` in Mule configuration:

```
<cors:config name="CORS.Config" doc:name="CORS Config">
  allowedOrigins="https://app.example.com"
  allowedMethods="GET,POST,PUT,DELETE,OPTIONS"
  allowedHeaders="X-Auth-Token,Content-Type"
  exposedHeaders="X-Rate-Limit-Remaining"
  allowCredentials="true"
  maxAge="3600" />
```

Apply to your API or HTTP Listener:

```
<httpListener:config name="HTTP.Listener.config">
  host="0.0.0.0" port="8081">
  <httpListener:connection>
    <cors:config ref="CORS.Config" />
  </httpListener:connection>
</httpListener:config>
```

### RESPONSE CODES (CORS)

- ✓ 200 OK  
CORS request successful.
- ✗ 403 Forbidden  
Origin not allowed.
- ! 403 Preflight Failed  
Preflight request failed (headers/methods not allowed).
- i No CORS Headers  
Browser blocks the response.

### SAMPLE CORS RESPONSE HEADERS

HTTP/1.1 200 OK  
Access-Control-Allow-Origin: <https://app.example.com>  
Vary: Origin  
Access-Control-Allow-Methods: GET,POST,PUT,DELETE,OPTIONS  
Access-Control-Allow-Headers: X-Auth-Token,Content-Type  
Access-Control-Allow-Credentials: true  
Access-Control-Max-Age: 3600  
Content-Type: application/json

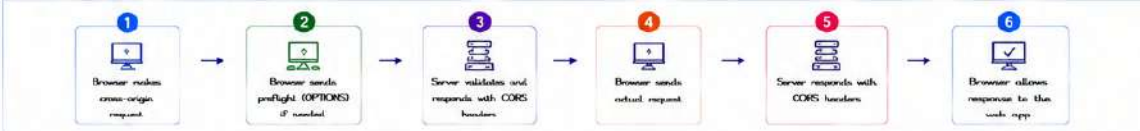
### CORS POLICY OPTIONS

| Option           | Description                               |
|------------------|-------------------------------------------|
| allowedOrigins   | List of origins or * to allow all         |
| allowedMethods   | HTTP methods allowed                      |
| allowedHeaders   | Request headers allowed                   |
| allowCredentials | Allow cookies / auth headers (true/false) |
| maxAge           | How long to cache preflight (in seconds)  |
| exposedHeaders   | Response headers accessible to client     |

### COMMON MISTAKES

- ✗ Using \* with allowCredentials="true".
- ✗ Not allowing required headers or methods.
- ✗ Forgetting to handle preflight OPTIONS.
- ✗ Placing CORS config only on some flows.
- ✗ Relying on client-side (browser) workarounds.

### CORS FLOW SUMMARY (VISUAL)



### REAL-WORLD EXAMPLE

- A SPA hosted on <https://app.example.com> calls an API on <https://api.example.com>.
- The API allows this origin and specific methods and headers.
- Preflight is cached for 1 hour (maxAge=3600).
- The browser allows the response and the app works securely.



### USE CASES

- Single Page Applications (SPA) calling external APIs.
- Mobile apps embedded in webviews.
- APIs consumed by partners from their web apps.
- Micro-frontends architectures.

### BEST PRACTICES

- ✓ Allow only required origins.
- ✓ Restrict methods and headers.
- ✓ Use allowCredentials only when required.
- ✓ Set appropriate maxAge.
- ✓ Keep CORS configuration centralized and consistent.

### INTERVIEW TIP

Q: What is CORS?  
A: CORS (Cross-Origin Resource Sharing) allows a web application from one origin to access resources from another origin securely. It uses HTTP headers to let the browser know what is allowed.

### KEY TAKEAWAY

CORS enables secure cross-origin communication by allowing servers to explicitly specify which origins, methods, headers, and credentials are permitted.



CORS is not a security mechanism by itself; it's a controlled way to relax the Same-Origin Policy. Always configure it carefully to enable access while keeping your APIs secure!



## MODULE 8

# 8.11 POLICY CHAINING

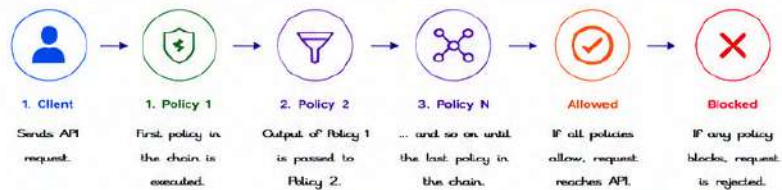


Policy Chaining in MuleSoft API Manager allows you to apply multiple policies in a specific order to a request or response. Policies are executed one after another (in sequence), and the output of one policy becomes the input to the next. It helps enforce **layered security**, **control**, and **governance** consistently.

### WHAT IS POLICY CHAINING?

- Apply multiple policies in a defined sequence to a request or response.
- Each policy is executed in the order you configure.
- If a policy blocks the request, the remaining policies in the chain are not executed.
- It enables a layered approach to enforce security, control, and governance.

### HOW POLICY CHAINING WORKS



### KEY CONCEPTS

- Order Matters:** Policies are executed in the configured order.
- Short-Circuit:** If a policy denies the request, remaining policies are skipped.
- Context Sharing:** Policies share data in the Mule event (attributes, variables).
- Request & Response:** Chains can be applied to both request and response flows.
- Scope:** Applied at API, Application, or Client level.

### COMMON POLICIES IN A CHAIN

- Client ID Enforcement** – Validates client identity.
- JWT Validation** – Validates access tokens.
- IP White Listing** – Allows only trusted IPs.
- Rate Limiting** – Prevents abuse and controls traffic.
- Spike Control** – Protects from sudden bursts.
- SLA Tiers** – Enforces usage limits per tier.
- CORS** – Controls cross-origin requests.

### EXAMPLE POLICY CHAIN (REQUEST FLOW)

| Order | Policy                | Purpose                          |
|-------|-----------------------|----------------------------------|
| 1     | Client ID Enforcement | Validates the client.            |
| 2     | JWT Validation        | Validates the access token.      |
| 3     | IP White Listing      | Allows only trusted IPs.         |
| 4     | Rate Limiting         | Limits requests per time window. |
| 5     | Spike Control         | Protects against sudden spikes.  |
| 6     | SLA Tiers             | Enforces tier-based limits.      |
| 7     | CORS                  | Validates origin of request.     |

If all policies pass → Request is allowed to reach the API.  
If any policy fails → Request is blocked.

### CONFIGURING POLICY CHAIN IN API MANAGER

- Create or Select an API.
- Go to API → Policies → Add Policy.
- Choose "Policy Chain".
- Add policies to the chain in the desired order.
- Configure each policy.
- Save and deploy the chain.
- Apply the policy chain to the required flow (Request and/or Response).

*i* You can reuse policy chains across multiple APIs.

### SAMPLE POLICY CHAIN CONFIGURATION

Policy Chain Name: Standard\_Security\_Chain

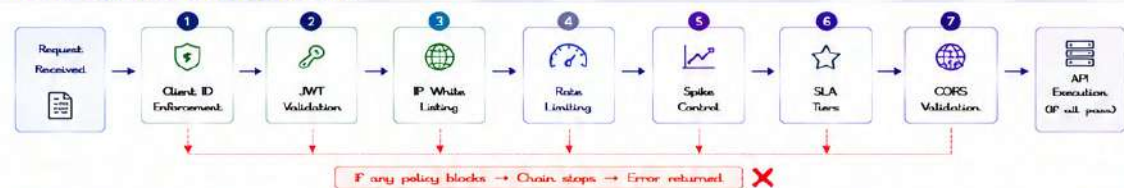
| Order | Policy                | Configuration (Example)    |
|-------|-----------------------|----------------------------|
| 1     | Client ID Enforcement | Enforce registered clients |
| 2     | JWT Validation        | Validate token & expiry    |
| 3     | IP White Listing      | 203.0.113.0/24 allowed     |
| 4     | Rate Limiting         | 1000 req / 1 min           |
| 5     | Spike Control         | 50 req / 2 sec             |
| 6     | SLA Tiers             | Gold / Silver / Bronze     |
| 7     | CORS                  | https://appsample.com      |

*i* Policies are executed from top to bottom.

### RESPONSE CODES

- 200 OK**  
Request passed through all policies.
- 401 Unauthorized**  
Client identity or token validation failed.
- 403 Forbidden**  
IP not allowed / policy violation.
- 429 Too Many Requests**  
Rate limit, spike limit or SLA limit exceeded.
- CORS Error**  
Origin not allowed or CORS policy failed.

### POLICY CHAIN EXECUTION FLOW (VISUAL)



### REAL-WORLD EXAMPLE

An e-commerce API applies the following chain to protect its services:

- Validate client.
- Validate token.
- Allow only office IPs.
- Limit usage per minute.
- Protect from spikes.
- Enforce SLA tier limits.
- Allow only web app domain via CORS.

Result: Only valid, trusted and within limits requests reach the backend services.



### BEST PRACTICES

- Design chains based on security first approach.
- Place cheaper (fast) validations first (e.g., Client ID, IP White Listing).
- Place heavier validations later (e.g., JWT, SLA).
- Use reusable policy chains for consistency.
- Monitor and analyze policy failures.
- Document your chain and review periodically.

### COMMON MISTAKES

- Incorrect order of policies.
- Placing heavy policies first (performance impact).
- Not handling CORS policy correctly.
- Using too many policies unnecessarily.
- Ignoring monitoring of policy failures.

### USE CASES

- Secure enterprise APIs with multiple controls.
- Enforce compliance and governance.
- Protect against abuse and overuse.
- Support multi-tenant and partner APIs.

### INTERVIEW TIP

**Q: What is Policy Chaining in API Manager?**  
**A:** Policy Chaining allows applying multiple policies in a specific order to a request or response. Each policy is executed in sequence, and if any policy blocks, the chain stops and the request is rejected.

### KEY TAKEAWAY

Policy Chaining helps you apply layered security and control by executing multiple policies in sequence. It ensures only valid, trusted, and within-limits requests reach your APIs, improving security, reliability, and performance.



Use Policy Chaining to enforce multiple controls in the right order and deliver **secure**, **reliable** and **governed** APIs!

## MODULE 9

# 9.1 CLOUDHUB 1.0

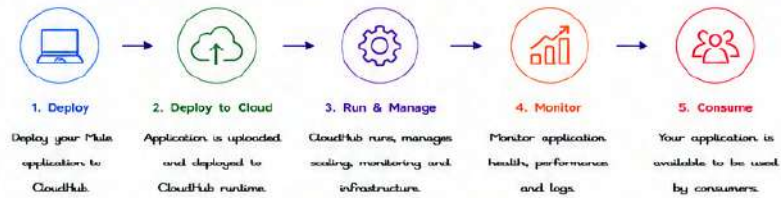


CloudHub 1.0 is MuleSoft's fully managed integration platform (PaaS) for deploying, managing, and scaling Mule applications in the cloud. It abstracts infrastructure management so you can focus on building and running integrations.

### WHAT IS CLOUDHUB 1.0?

- A fully managed runtime for Mule applications.
- Hosted on a multi-tenant cloud infrastructure managed by MuleSoft.
- Provides elastic scaling, high availability, and automatic failover.
- You deploy applications; MuleSoft manages the infrastructure.

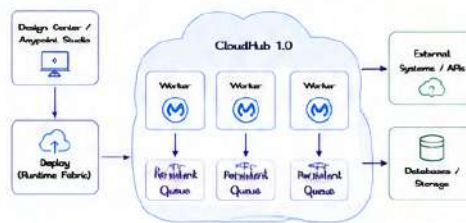
### HOW CLOUDHUB 1.0 WORKS



### KEY FEATURES

- Fully Managed: MuleSoft manages the runtime, OS, patches, scaling.
- Elastic Scaling: Scale up/down workers based on load.
- High Availability: Built-in redundancy and automatic failover.
- Zero Infrastructure: No servers to provision or manage.
- Monitoring & Logging: Out-of-the-box metrics, logs, and alerts.
- Secure by Default: Runs in a secure, isolated cloud environment.
- Global Cloud: Deploy across multiple regions.

### CLOUDHUB 1.0 ARCHITECTURE



### CLOUDHUB 1.0 REGIONS

Examples:

- US East (N. Virginia)
- US West (Oregon)
- Europe (Ireland)
- Asia Pacific (Sydney)
- South America (São Paulo)

Deploy in the region closest to your users for better performance and compliance.

### APPLICATION LIFECYCLE IN CLOUDHUB 1.0

- 1 Build your Mule application.
- 2 Package it as a deployable JAR.
- 3 Deploy to CloudHub via Anypoint Platform.
- 4 CloudHub provisions workers (instances).
- 5 Application starts and is registered.
- 6 Scale automatically or manually as needed.
- 7 Monitor, manage, and update continuously.

### WORKERS AND SCALING

- Worker: A runtime container that runs your application.
- Each worker has allocated vCores and memory.
- Scale Types:

| Type                | Description                          |
|---------------------|--------------------------------------|
| Fixed               | Fixed number of workers.             |
| Dynamic             | Auto-scale based on CPU utilization. |
| Static (deprecated) | Legacy static scaling.               |

Typical sizes: 0.1 vCore - 4 vCores, 1 GB - 16 GB RAM.

### EXAMPLE

You deploy an order-processing API to CloudHub 1.0 with the following configuration:

- Region: US East (N. Virginia)
- Workers: 2 (Dynamic)
- Worker Size: 1 vCore, 2 GB RAM
- Ingress: Enabled
- Auto-scaling: 1 - 5 workers based on CPU (up to 70%)

CloudHub runs and scales your application automatically based on traffic.

### CONFIGURING CLOUDHUB 1.0 - KEY STEPS

- 1 Go to Anypoint Platform → Runtime Manager.
- 2 Click Deploy application.
- 3 Upload your application (.JAR file).
- 4 Select Runtime: CloudHub.
- 5 Choose Region and worker configuration.
- 6 Configure properties (env variables, queues, etc.).
- 7 Review and deploy.

### APPLICATION PROPERTIES (EXAMPLES)

| Property        | Description                        |
|-----------------|------------------------------------|
| workerType      | Worker size (e.g., 1 vCore, 2 GB)  |
| workerCount     | Initial number of workers          |
| maxWorkerCount  | Max workers (for dynamic scaling)  |
| autoScaling     | Enable/disable auto-scaling        |
| persistentQueue | Enable persistent queues           |
| logLevel        | Logging level (ERROR, INFO, DEBUG) |

### CLOUDHUB 1.0 LIMITS (EXAMPLES)

- Max vCores per worker: 4
- Max RAM per worker: 16 GB
- Max workers per application: 100 (region dependent)
- Max applications per account: Varies by subscription
- Persistent queues: Supported (with storage limit)
- For latest limits, refer to MuleSoft documentation.

### MONITORING & MANAGEMENT

- Monitor applications in Runtime Manager.
- View metrics: CPU, Memory, Throughput, Errors.
- Access logs and set log levels.
- Integrate with Anypoint Monitoring / External APM.
- Set alerts and notifications.

### SECURITY IN CLOUDHUB 1.0

- Applications run in a secure, isolated environment.
- TLS enabled by default.
- Use Client ID Enforcement / OAuth 2.0 for API security.
- Private spaces and VPN for network isolation.
- Secure properties (encrypted at rest).

### CLOUDHUB 1.0 vs CLOUDHUB 2.0 (AT A GLANCE)

| Feature           | CloudHub 1.0          | CloudHub 2.0                  |
|-------------------|-----------------------|-------------------------------|
| Runtime Isolation | Shared (multi-tenant) | Customer-dedicated            |
| Networking        | At runtime level      | Full infrastructure isolation |
| Deployment        | Public endpoints      | VPC, Private Networking       |
| Use Case          | WAA/JAR               | Container-based               |
|                   | Standard integrations | Advanced security & control   |

### CLOUDHUB 1.0 DEPLOYMENT FLOW (VISUAL)



### REAL-WORLD EXAMPLE

A retail company deploys an inventory API to CloudHub 1.0 to ensure product availability to mobile and web apps.

- Handles variable traffic during promotions.
- Auto-scales workers to meet demand.
- Guarantees high availability and reliability.

### USE CASES

- API-led connectivity
- System integration
- Data transformation
- Event-driven architectures
- Real-time process orchestration

### BEST PRACTICES

- ✓ Choose the right region for performance.
- ✓ Use dynamic scaling for variable traffic.
- ✓ Enable persistent queues for reliability.
- ✓ Monitor regularly and set alerts.
- ✓ Secure APIs with proper policies.

### INTERVIEW TIP

**Q: What is CloudHub 1.0?**  
**A:** CloudHub 1.0 is MuleSoft's fully managed cloud platform that allows you to deploy and run Mule applications without managing infrastructure.

### KEY TAKEAWAY

CloudHub 1.0 provides a simple, scalable, and reliable platform to deploy and manage Mule applications in the cloud.



CloudHub 1.0 lets you focus on your integrations while MuleSoft takes care of the infrastructure!



## MODULE 9

# 9.2 CLOUDHUB 2.0

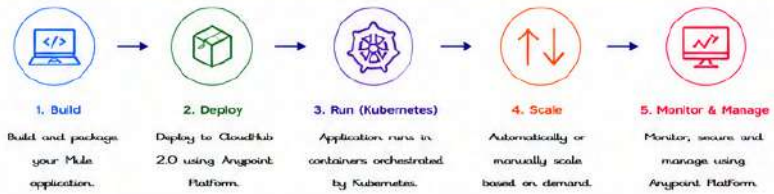


CloudHub 2.0 is MuleSoft's next-generation runtime for deploying and running Mule applications. It is built on a container-based architecture with Kubernetes, providing improved **performance**, **flexibility**, **observability**, and **cost-efficiency**.

### WHAT IS CLOUDHUB 2.0?

- Next-generation, container-based runtime built on Kubernetes.
- Better performance and resource efficiency.
- Fine-grained scaling, zero-downtime deployments, and high availability.
- Supports both stateless and stateful workloads.
- Enhanced security, networking, and observability.

### HOW CLOUDHUB 2.0 WORKS



### CLOUDHUB 2.0 ARCHITECTURE



### KEY FEATURES

- **Kubernetes-based:** Built on EKS (AWS) and AKS (Azure).
- **Improved Performance:** Better throughput, lower latency, efficient resource utilization.
- **Granular Scaling:** Scale workers with fine control over CPU and memory.
- **Zero-downtime Deployments:** Rolling updates with no service interruption.
- **High Availability:** Multi-zone deployment and automatic failover.
- **Observability:** Built-in metrics, logs, and distributed tracing with deeper insights.
- **Enhanced Security:** Network isolation, encryption, secrets management, and more.
- **Stateful Support:** Persistent volumes for stateful applications.

### CLOUDHUB 1.0 VS CLOUDHUB 2.0

| Feature      | CloudHub 1.0              | CloudHub 2.0                                |
|--------------|---------------------------|---------------------------------------------|
| Architecture | Shared (Multi-tenant)     | Dedicated (Container on K8s)                |
| Runtime      | Mule Runtime (Shared VM)  | Mule Runtime in Containers                  |
| Isolation    | Limited                   | Strong (Management + K8s)                   |
| Scaling      | Limited, coarse-grained   | Fine-grained, per worker                    |
| Deployment   | Rolling (limited control) | Zero-downtime, rolling, blue/green          |
| Networking   | Public endpoints          | VPC/VNet, PrivateLink, Ingress              |
| Monitoring   | Basic metrics             | Enhanced metrics, logs, distributed tracing |
| Use Cases    | Standard integrations     | Advanced, mission-critical workloads        |

### WORKERS AND SCALING

- A worker is a container that runs your Mule application.
- Each worker has allocated vCores and memory.
- Scaling can be done manually or automatically.

#### Types

| Type                | Description                          |
|---------------------|--------------------------------------|
| Fixed               | Fixed number of workers.             |
| Dynamic             | Auto-scale based on CPU utilization. |
| Static (Deprecated) | Legacy static scaling.               |

Supported worker sizes (varies by region):  
0.1 vCore - 16 vCores    1 GB - 64 GB RAM

### APPLICATION LIFECYCLE

1. Build your application and package as JAR.
2. Deploy to CloudHub 2.0 using Runtime Manager or CLI.
3. Application is uploaded and container image is created.
4. Kubernetes schedules pods (workers) in your namespace.
5. Application starts and is registered.
6. Monitor, manage, and scale as needed.

### NETWORKING OPTIONS

- Public Endpoint (default).
- PrivateLink (AWS) / Private Endpoint (Azure).
- VPC/VNet peering.
- Ingress for inbound traffic.
- Egress control for outbound traffic.



### APPLICATION PROPERTIES (EXAMPLES)

| Property         | Description                  | Example        |
|------------------|------------------------------|----------------|
| workers          | Initial number of workers.   | 2              |
| workerType       | Worker type (fixed/dynamic). | Dynamic        |
| maxWorkers       | Max workers (for dynamic).   | 10             |
| workerSize       | vCores and memory.           | 2 vCores, 4 GB |
| persistentVolume | Enable persistent volume.    | true           |
| logLevel         | Logging level.               | INFO           |

### DEPLOYMENT STRATEGIES

- **Rolling Update:** Gradually replaces old pods with new ones.
- **Blue/Green Deployment:** Deploy to a new environment and switch traffic.
- **Canary Deployment (via API Manager):** Route a small % of traffic to new version.

### OBSERVABILITY & MANAGEMENT

- Metrics: CPU, memory, request rate, etc.
- Logs: Centralized log management.
- Distributed Tracing: End-to-end request tracing.
- Alerts: Threshold-based notifications.
- Dashboards: Pre-built and custom dashboards.

### SECURITY

- TLS 1.2+ enabled by default.
- Secrets Manager for secure properties.
- IAM roles / Workload Identity for cloud access.
- Network isolation using VPC/VNet.
- Regular patching and hardened containers.

### CLOUDHUB 2.0 LIMITS (EXAMPLES)

- Max vCores per worker: 16.
- Max RAM per worker: 64 GB.
- Max workers per application: 200.
- Max applications per environment: Varies by subscription.
- For latest limits, refer to MuleSoft documentation.

### DEPLOYMENT FLOW (VISUAL)



### REAL-WORLD USE CASES

- High-throughput APIs and integrations.
- Event-driven architectures.
- Mission-critical enterprise applications.
- Stateful applications with persistent data.
- Microservices and B2B integrations.

### BEST PRACTICES

- ✓ Right-size your workers and scaling policies.
- ✓ Use dynamic scaling for variable traffic.
- ✓ Enable alerts and dashboards.
- ✓ Use PrivateLink / Private Endpoint for security.
- ✓ Keep dependencies and patches up to date.

### INTERVIEW TIP

#### Q: What is CloudHub 2.0?

A: CloudHub 2.0 is MuleSoft's next-generation runtime built on Kubernetes, offering better performance, granular scaling, high availability, and advanced networking and security options.

### KEY TAKEAWAY

CloudHub 2.0 leverages Kubernetes to deliver a modern, secure, scalable, and observable runtime for your Mule applications in the cloud.



CloudHub 2.0 helps you build, run, and scale enterprise integrations with confidence in a **cloud-native world!**

## MODULE 9

# 9.3 RUNTIME FABRIC



Runtime Fabric is the underlying infrastructure layer of CloudHub that provides, manages, and secures the runtime environments where your Mule applications run. It **abstracts** servers and infrastructure complexities so you can **focus** on your applications.

### WHAT IS RUNTIME FABRIC?

- The Foundation of CloudHub that runs your Mule applications.
- Manages infrastructure, availability, scalability, and security.
- Built on container orchestration technology (Kubernetes).
- Provides a consistent, secure, and highly available runtime.

### HOW RUNTIME FABRIC WORKS



### RUNTIME FABRIC ARCHITECTURE



### KEY CAPABILITIES

- **Isolation:** Each customer application runs in isolated environments.
- **Auto-Scaling:** Scales applications up or down based on demand.
- **High Availability:** Multi-zone deployment with automatic failover.
- **Self-Healing:** Detects and recovers from failures automatically.
- **Zero Infrastructure Management:** MuleSoft manages servers, OS, patching, and upgrades.
- **Security by Default:** Built-in network isolation, encryption, and secure access.
- **Observability:** Built-in metrics, logs, and distributed tracing.
- **Consistency:** Same runtime experience across all regions.

### RUNTIME FABRIC COMPONENTS

- **Kubernetes Cluster:** Orchestrates containers (pods) that run Mule applications.
- **Worker Nodes:** Provide compute, memory, and storage for pods.
- **Service Mesh:** Handles secure service-to-service communication, traffic management, and resilience.
- **Networking Layer:** Provides routing, load balancing, and network isolation.
- **Storage Layer:** Persistent volumes for stateful workloads (if required).
- **Observability Stack:** Collects and aggregates metrics, logs, and traces.

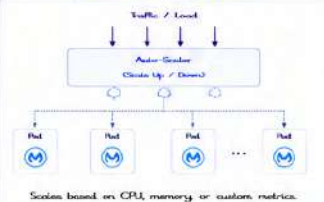
### APPLICATION DEPLOYMENT VIEW



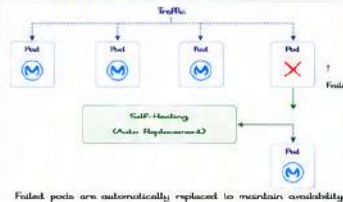
### BENEFITS

- ✓ Focus on building integrations, not infrastructure.
- ✓ Elastic scalability and performance.
- ✓ High availability and reliability.
- ✓ Faster time to market.
- ✓ Consistent runtime across regions.
- ✓ Built-in monitoring and management.
- ✓ Enterprise-grade security.

### SCALING IN RUNTIME FABRIC



### FAULT TOLERANCE & SELF-HEALING



### OBSERVABILITY



### NETWORKING & SECURITY

- Private, isolated networks for each environment.
- Ingress and egress control.
- TLS encryption in transit and at rest.
- Secrets management and secure properties.
- Identity & Access Management (IAM) integration.

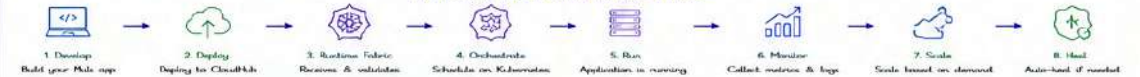
### RUNTIME FABRIC REGIONS

- Available in multiple global regions to ensure low latency and compliance.
- US East (N. Virginia)
  - US West (Oregon)
  - Europe (Ireland)
  - Asia Pacific (Sydney)
  - ...and more.

### RUNTIME FABRIC VS SELF-MANAGED

| Aspect               | Runtime Fabric (CloudHub) | Self-Managed     |
|----------------------|---------------------------|------------------|
| Infrastructure       | Managed by MuleSoft       | Managed by You   |
| Scaling              | Auto (on demand)          | Manual           |
| High Availability    | Built-in (multi-zone)     | Must be designed |
| Upgrades & Patching  | Automatic                 | Manual           |
| Monitoring           | Built-in                  | You set up       |
| Time to Market       | Faster                    | Slower           |
| Operational Overhead | Low                       | High             |

### END-TO-END FLOW WITH RUNTIME FABRIC



### REAL-WORLD USE CASES

- APIs with variable traffic patterns.
- Mission-critical integrations.
- Global applications requiring low latency.
- Event-driven and real-time processing.
- Enterprises with strict compliance needs.

### BEST PRACTICES

- ✓ Design stateless applications.
- ✓ Use external systems for state management.
- ✓ Set appropriate autoscaling policies.
- ✓ Monitor regularly and set alerts.
- ✓ Use secure properties and secrets.

### INTERVIEW TIP

- Q: What is Runtime Fabric?
- A: Runtime Fabric is the container-based infrastruct. layer in CloudHub that runs and manages Mule applications. It uses Kubernetes for orchestration, providing scalability, high availability, security, and observability.

### KEY TAKEAWAY

Runtime Fabric handles the heavy lifting of infrastructure so you can deliver integrations faster with confidence and focus on business outcomes.



Runtime Fabric powers CloudHub, giving you a secure, scalable, and reliable foundation for your integrations in the cloud.



MODULE  
9

## 9.4 ON-PREM DEPLOYMENT

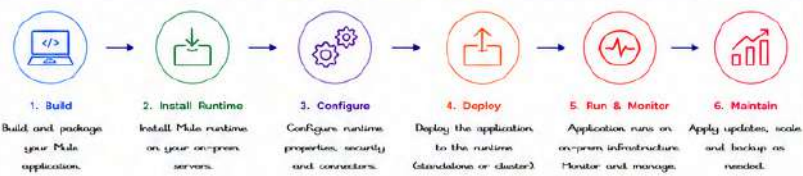


On-prem deployment allows you to run Mule applications and Anypoint Platform components in your own data center or private cloud. It provides **maximum control, data residency, and integration with on-prem systems** while maintaining enterprise-grade security and manageability.

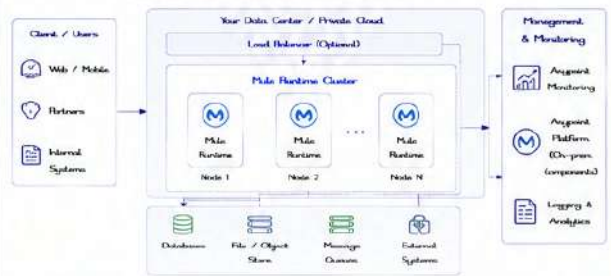
## WHAT IS ON-PREM DEPLOYMENT?

- Deploy Mule runtime and Anypoint Platform components in your own infrastructure.
- Ideal for data residency, regulatory compliance and network isolation.
- Full control over infrastructure, security, patching and access.
- Supports single-server, cluster, or highly available (HA) deployments.

## ON-PREM DEPLOYMENT FLOW



## ON-PREM DEPLOYMENT ARCHITECTURE



## KEY CAPABILITIES

- **Data Residency:** Keep data within your organization or region.
- **Network Isolation:** Operate in private networks with no public exposure.
- **High Availability:** Clustering and Failover for zero-/low-downtime.
- **Scalability:** Scale up (bigger servers) or scale out (more nodes).
- **Security:** TLS, keystore management, IP allow-listing, OS hardening.
- **Connectors:** Access to on-prem systems, databases and legacy apps.
- **Manageability:** Monitor log and manage with on-prem Anypoint Platform components.

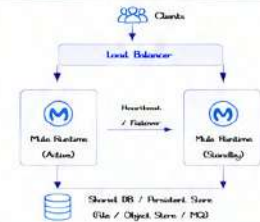
## PREREQUISITES

- Supported OS: RHEL, CentOS, Windows, AIX, etc.
  - Java: JDK 8 or 11 (as per runtime version).
  - Hardware: CPU, Memory, Disk as per using guide.
  - Network: Required ports open between components.
  - Access: OS user with proper permissions.
  - Licensing: Mule runtime license (self-managed).
- Refer to MuleSoft documentation for the latest system requirements.

## DEPLOYMENT OPTIONS

| Option                     | Description                                      | Use Case                                       |
|----------------------------|--------------------------------------------------|------------------------------------------------|
| Single Server (Standalone) | One Mule runtime instance on a single server.    | Dev, Test, or low-volume non-critical apps.    |
| HA (Active / Passive)      | Two servers; one active, one standby (failover). | High availability with minimal infrastructure. |
| Cluster (Active / Active)  | Multiple runtimes behind load balance.           | High throughput and scalability.               |
| Hybrid                     | Combine on-prem with cloud resources.            | Building DR, or cloud integration.             |

## EXAMPLE: HA DEPLOYMENT



## IMPORTANT CONFIGURATIONS

- **unzip/mkdir:** JVM, memory, logging, and system properties.
  - **mule-daemon.properties:** Environment-specific properties.
  - **log4j2.xml:** Logging configuration.
  - **keytoolgen.sh:** TLS / SSL configuration.
  - **mule-cluster.yaml:** Cluster configuration.
  - **application-deployment.properties:** Deployment settings.
- External sensitive information using encrypted properties or a secrets manager.

## RUNTIME SIZING GUIDE (EXAMPLE)

| Workload               | vCPU | RAM    | Disk    | Notes               |
|------------------------|------|--------|---------|---------------------|
| Small (Up to 50 TPS)   | 2    | 4 GB   | 40 GB   | Standalone          |
| Medium (50 - 500 TPS)  | 4    | 8 GB   | 80 GB   | HA or Small Cluster |
| Large (500 - 2000 TPS) | 8    | 16 GB  | 160 GB  | Cluster             |
| X-Large (> 2000 TPS)   | 16+  | 32 GB+ | 300 GB+ | Cluster + Tuning    |

ⓘ Sizing depends on application, payload size, connectors and SLAs.

## COMMON PORTS

| Port        | Protocol / Use        |
|-------------|-----------------------|
| 8081 / 8082 | HTTP / HTTPS (app)    |
| 8091        | JMX (management)      |
| 50081       | Mule clustering (TCP) |
| 5712        | P2P / Discovery       |
| 61616       | ActiveMQ (if used)    |
| 443         | HTTPS (management)    |

ⓘ Ensure these ports are allowed in network/firewall rules.

## SECURITY BEST PRACTICES

- ✓ Enable TLS for all inbound and outbound traffic.
- ✓ Use strong ciphers and secure protocols.
- ✓ Restrict access using firewalls and IP allow-listing.
- ✓ Apply OS hardening and least-privilege access.
- ✓ Encrypt sensitive properties and connectors.
- ✓ Regularly patch OS runtime and dependencies.

## MONITORING &amp; LOGGING



## BACKUP &amp; DISASTER RECOVERY

- Backup databases, persistent stores and config.
- Store backups in a secure, off-site location.
- Test recovery procedures regularly.
- Use HA or clustering for high availability.
- Plan for region-level DR if required.

## ON-PREM DEPLOYMENT LIFECYCLE



## REAL-WORLD USE CASES

- Integrate with legacy systems in data center.
- Financial services with data sovereignty needs.
- Manufacturing with OT / IT integration.
- Government and regulated industry workloads.
- Low-latency integrations within internal network.

## BEST PRACTICES

- ✓ Design for high availability and resilience.
- ✓ Automate deployments and configuration.
- ✓ Use internalized configuration.
- ✓ Monitor proactively and act on alerts.
- ✓ Document and test DR procedures.

## INTERVIEW TIP

Q: When would you choose on-prem deployment?  
A: Choose on-prem when you need data residency, network isolation, strict compliance, low latency to internal systems, or have policies that restrict using public cloud.

## KEY TAKEAWAY

On-prem deployment gives you full control and security over your Mule applications and data while still leveraging the power and flexibility of the Anypoint Platform.



On-prem deployment empowers you to run **secure, reliable and high-performance integrations** on your own terms!

## MODULE 9

# 9.5 HYBRID DEPLOYMENT

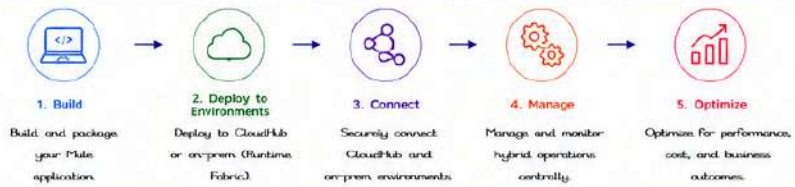


Hybrid deployment lets you run your Mule applications across MuleSoft-managed cloud (CloudHub) and your own on-premises environments using Runtime Fabric. It gives you the **flexibility** to meet data residency, latency, and compliance needs while optimizing for **performance** and **cost**.

### WHAT IS HYBRID DEPLOYMENT?

- Run applications in both CloudHub and on-premises environments.
- Securely connect and manage both environments as one.
- Choose the right environment for each application or workload.
- Balance data residency, performance, compliance, and cost.

### HYBRID DEPLOYMENT FLOW



### HYBRID ARCHITECTURE OVERVIEW



### KEY CAPABILITIES

- **Flexible Placement:** Deploy where it makes the most sense.
- **Secure Connectivity:** Encrypted communication between environments.
- **Centralized Management:** Manage all applications from a single Anypoint Platform.
- **High Availability:** Use HA and load balancing on both sides.
- **Data Residency:** Keep sensitive data on-prem as required.
- **Performance Optimization:** Reduce latency by running closer to data or systems.
- **Consistent Operations:** Unified monitoring, logging, and alerting across environments.

### DEPLOYMENT OPTIONS

| Option               | Description                                                  |
|----------------------|--------------------------------------------------------------|
| All Cloud            | All applications deployed to CloudHub.                       |
| All On-Prem          | All applications deployed on-prem using Runtime Fabric.      |
| Hybrid (by App)      | Some applications in CloudHub, others on-prem.               |
| Hybrid (by Use Case) | Place workloads based on data, latency, compliance, or cost. |

### USE CASES

- Keep customer data on-prem for compliance while exposing APIs in the cloud.
- Integrate with on-prem legacy systems while leveraging cloud services.
- Reduce latency by processing closer to on-prem systems.
- Handle traffic spikes in CloudHub while keeping core systems on-prem.

### CONNECTIVITY OPTIONS

| Option                        | Description                                                     |
|-------------------------------|-----------------------------------------------------------------|
| VPN / IPsec                   | Site-to-site VPN between cloud VPC and on-prem network.         |
| MuleSoft Private Link         | Private connectivity to Anypoint Platform over private network. |
| Direct Connect / ExpressRoute | Dedicated private connection to cloud provider.                 |
| Firewall Rules                | Allow required ports and IPs between environments.              |

### NETWORK & SECURITY CONSIDERATIONS

- Use private networks and IP allow-listing.
- Encrypt traffic (TLS 1.2+) everywhere.
- Use VPN, Private Link, or Direct Connect for secure connectivity.
- Apply least privilege access with Anypoint Platform roles.
- Monitor for unusual traffic and access patterns.
- Keep OS, runtime, and connectors patched.

### APPLICATION PLACEMENT GUIDELINES

| Criteria       | Place in CloudHub When...        | Place On-Prem When...              |
|----------------|----------------------------------|------------------------------------|
| Data Residency | Data can be in public cloud.     | Data must stay on-prem.            |
| Latency        | Cloud access is acceptable.      | Need low latency to local systems. |
| Compliance     | No specific on-prem requirement. | Regulations require on-prem.       |
| Scalability    | Need elastic scaling.            | Prefer predictable capacity.       |
| Cost           | Variable / elastic workloads.    | Stable long-running workloads.     |

### WORKLOAD DISTRIBUTION EXAMPLES



### MONITORING & OBSERVABILITY

- Unified dashboards for metrics, logs, and alerts.
- Distributed tracing across cloud and on-prem.
- End-to-end visibility of hybrid flows.
- Custom alerts and notifications.

### MANAGEMENT & OPERATIONS

- ✓ Deploy, manage, and update from Runtime Manager.
- ✓ Use environments and business groups to organize assets.
- ✓ Consistent SLAs and policies across environments.
- ✓ Manage connectors and shared assets centrally.
- ✓ Automate deployments with CI/CD.

### DISASTER RECOVERY STRATEGY

- Deploy critical apps in multiple environments.
- Use persistent queues and database replication.
- Regularly back up configurations and data.
- Test DR plans for both cloud and on-prem.
- Define RPO and RTO for each workload.

### HYBRID DEPLOYMENT BENEFITS

- ✓ Flexibility to choose the right environment.
- ✓ Meet compliance and data residency needs.
- ✓ Optimize performance and reduce latency.
- ✓ Better cost management.
- ✓ Business continuity and resilience.
- ✓ Centralized management and governance.



### REAL-WORLD EXAMPLE

- A global retail company:
- Deploys customer-facing APIs in CloudHub for global reach and elasticity.
  - Keeps order processing and inventory systems on-prem for data residency and low latency.
  - Connects both securely using VPN.
  - Manages everything from a single Anypoint Platform.



### BEST PRACTICES

- ✓ Design with modular, loosely-coupled APIs.
- ✓ Use asynchronous patterns where possible.
- ✓ Place data-sensitive workloads on-prem.
- ✓ Monitor and test end-to-end flows.
- ✓ Document network, ports, and SLAs.
- ✓ Review and optimize placement regularly.

### INTERVIEW TIP

- Q: What is hybrid deployment in MuleSoft?
- A: Hybrid deployment allows you to run Mule applications both on CloudHub and on-premises using Runtime Fabric, securely connected and managed from Anypoint Platform for flexibility, compliance, and performance optimization.

### KEY TAKEAWAY

Hybrid deployment gives you the best of both worlds—cloud agility and on-prem control—so you can deliver secure, reliable, and high-performance integrations that meet your business and regulatory needs.

Hybrid deployment with MuleSoft gives you the **flexibility** to run, connect, and manage applications across cloud and on-prem—securely and efficiently.



## MODULE 9

# 9.6 VPC (VIRTUAL PRIVATE CLOUD)

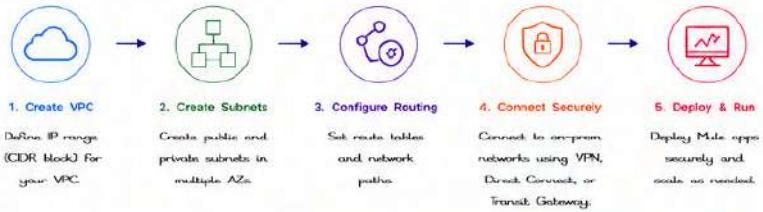


A Virtual Private Cloud (VPC) is an isolated virtual network in the cloud where you deploy your Mule applications **securely**. It gives you **control** over IP ranges, subnets, routing, **connectivity**, and **access** to on-prem systems while protecting your workloads.

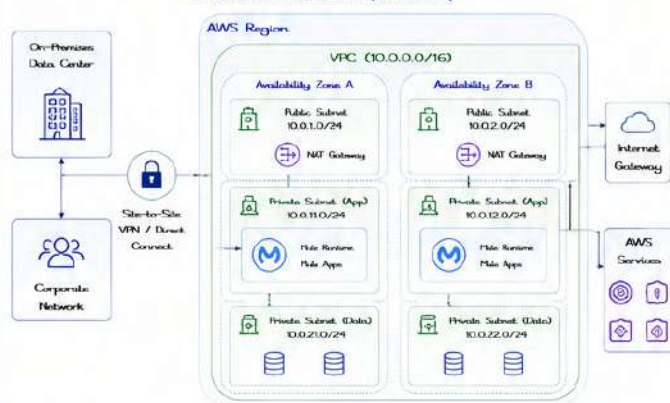
### WHAT IS VPC?

- A logically isolated network in the cloud.
- You define IP address ranges, subnets, routing tables, and gateways.
- Provides secure connectivity between Mule applications, cloud services, and on-prem systems.
- Enables fine-grained control over network traffic and access.

### HOW VPC WORKS



### VPC ARCHITECTURE (EXAMPLE)



### KEY CAPABILITIES

- **Isolated network:** Your own logical network in the cloud.
- **Custom IP ranges:** Choose non-overlapping CIDR blocks.
- **Subnets:** Segment your network for security and organization.
- **Routing control:** Route traffic using route tables.
- **Internet access:** Use Internet Gateway for public access.
- **Private access:** Use NAT Gateway for outbound internet from private subnets.
- **Secure connectivity:** Connect to on-prem via VPN or Direct Connect.
- **High availability:** Deploy across multiple Availability Zones.
- **Security:** Use Security Groups and NACLs to control traffic.

### VPC COMPONENTS

- VPC:** The virtual network container.
- Subnet:** A range of IP addresses within a VPC.
- Route Table:** Contains rules (routes) to direct traffic.
- Internet Gateway:** Enables communication between VPC and the internet.
- NAT Gateway:** Allows instances in private subnets to access the internet.
- Security Group:** Stateless firewall to control inbound/outbound traffic.
- Network ACL:** Stateless firewall to control traffic at subnet level.

### ROUTING IN VPC

| Destination    | Target                        |
|----------------|-------------------------------|
| 10.0.0.0/16    | local                         |
| 0.0.0.0/0      | igw-123456 (Internet Gateway) |
| 192.168.0.0/16 | vgnw-abcdef (VPN Gateway)     |

- Every subnet is associated with a route table.
- Routes determine where traffic is sent.
- The "local" route enables communication within the VPC.

### CONNECTIVITY OPTIONS

- VPN (Site-to-Site):** Encrypted IPsec connection over the internet.
- AWS Direct Connect:** Dedicated private connection to AWS.
- Transit Gateway:** Connect multiple VPCs and on-prem networks.
- VPC Peering:** Connect two VPCs privately.

### SECURITY IN VPC

- Security Groups (Stateful):**
  - Allow traffic based on rules.
  - Applied to instances (e.g., Mule Runtime).
- Network ACLs (Stateless):**
  - Allow or deny traffic at subnet level.
  - Applied to subnets (inbound & outbound).

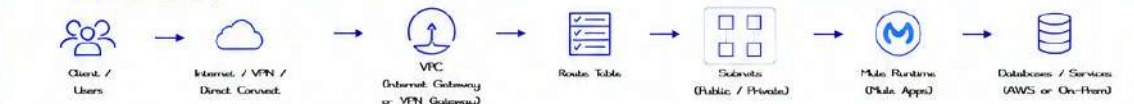
### USE CASES

- ✓ Host Mule applications in a private, secure network.
- ✓ Securely connect to on-prem systems and databases.
- ✓ Isolate environments (dev, test, prod) in separate VPCs.
- ✓ Control outbound internet access for compliance.
- ✓ Enable hybrid cloud integrations with low latency.

### BEST PRACTICES

- ✓ Use private subnets for Mule Runtime applications.
- ✓ Use NAT Gateway for outbound internet from private subnets.
- ✓ Deploy across multiple AZs for high availability.
- ✓ Use the least-privilege rules in Security Groups and NACLs.
- ✓ Monitor VPC Flow Logs for traffic visibility and troubleshooting.

### VPC FLOW (VISUAL)



### VPC CIDR EXAMPLE

VPC CIDR: 10.0.0.0/16

- Subnets:**
- Public Subnet A: 10.0.1.0/24
  - Public Subnet B: 10.0.2.0/24
  - Private Subnet A: 10.0.11.0/24
  - Private Subnet B: 10.0.12.0/24
  - Data Subnet A: 10.0.21.0/24
  - Data Subnet B: 10.0.22.0/24

### MONITORING & OBSERVABILITY

- VPC Flow Logs:** Monitor network traffic.
- CloudWatch Metrics:** Monitor network performance and health.
- CloudWatch Alarms:** Set alerts on anomalies.
- X-Ray:** Trace requests across networks.

### TROUBLESHOOTING TIPS

- Check route table for correct routes.
- Verify Security Group and NACL rules.
- Ensure internet access via NAT or IGW.
- Check VPN/Direct Connect status.
- Use VPC Flow Logs to analyze traffic.

### KEY TAKEAWAY

VPC provides a secure and isolated network foundation for your Mule applications. With proper design, routing, and connectivity, you get performance, security, and control for both cloud and hybrid use cases.

A well-designed VPC is the foundation for secure, reliable, and high-performance Mule integrations in the cloud and hybrid environments.

MODULE  
9

## 9.7 DEDICATED LOAD BALANCER

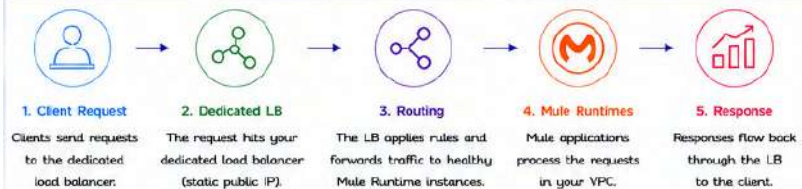


A Dedicated Load Balancer gives you a dedicated, isolated load balancer for your Mule applications. It provides enhanced **performance**, **security**, **isolation**, **predictable capacity**, and **greater control** compared to the shared load balancer.

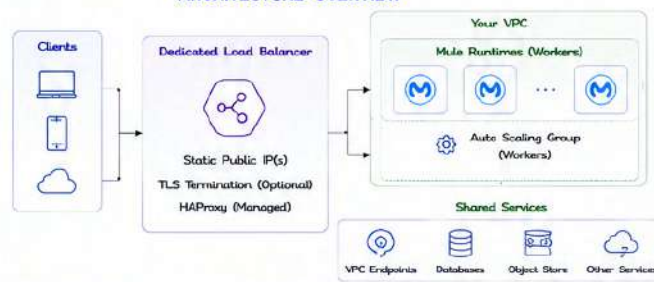
## WHAT IS DEDICATED LOAD BALANCER?

- A dedicated, managed load balancer exclusively for your Anypoint VPC.
- Isolated from other customers for performance and security.
- Static, public IP addresses.
- Full control over listeners, TLS certificates, ciphers, and timeouts.
- Built on industry-standard technology (HAProxy) and fully managed by MuleSoft.

## HOW DEDICATED LOAD BALANCER WORKS



## ARCHITECTURE OVERVIEW



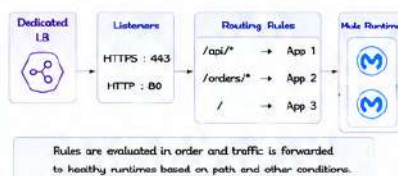
## KEY CAPABILITIES

- Dedicated and isolated load balancer for your VPC.
- Static public IP addresses for predictable access.
- High availability with automatic failover across nodes.
- TLS termination with custom certificates and ciphers.
- Layer 7 (HTTP/HTTPS) intelligent routing.
- Custom timeouts and connection settings.
- IP allowlisting and access control.
- Detailed access logs and metrics.
- Integration with Anypoint Runtime Fabric.

## FEATURES

- Dedicated**: Isolated load balancer just for your organization.
- Static IPs**: One or more static public IP addresses.
- Performance**: Predictable performance and throughput.
- Security**: TLS 1.2 / 1.3, modern ciphers, IP allowlisting.
- High Availability**: Multi-AZ, auto-failover, health monitoring.
- Full Control**: Listeners, rules, timeouts, and certificates.
- Observability**: Metrics, logs, and health dashboards.

## LISTENER &amp; ROUTING EXAMPLE



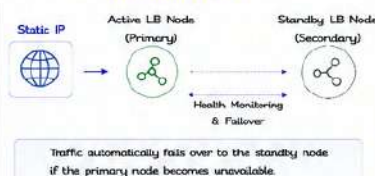
## BENEFITS

- ✓ Better performance and predictable capacity.
- ✓ Enhanced security with isolation and controls.
- ✓ Consistent entry point with static IPs.
- ✓ Improved reliability and high availability.
- ✓ Greater control over traffic management.
- ✓ Ideal for production and mission-critical workloads.

## CONFIGURATION OPTIONS

- Listeners**: HTTP / HTTPS
- TLS Certificates**: Custom or MuleSoft managed
- Ciphers & Protocols**: TLS 1.2 / 1.3, modern ciphers
- Timeouts**: Client, server, and idle timeouts
- IP Allowlist**: Restrict access by IP ranges
- HTTP Settings**: Compression, headers, max size
- Access Logs**: Enable and publish to Object Store.

## HIGH AVAILABILITY



## WHEN TO USE DEDICATED LOAD BALANCER?

- ✓ When you need static IP addresses.
- ✓ When you require better performance and isolation.
- ✓ When you need custom TLS, ciphers, or timeouts.
- ✓ When you need IP allowlisting or strict access control.
- ✓ For mission-critical and regulated workloads.
- ✓ When you expect high or unpredictable traffic.

## DEDICATED vs SHARED LOAD BALANCER

| Capability    | Shared Load Balancer        | Dedicated Load Balancer             |
|---------------|-----------------------------|-------------------------------------|
| Isolation     | Shared with other customers | Fully isolated                      |
| IP Addresses  | Dynamic                     | Static (one or more)                |
| Performance   | Best-effort                 | Predictable & consistent            |
| Customization | Limited                     | Full control                        |
| Security      | Standard                    | Enhanced (IP allowlist, TLS)        |
| Use Cases     | Dev/Test, Low traffic       | Production, High traffic, Regulated |

## SUPPORTED TLS &amp; CIPHERS (EXAMPLE)

| Category                      | Support                                   |
|-------------------------------|-------------------------------------------|
| TLS Versions                  | TLS 1.2, TLS 1.3                          |
| Cipher Suites                 | Modern suites (e.g. ECDHE, AES-GCM, etc.) |
| Certificate Types             | RSA, ECDSA                                |
| 9th (Forward) Name Indication | Supported                                 |
| Mutual TLS                    | Supported (mTLS)                          |

## TIMEOUTS (EXAMPLE)

| Timeout        | Description                       | Example (Default) |
|----------------|-----------------------------------|-------------------|
| Client Timeout | Maximum time to receive a request | 60s               |
| Server Timeout | Maximum time waiting for response | 60s               |
| Idle Timeout   | Idle connection timeout           | 30s               |
| TCP Timeout    | TCP connection timeout            | 75s               |

## END-TO-END FLOW



## REAL-WORLD USE CASES

- Expose APIs to the internet with static IPs.
- Meet regulatory or compliance requirements.
- Protect APIs with IP allowlisting and TLS.
- Ensure consistent performance for critical APIs.
- Support custom TLS and security policies.

## BEST PRACTICES

- ✓ Use HTTPS with TLS 1.2 / 1.3.
- ✓ Restrict access using IP allowlisting.
- ✓ Enable access logs and monitor regularly.
- ✓ Use appropriate timeouts for your workload.
- ✓ Test failover and high availability.

## TROUBLESHOOTING TIPS

- Check listener and routing rule configurations.
- Verify backend runtimes are healthy.
- Review access logs for errors or blocks.
- Validate TLS certificate and cipher settings.
- Confirm security group and NACL rules.

## KEY TAKEAWAY

Dedicated Load Balancer provides a secure, isolated, and high-performance entry point with static IPs and full control—ideal for production and mission-critical Mule applications.



Dedicated Load Balancer gives you the **control**, **performance**, and **security** you need to deliver reliable and scalable APIs.



## MODULE 9

# 9.8 WORKER SIZE



Worker size defines the amount of CPU, memory, and resources allocated to a Mule runtime instance (worker). Choosing the right worker size helps you optimize **performance**, **reliability**, and **cost** for your Mule applications.

### WHAT IS WORKER SIZE?

- A worker is a Mule runtime instance that runs your application.
- Worker size determines the amount of vCPU and memory available to each worker.
- You can configure worker size when deploying to CloudHub 2.0.
- The right size ensures optimal performance and cost efficiency.

### HOW WORKER SIZE IMPACTS YOUR APPLICATION



### CLOUDHUB 2.0 WORKER SIZES

| Worker Size | vCPU | Memory (GB) | Use Cases / Characteristics                     |
|-------------|------|-------------|-------------------------------------------------|
| 0.1 vCore   | 0.1  | 0.2         | Development, demos, low throughput, simple apps |
| 0.2 vCore   | 0.2  | 0.4         | Small apps, low traffic                         |
| 0.5 vCore   | 0.5  | 1           | Light workloads, small to medium traffic        |
| 1 vCore     | 1    | 2           | General purpose, medium traffic                 |
| 2 vCore     | 2    | 4           | Higher throughput, heavier processing           |
| 4 vCore     | 4    | 8           | High traffic, complex flows                     |
| 8 vCore     | 8    | 16          | Very high traffic, CPU intensive workloads      |
| 16 vCore    | 16   | 32          | Extreme workloads, large payloads, batch jobs   |

Note: vCPU and memory values are indicative. MuleSoft may update available sizes over time.

### KEY CONSIDERATIONS

- Application Profile**: Understand the CPU, memory, and I/O requirements of your application.
- Throughput & Concurrency**: Higher load or more concurrent requests need larger workers.
- Memory Usage**: Large payloads, data transformations, or caching require more memory.
- Latency Sensitivity**: CPU-intensive or low-latency use cases may need more vCPUs.
- Cost Optimization**: Start smaller, monitor, and scale up only when needed.

### HOW TO CONFIGURE WORKER SIZE (CLOUDHUB 2.0)



### HOW TO CHOOSE THE RIGHT WORKER SIZE

- Start small and load test your application.
- Monitor CPU utilization, memory usage, and response time.
- If CPU > 70% consistently or frequent throttling occurs, go higher.
- If memory is > 75% consistently or OOM errors occur, go higher.
- Balance worker size (resources per worker) with worker count (scale out).

### RECOMMENDED STARTING POINTS

| Use Case                            | Suggested Worker Size | Notes                             |
|-------------------------------------|-----------------------|-----------------------------------|
| Development / Test                  | 0.1 - 0.2 vCore       | Cost-effective for non-production |
| Low Traffic / Simple APIs           | 0.2 - 0.5 vCore       | Good for small apps               |
| Medium Traffic / Standard APIs      | 0.5 - 1 vCore         | Balanced performance and cost     |
| High Traffic / Complex Processing   | 2 - 4 vCore           | For higher throughput             |
| Very High Traffic / Heavy Workloads | 8+ vCore              | For extreme workloads             |

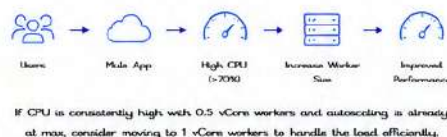
### MONITORING METRICS TO WATCH

- CPU Utilization (%)** - High usage indicates need for more vCPU.
- Memory Utilization (%)** - High usage or frequent GC implies need for more memory.
- Response Time / Latency** - Degradation indicates need for tuning.
- Throughput (TPS/RPS)** - Ensure capacity meets demand.
- Error Rate / Throttling** - May indicate insufficient resources.

### BEST PRACTICES

- Always test under production-like load.
- Use Auto-scaling with the right worker size.
- Avoid over-provisioning (costs) and under-provisioning (performance issues).
- Regularly review metrics and right-size.
- Consider both worker size and worker count.

### EXAMPLE SCENARIO



### POTENTIAL OUTCOMES

- Right size**: Optimal performance, stability, and cost.
- Too Small**: High CPU/memory, timeouts, errors, restarts.
- Too Large**: Increased cost with no performance benefit.

### WORKER SIZE vs WORKER COUNT

- |                                                                                                                                                                                                             |    |                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Worker Size (Scale Up)</b> <ul style="list-style-type: none"> <li>More resources per worker (vCPU, Memory)</li> <li>Better for CPU/memory intensive workloads</li> <li>Less number of workers</li> </ul> | VS | <b>Worker Count (Scale Out)</b> <ul style="list-style-type: none"> <li>More workers, same size</li> <li>Better for handling more concurrent requests</li> <li>Improves availability and fault tolerance</li> </ul> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### RELATED FEATURES

- Auto-scaling** - Adjusts worker count based on load.
- VPC & Load Balancer** - Network and traffic management.
- Persistent Queue** - Decouple and handle load spikes.
- Monitoring & Alerts** - Ensure proactive tuning.



### KEY TAKEAWAY

Choosing the right worker size ensures your Mule applications run fast, reliably, and cost-effectively. Monitor, test, and right-size continuously to adapt to changing business needs.

Right worker size + Right worker count = High performance, High reliability, and Optimal cost.

## MODULE 9

# 9.9 HORIZONTAL SCALING

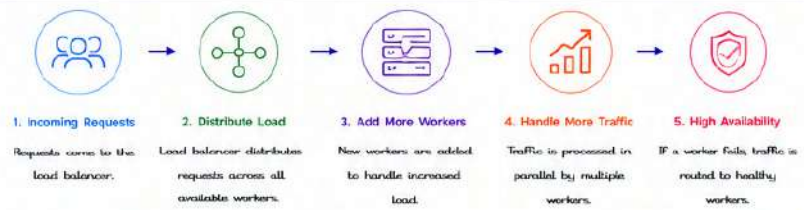


Horizontal scaling (scale out) in MuleSoft increases capacity by adding more worker instances to handle higher load. It improves **performance**, **reliability**, and **availability** without changing your application.

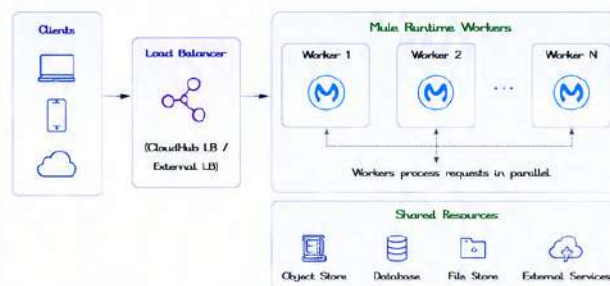
### WHAT IS HORIZONTAL SCALING?

- Add more Mule runtime workers to distribute the load.
- Each worker is identical and handles a portion of the traffic.
- No changes to application code.
- Workers are added or removed based on demand (manual or auto-scaling).
- Improves throughput, availability, and fault tolerance.

### HOW HORIZONTAL SCALING WORKS



### ARCHITECTURE OVERVIEW



### KEY BENEFITS

- **Increased Capacity:** Handle more traffic and higher loads.
- **High Availability:** Traffic is automatically routed to healthy workers.
- **Better Performance:** Parallel processing reduces response time.
- **Fault Tolerance:** If a worker fails, others continue processing.
- **Cost Effective:** Scale only when needed.

### HORIZONTAL SCALING OPTIONS



### AUTO-SCALING POLICY EXAMPLE

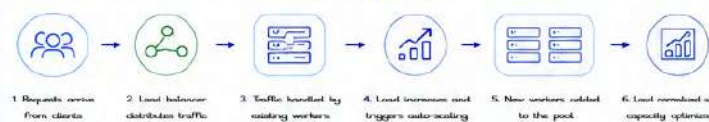
- **Scale out when:**
  - CPU > 70% for 5 minutes
  - Average response time > 1 sec
  - Queue depth > 1000 messages
- **Scale in when:**
  - CPU < 30% for 10 minutes
  - Queue depth < 200 messages
- **Min workers:** 2  
**Max workers:** 10



### WHEN TO USE HORIZONTAL SCALING?

- ✓ When traffic volume increases.
- ✓ When application performance degrades under load.
- ✓ When you need higher availability and fault tolerance.
- ✓ When you want to handle traffic spikes.
- ✓ When your workload is stateless or can be made stateless.

### HORIZONTAL SCALING FLOW



### THINGS TO CONSIDER

- ⚠ Application must be stateless or store state in external systems.
- ⚠ Shared resources (DB, queues, object store) should handle increased connections.
- ⚠ Set proper auto-scaling policies to avoid throttling.
- ⚠ Monitor performance and costs regularly.

### CLOUDHUB 2.0 HORIZONTAL SCALING

| Feature          | Description                                                                |
|------------------|----------------------------------------------------------------------------|
| Worker Instances | Multiple identical worker instances behind a load balancer.                |
| Auto-Scaling     | Scale workers based on CPU, memory, response time, or message queue depth. |
| Min/Max Workers  | Configure minimum and maximum number of workers.                           |
| Zero-Downtime    | Workers are added/removed without affecting traffic.                       |
| Region & Zones   | Workers distributed across availability zones for resilience.              |

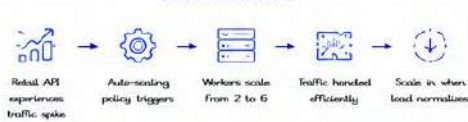
### SCALING METRICS (EXAMPLES)

| Metric                     | Description                                        |
|----------------------------|----------------------------------------------------|
| CPU Utilization (%)        | High CPU indicates need for more workers.          |
| Memory Utilization (%)     | High memory usage indicates need for more workers. |
| Average Response Time (ms) | High response time indicates performance issues.   |
| Request Rate (RPS)         | High requests per second indicates load increase.  |
| Queue Depth                | High queue depth indicates backlog and load.       |

### BEST PRACTICES

- ✓ Design applications to be stateless.
- ✓ Use external systems for state management.
- ✓ Define sensible auto-scaling policies.
- ✓ Monitor key metrics and logs.
- ✓ Test scaling under load.
- ✓ Use connection pooling for databases.

### EXAMPLE SCENARIO



### POTENTIAL OUTCOMES

- ✓ **Right Scaling:** High performance, low latency, happy users.
- ⚠ **Under Scaling:** Slow response, timeouts, poor user experience.
- ✗ **Over Scaling:** Increased cost, underutilized resources.

### RELATED FEATURES

- Auto-Scaling
- Dedicated Load Balancer
- Worker Size
- Runtime Fabric
- Monitoring & Alerts



### KEY TAKEAWAY

Horizontal scaling lets you handle more load by adding more workers. With the right policies and monitoring, you get the best balance of performance, reliability, and cost.



### REMEMBER

Scale out to handle more, scale in to save more!



Horizontal scaling = More workers, more power, better performance!



**MODULE  
9**

# 9.10 VERTICAL SCALING

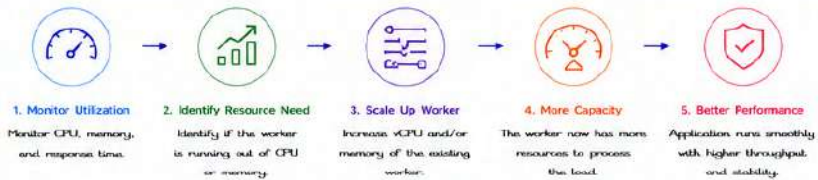


Vertical scaling (scale up) in MuleSoft increases the capacity of an existing worker by allocating more resources (CPU and memory) to handle higher load. It improves **performance** and **stability** without changing the number of worker instances.

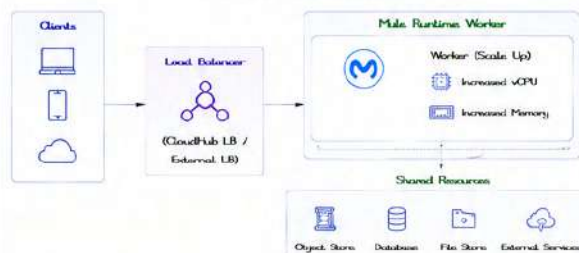
## WHAT IS VERTICAL SCALING?

- Increase the size (resources) of an existing worker.
- Allocate more vCPU and memory to the worker.
- No changes to application code.
- The number of workers remains the same.
- Useful when a single worker is hitting its resource limits.
- Improves performance and avoids resource-related errors.

## HOW VERTICAL SCALING WORKS



## ARCHITECTURE OVERVIEW



## KEY BENEFITS

- Better Performance:** More CPU and memory handle higher loads and complex processing.
- Improved Stability:** Reduces timeouts, errors, and throttling caused by resource constraints.
- Simplicity:** No change in architecture or number of workers.
- Cost Effective:** Cost-efficient up to a point compared to adding more workers.
- Ideal For Stateful Workloads:** Useful when the workload requires more resources per instance.

## CLOUDHUB 2.0 WORKER SIZES (VERTICAL SCALING OPTIONS)

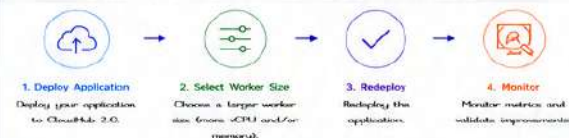
| Worker Size | vCPU | Memory (GB) | Use Cases / Characteristics                       |
|-------------|------|-------------|---------------------------------------------------|
| 0.1 vCores  | 0.1  | 0.2         | Development, clones, low throughput, simple apps. |
| 0.2 vCores  | 0.2  | 0.4         | Small apps, low traffic.                          |
| 0.5 vCores  | 0.5  | 1           | Light workloads, small to medium traffic.         |
| 1 vCore     | 1    | 2           | General purpose, medium traffic.                  |
| 2 vCores    | 2    | 4           | Higher throughput, heavier processing.            |
| 4 vCores    | 4    | 8           | High traffic, complex flows.                      |
| 8 vCores    | 8    | 16          | Very high traffic, CPU intensive workloads.       |
| 16 vCores   | 16   | 32          | Extreme workloads, large payloads, batch jobs.    |

① Note: vCPU and memory values are inclusive. MuleSoft may update available sizes over time.

## KEY CONSIDERATIONS

- Application Profile:** Understand the CPU, memory, and I/O requirements of your application.
- Resource Limits:** Every worker size has limits. Scale up within appropriate boundaries.
- Monitoring is Essential:** Regularly monitor CPU, memory, response time, and error rates.
- Not Always the Answer:** If the workload continues to grow, horizontal scaling may be more appropriate.
- Downtime for Scale Up:** Scaling up a worker requires redeployment (brief downtime).

## HOW TO CONFIGURE VERTICAL SCALING (CLOUDHUB 2.0)



## WHEN TO USE VERTICAL SCALING?

- ✓ When a single worker's CPU or memory usage is consistently high.
- ✓ When you need better performance without adding more workers.
- ✓ When the workload is stateful or session-dependent.
- ✓ When increasing resources is more cost-effective than adding workers.
- ✓ When traffic increase is moderate and can be handled by a bigger worker.

## RECOMMENDED STARTING POINTS

| Use Case                            | Suggested Worker Size | Notes                              |
|-------------------------------------|-----------------------|------------------------------------|
| Development / Test                  | 0.1 - 0.2 vCores      | Cost-effective for non-production. |
| Low Traffic / Simple APIs           | 0.2 - 0.5 vCores      | Good for small apps.               |
| Medium Traffic / Standard APIs      | 0.5 - 1 vCore         | Balanced performance and cost.     |
| High Traffic / Complex Processing   | 2 - 4 vCores          | For higher throughput.             |
| Very High Traffic / Heavy Workloads | 8+ vCores             | For extreme workloads.             |

## MONITORING METRICS TO WATCH

- CPU Utilization (%)** - High usage indicates need for more vCPU.
- Memory Utilization (%)** - High usage or frequent GC implies need for more memory.
- Response Time / Latency** - Degradation indicates need for more resources.
- Throughput (TPS/RPS)** - Ensures capacity meets demand.
- Error Rates / Throttling** - May indicate insufficient resources.

## VERTICAL vs HORIZONTAL SCALING

|                   | Vertical Scaling (Scale Up)            | Horizontal Scaling (Scale Out) |
|-------------------|----------------------------------------|--------------------------------|
| What it does      | Increase resources of existing worker. | Add more worker instances.     |
| Number of Workers | Same.                                  | Increases.                     |
| Complexity        | Simple.                                | Higher.                        |
| Cost              | Lower up to a point.                   | Higher as workers increase.    |
| Use Case          | High resource usage on single worker.  | High traffic or concurrency.   |
| Downtime          | Requires Redeploy.                     | Not required (Auto-scaling).   |

## EXAMPLE SCENARIO



## BEST PRACTICES

- ✓ Right-size your worker based on observed metrics.
- ✓ Avoid over-provisioning to control costs.
- ✓ Regularly review and tune resources.
- ✓ Use monitoring and alerts to identify bottlenecks.
- ✓ Combine with horizontal scaling when needed.

## RELATED FEATURES

- Worker Size
- Horizontal Scaling
- Auto-scaling Policies
- Monitoring & Alerts
- Runtime Manager

## KEY TAKEAWAY

Vertical scaling (scale up) increases the power of your existing worker by adding more CPU and memory. It is simple, cost-effective, and ideal for handling increased load on a single instance.

## REMEMBER

Scale up to give your worker more power when it needs it most!

💡 Right-size your worker = Better performance, higher reliability, and optimized cost.

## MODULE 9

# 9.11 ZERO DOWNTIME DEPLOYMENT



Zero Downtime Deployment ensures your Mule applications are updated without interrupting in-flight requests. MuleSoft achieves this using rolling deployments with automatic traffic shift and health checks, delivering *seamless* and *reliable* releases.

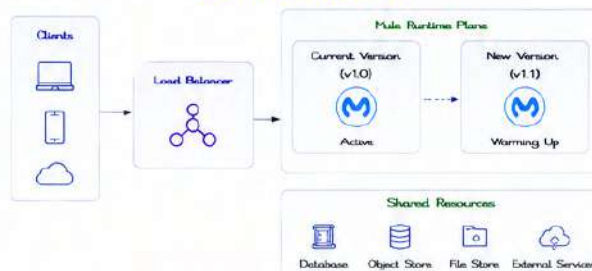
### WHAT IS ZERO DOWNTIME DEPLOYMENT?

- Updates are deployed without stopping the application or dropping requests.
- New version is deployed alongside the current version.
- Traffic is gradually shifted to the new version.
- If issues are detected, traffic automatically shifts back.
- Ensures high availability and better user experience.

### HOW ZERO DOWNTIME DEPLOYMENT WORKS



### ARCHITECTURE OVERVIEW



### KEY BENEFITS

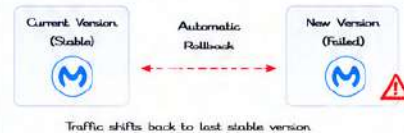
- No Downtime:** Continuous availability during deployments.
- High Availability:** Automatic traffic management and rollback.
- Better User Experience:** No failed requests or interruptions.
- Safe Deployments:** Validate new version with real traffic.
- Operational Efficiency:** Faster releases with reduced risk.

### DEPLOYMENT FLOW (ROLLING DEPLOYMENT)



### AUTOMATIC ROLLBACK

If the new version fails health checks or metrics exceed thresholds, MuleSoft automatically shifts traffic back to the last stable version.



### DEPLOYMENT STRATEGIES

- Rolling Deployment (Default)**
  - New version is deployed alongside current version.
  - Traffic is shifted gradually.
  - Old version is removed after 100% traffic on new version.
- Blue/Green Deployment (Advanced)**
  - Two identical environments (Blue & Green).
  - Traffic switched to the new environment once validated.
  - Old environment is kept as fallback until switched.

### KEY CONSIDERATIONS

- ✓ Ensure backward compatibility of APIs when possible.
- ✓ Use proper health checks (HTTP, custom, DB connectivity, etc.).
- ✓ Monitor metrics and set appropriate alert thresholds.
- ✓ Test in lower environments before production deployment.
- ✓ Use canary release approach for critical applications.

### EXAMPLE SCENARIO

You are deploying version v1.1 of your Mule application.



### BEST PRACTICES

- ✓ Keep deployments frequent and small.
- ✓ Automate health checks and validations.
- ✓ Use proper alerting and monitoring.
- ✓ Validate business-critical flows after deployment.
- ✓ Have a rollback plan and test it regularly.

### MONITORING METRICS TO WATCH

- Response Time / Latency**  
Detect performance degradation.
- Error Rate (%)**  
Monitor increase in errors.
- Throughput (TPS/RPS)**  
Ensure capacity meets demand.
- CPU & Memory Utilization**  
Ensure new version is healthy.

### SUPPORTED RUNTIMES

Zero downtime deployment is supported on:

- CloudHub 2.0
- Runtime Fabric
- Kubernetes (Using Mule 4 rolling update)

### COMMON USE CASES

- API and integration updates
- Bug fixes and patches
- Feature rollouts
- Security updates
- Configuration changes

Zero downtime deployment = Continuous availability + Safe releases + Happy users!



MODULE  
10

## 10.1 STREAMING



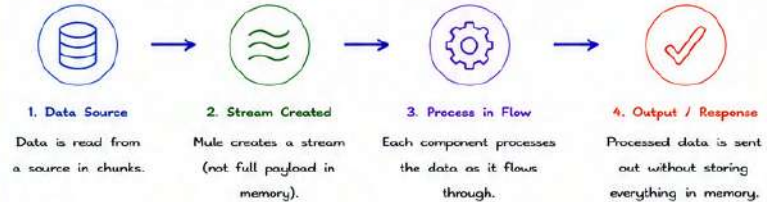
Streaming in MuleSoft allows data to be processed as it flows through the application, instead of loading the entire data into memory at once.

It helps in handling large payloads efficiently and improves performance and scalability.

## WHAT IS STREAMING?

- Streaming is the ability to process data in chunks as it flows through the application.
- Data is not fully loaded into memory.
- Reduces memory usage and GC impact.
- Improves throughput for large payloads.
- Enabled by default in Mule 4.

## HOW STREAMING WORKS



## STREAMING vs NON-STREAMING

## Streaming (Default in Mule 4)

- ✓ Data is processed in chunks.
- ✓ Low memory usage.
- ✓ Better for large payloads.
- ✓ Higher throughput.
- ✓ Garbage Collection (GC) impact is reduced.

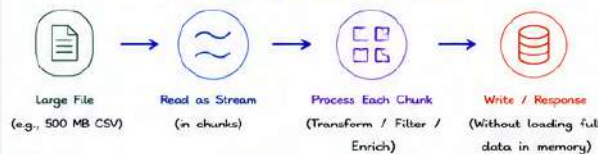
## Non-Streaming (Memory Mode)

- ✗ Entire payload is loaded into memory.
- ✗ High memory usage.
- ✗ May cause OutOfMemoryError for large payloads.
- ✗ Lower throughput.
- ✗ Higher GC impact.

## WHEN TO USE STREAMING?

- When handling large files (e.g., CSV, JSON, XML).
- When payload size is unknown or very large.
- When connecting to databases, queues, or file systems with large data.
- When you want to improve performance and scalability.
- When low memory footprint is critical.

## EXAMPLE: STREAMING DATA FLOW



Only a small portion (chunk) of data stays in memory at any given time. Mule processes and discards it before reading the next chunk.

## KEY BENEFITS

- Lower Memory Usage:** Process large payloads without exhausting memory.
- Better Performance:** Higher throughput and faster response times.
- Scalability:** Handle more requests with same resources.
- Stability:** Reduces GC pauses and OutOfMemory errors.
- Cost Effective:** Run large workloads on smaller servers.

## TYPES OF PAYLOADS IN MULE 4

| Type                | Description                               | Memory Usage  | Repeatable?            |
|---------------------|-------------------------------------------|---------------|------------------------|
| Streaming (Default) | Data is streamed in chunks.               | Low           | No<br>(Non-repeatable) |
| Repeatable Stream   | Stream is cached so it can be read again. | Medium / High | Yes                    |
| In-memory (Buffer)  | Entire payload is loaded in memory.       | High          | Yes                    |

## HOW TO ENABLE / CONTROL STREAMING

- Streaming is enabled by default in Mule 4.
- You can enable it explicitly in DataSense / Global Configuration:

```
<mule-streaming:config name="Streaming_Config"
  maxBufferSize="512KB"
  bufferUnit="KB"/>
```

- To disable streaming (not recommended):

```
<mule-streaming:config name="Non_Streaming_Config"
  streamingStrategy="OFF"/>
```

## BEST PRACTICES

- ✓ Always prefer streaming for large payloads.
- ✓ Avoid converting streams to String or Object unless necessary.
- ✓ Use Repeatable Streams only when the payload needs to be read multiple times.
- ✓ Monitor memory and tuning parameters for optimal performance.
- ✓ Test with realistic payload sizes.

## INTERVIEW TIPS

- Q. What is streaming in MuleSoft?
- Q. What is the difference between streaming and non-streaming?
- Q. What are repeatable and non-repeatable streams?
- Q. How does streaming improve performance?
- Q. How do you enable / disable streaming?

## REAL-WORLD EXAMPLE

A finance company receives a 1 GB daily transaction file in CSV format. Using streaming, MuleSoft processes the file in chunks, validates each record, and writes to the database without loading the entire file into memory. This ensures fast processing and no memory issues.



## KEY TAKEAWAY

Streaming is the backbone of high-performance Mule applications.

It allows you to process large data efficiently with low memory usage, high throughput, and better scalability.

**MODULE  
10**

# 10.2 BATCH PROCESSING

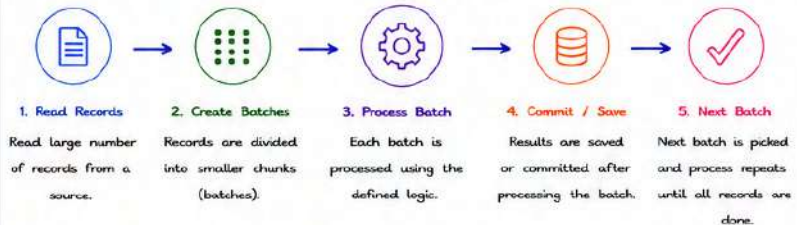


Batch Processing in MuleSoft allows you to group and process a large number of records in chunks (batches) instead of one by one. It improves **throughput**, reduces resource consumption, and is ideal for handling large volumes of data **efficiently**.

## WHAT IS BATCH PROCESSING?

- Processing a large set of records in smaller chunks called batches.
- Each batch is processed as a unit.
- Reduces the number of transactions and improves performance.
- Ideal for large data operations like file processing, DB operations, etc.

## HOW BATCH PROCESSING WORKS



## BATCH vs SINGLE RECORD PROCESSING

### Single Record Processing

- ❌ Processes one record at a time.
- ❌ High number of transactions.
- ❌ Slower for large volumes.
- ❌ More system resources used.

### Batch Processing

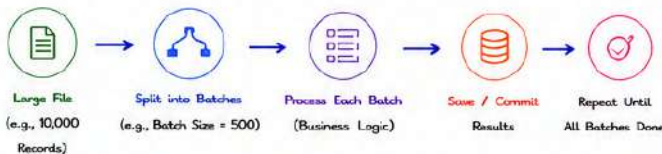
- ✅ Processes multiple records at once.
- ✅ Fewer transactions.
- ✅ Faster and more efficient.
- ✅ Optimized resource utilization.
- ✅ Better scalability.



## WHEN TO USE BATCH PROCESSING?

- When processing large files (CSV, Excel, XML, JSON).
- When loading large data into a database.
- When performing bulk updates or inserts.
- When integrating with systems that support bulk operations.
- When you want to improve performance and reduce execution time.

## EXAMPLE: BATCH PROCESSING FLOW



Instead of 10,000 transactions, if batch size is 500, only 20 transactions will be executed. Huge performance gain!



## KEY BENEFITS

- High Performance:** Reduces execution time for large datasets.
- Lower Resource Usage:** Efficient memory and CPU utilization.
- Scalability:** Handles more records with same infrastructure.
- Reliability:** Fewer failures and easy retry at batch level.
- Cost Effective:** Reduces load on systems and infrastructure.

## BATCH CONFIGURATION COMPONENTS

| Component        | Description                                   |
|------------------|-----------------------------------------------|
| Batch Job        | Controls the overall batch execution.         |
| Batch Input      | Reads records from the source into batches.   |
| Batch Step       | Processes each batch.                         |
| Batch Commit     | Commits or saves the results.                 |
| Batch Aggregator | (Optional) Aggregates results of all batches. |

## COMMON PROPERTIES

| Property         | Description                                                  |
|------------------|--------------------------------------------------------------|
| batchSize        | Number of records in each batch.                             |
| maxFailedRecords | Max number of failed records allowed before failing the job. |
| continueOnError  | If true, job continues even if some records fail.            |
| initialState     | Initial state of the job (STARTED or STOPPED).               |

## USE CASE EXAMPLES

- Processing daily large CSV files.
- Bulk insert/update in database.
- Sending bulk emails.
- Migrating large data between systems.
- Generating reports from large datasets.

## BEST PRACTICES

- ✓ Choose an optimal batch size (too small = slow, too large = memory issues).
- ✓ Use streaming with batch for huge data.
- ✓ Handle exceptions and retries at batch level.
- ✓ Log batch statistics (success, failure, time).
- ✓ Test with different batch sizes to find optimal.

## INTERVIEW TIPS

- Q. What is Batch Processing in MuleSoft?
- Q. How is it different from single record processing?
- Q. What are the key components of Batch Processing?
- Q. How does batch size impact performance?
- Q. What is the use of maxFailedRecords?
- Q. When would you use Batch Processing in real-time projects?

## REAL-WORLD EXAMPLE

A retail company receives a file with 50,000 orders every day. Instead of inserting each order one by one into the database, MuleSoft splits the data into batches of 1,000 orders. Each batch is processed and inserted in a single transaction, resulting in faster processing and less load on the database.



## KEY TAKEAWAY

Batch Processing helps you handle large volumes of data efficiently by processing records in chunks, reducing execution time, resource usage, and improving system performance and scalability.



**MODULE  
10**

# 10.3 MEMORY OPTIMISATION

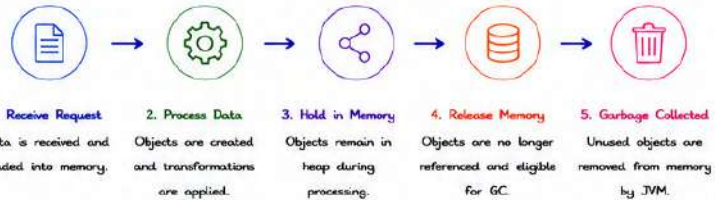


Memory Optimization in MuleSoft ensures efficient use of heap memory by reducing unnecessary object creation, managing data streams, and using the right configurations. It helps improve **performance**, prevent **OutOfMemoryError** and ensure application **stability**.

## WHAT IS MEMORY OPTIMIZATION?

- Ensures efficient usage of JVM heap memory.
- Reduces memory consumption and GC pressure.
- Prevents OutOfMemoryError (OOM).
- Improves throughput and response time.
- Involves optimal configuration, streaming, caching, and data handling.

## HOW MEMORY IS USED IN MULE APPLICATIONS



## WHY MEMORY OPTIMIZATION IS IMPORTANT?



- High memory usage causes frequent Garbage Collection (GC).
- Long GC pauses lead to performance degradation.
- OutOfMemoryError can crash the application.
- Efficient memory usage improves scalability and stability.
- Lower infrastructure cost and better resource utilization.



## KEY AREAS FOR MEMORY OPTIMIZATION

- ✓ **Use Streaming:** Avoid loading large payloads completely in memory.
- ✓ **Avoid Unnecessary Object Creation:** Reuse objects and variables.
- ✓ **Use Repeatable Streams Wisely:** Only when needed.
- ✓ **Efficient Data Transformations:** Avoid heavy transformations.
- ✓ **Use Caching & Object Store:** Reduce repeated processing.
- ✓ **Proper Configuration:** Tune JVM, heap size, and buffers.

## BEST PRACTICES

- Use Streaming (Default in Mule 4):** Process data in chunks instead of loading entire payload.
- Use Repeatable Streams Only When Needed:** Non-repeatable streams consume less memory.
- Avoid Unnecessary Data in Message:** Remove unused variables and properties.
- Avoid Large Payload Transformations:** Transform only required fields.
- Use Caching:** Cache frequently used data to avoid repeated calls.
- Use Object Store for Large Data:** Store large or temporary data outside heap.
- Optimize JVM Settings:** Configure heap size, GC, and metaspace properly.
- Monitor Memory Usage:** Track heap usage and GC logs regularly.

## COMMON MEMORY ISSUES

| Issue            | Cause                                                        | Impact                          | Solution                                   |
|------------------|--------------------------------------------------------------|---------------------------------|--------------------------------------------|
| High Heap Usage  | Large payloads, non-streaming, heavy transformations         | Slow performance, GC pauses     | Enable streaming, optimize transformations |
| OutOfMemoryError | Insufficient heap or memory leak                             | Application crash, downtime     | Increase heap size, fix memory leak        |
| Frequent GC      | Too many objects created                                     | High CPU usage, slow throughput | Reduce object creation, optimize code      |
| Memory Leak      | Objects not released (unclosed resources, static references) | Continuous memory growth        | Close resources, fix references            |

## CONFIGURATION TIPS

- Set Appropriate Heap Size**
  - Configure heap size based on workload and server capacity.
 

```
-Xms512m -Xmx1024m
```
- Use G1GC (Recommended)**
  - G1GC handles large heap efficiently and reduces GC pauses.
 

```
-XX:+UseG1GC
```
- Tune JVM Parameters**
  - Adjust based on monitoring and application behavior.
 

```
-XX:MaxGCPauseMillis=200
```
- Increase Streaming Buffer Size (if needed)**
  - Adjust buffer size for better throughput without high memory usage.
 

```
<inbound-streaming:config name="streamingConfig" maxBufferSize="512KB" bufferUnit="KB"/>
```

## EXAMPLE: EFFECT OF STREAMING

|               | Without Streaming<br>(Load Entire Payload) | With Streaming<br>(Process in Chunks) |
|---------------|--------------------------------------------|---------------------------------------|
| Memory Usage  | High                                       | Low                                   |
| GC Frequency  | High                                       | Low                                   |
| Response Time | Slow                                       | Fast                                  |
| Scalability   | Low                                        | High                                  |
| Stability     | Low                                        | High                                  |

## TOOLS FOR MEMORY MONITORING

- Runtime Manager:** Monitor heap usage, GC, and worker stats.
- JMX / VisualVM:** Analyze memory usage and heap dumps.
- GC Logs:** Analyze GC behavior and pauses.
- Anypoint Monitoring:** Custom dashboards and alerts.

## INTERVIEW TIPS

- Q. What is Memory Optimization in MuleSoft?
- Q. Why is streaming important for memory optimization?
- Q. Difference between repeatable and non-repeatable streams?
- Q. What causes OutOfMemoryError?
- Q. How do you reduce memory usage in Mule applications?
- Q. What JVM settings are important for Mule applications?

## REAL-WORLD EXAMPLE

A retail company processes large order files (10GB) daily.

- Without optimization: High memory usage, frequent GC, and eventual OOM errors.
- With optimization: Streaming enabled, payload processed in chunks, caching used for lookups, memory usage reduced by 70%, throughput improved by 3x.

## KEY TAKEAWAY

Memory Optimization is all about using memory efficiently by processing data smartly, avoiding unnecessary objects, and tuning the JVM. It improves performance, prevents failures, and ensures your Mule application can handle more with less.



Optimize memory today for a faster, more stable, and scalable Mule application tomorrow!

**MODULE  
10**

# 10.4 CONNECTION POOLING

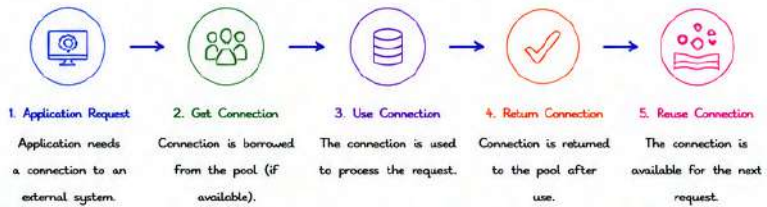


Connection Pooling in MuleSoft allows reusable connections to external systems (databases, APIs, JMS, etc.) instead of creating a new connection for every request. It improves performance, reduces latency, and optimizes resource usage.

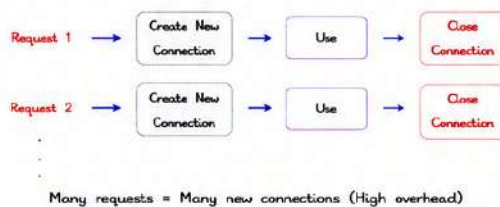
## WHAT IS CONNECTION POOLING?

- Maintains a pool of reusable connections.
- Reuses existing connections instead of creating new ones.
- Reduces connection creation overhead.
- Improves response time and throughput.
- Prevents exhaustion of resources (threads, sockets, DB connections).

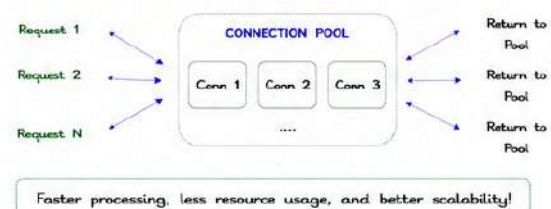
## HOW CONNECTION POOLING WORKS



## WITHOUT CONNECTION POOLING



## WITH CONNECTION POOLING



## WHEN TO USE CONNECTION POOLING?

- ✓ When connecting to databases (JDBC, RDBMS).
- ✓ When calling external APIs frequently.
- ✓ When working with JMS/Exchange connections.
- ✓ When processing high volume of requests.
- ✓ When connection creation is expensive.



## KEY BENEFITS

- Better Performance:** Reuses existing connections, reduces latency.
- High Throughput:** Handles more requests with same resources.
- Resource Optimization:** Reduces CPU, memory and network usage.
- Scalability:** Supports more concurrent users and transactions.
- Reliability:** Reduces failures due to connection timeouts.

## COMMON CONFIGURATION PROPERTIES (EXAMPLE: HTTP / DATABASE)

| Property                                   | Description                                                           | Example Value |
|--------------------------------------------|-----------------------------------------------------------------------|---------------|
| <code>maxConnections</code>                | Maximum number of connections in the pool.                            | 10            |
| <code>maxWait</code>                       | Maximum time to wait for a connection when pool is exhausted (in ms). | 30000         |
| <code>validationQuery</code>               | Query used to validate a connection (before using it).                | SELECT 1      |
| <code>idleTimeout</code>                   | Time (in ms) after which an idle connection is closed.                | 600000        |
| <code>timeBetweenEvictionRunsMillis</code> | Interval (in ms) to check idle connections for eviction.              | 30000         |

## SUPPORTED CONNECTORS WITH POOLING

| Connector Type | Examples                                        |
|----------------|-------------------------------------------------|
| Database       | JDBC, Oracle, MySQL, Postgres, SQL Server, etc. |
| HTTP / HTTPS   | HTTP Request, HTTP Listener (outbound)          |
| Email          | SMTP, IMAP, POP3                                |
| JMS            | ActiveMQ, Arqpoint MQ                           |
| Others         | FTP, SFTP, Redis, etc.                          |

## BEST PRACTICES

- ✓ Set optimal `maxConnections` based on system capacity.
- ✓ Set `maxWait` to a reasonable value to avoid long delays.
- ✓ Use `validationQuery` to ensure healthy connections.
- ✓ Enable `idleTimeout` and eviction to remove stale connections.
- ✓ Monitor pool usage and tune regularly.

## INTERVIEW TIPS

- Q. What is connection pooling in MuleSoft?
- Q. Why do we use connection pooling?
- Q. What happens when the pool is exhausted?
- Q. What are important pool configuration parameters?
- Q. How does pooling improve performance?

## REAL-WORLD EXAMPLE

An e-commerce application calls a payment gateway API for every order. Without connection pooling, a new HTTP connection is created for each call, increasing latency and resource usage. With pooling, connections are reused, reducing response time by 60% and improving throughput by 3x.



## KEY TAKEAWAY

Connection Pooling is essential for high performance and scalability in Mule applications. It reuses connections, reduces overhead, and ensures optimal resource utilization.



Use Connection Pooling wisely to make your Mule application faster, scalable and reliable!



**MODULE  
10**

# 10.5 CACHING

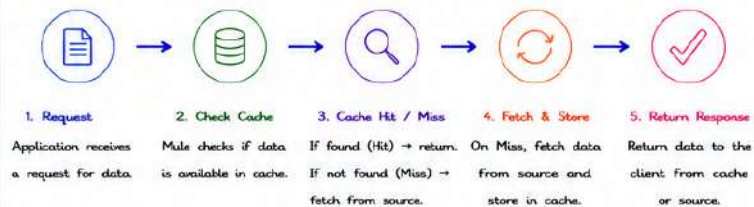


Caching in MuleSoft stores frequently accessed data temporarily so that it can be retrieved quickly for future requests. It reduces external calls, improves response time, lowers system load, and enhances overall performance.

## WHAT IS CACHING?

- Stores data in memory for quick access.
- Returns cached data instead of calling the external system.
- Reduces latency and improves throughput.
- Decreases load on databases and APIs.
- Mule provides several caching scopes: Application, Flow, Session.

## HOW CACHING WORKS



## CACHE HIT vs CACHE MISS

### Cache Hit

- ✓ Data found in cache.
- ✓ Fast response.
- ✓ No external call.
- ✓ Low resource usage.

### Cache Miss

- ✗ Data not in cache.
- ✗ Data fetched from source.
- ✗ Slower response.
- ✗ Higher resource usage.



## WHEN TO USE CACHING?

- When data is read frequently but changes infrequently.
- When external systems (DB, APIs) are slow or have rate limits.
- For reference/master data, configurations, lookups.
- When you want to improve performance and reduce cost.
- When data can be slightly stale (not real-time critical).

## MULESOFT CACHE SCOPES

| Scope       | Description                           | Use Case                                | Lifetime                     |
|-------------|---------------------------------------|-----------------------------------------|------------------------------|
| Application | Shared across the entire application. | Global data, config, reference data.    | Till application is running. |
| Flow        | Shared within the same flow.          | Data used multiple times within a flow. | Till flow completes.         |
| Session     | Unique to a session (per client).     | User-specific data, shopping cart, etc. | Till session expires.        |



## KEY BENEFITS

- Better Performance:** Faster responses for repeated requests.
- Reduced Load:** Fewer calls to databases and external systems.
- Cost Optimization:** Lower infrastructure and API call costs.
- Scalability:** Handles more requests with less system load.
- Improved Reliability:** Less dependency on external systems.

## CACHE CONFIGURATION (EXAMPLE)

1. Enable Caching in the Mule application (Global Configuration)

```
<mule:cache-config name="Caching_Config"
  defaultScope="APPLICATION"
  entryTtl="200" entryIdleTimeout="120" />
```

2. Use Caching Strategy in a Flow

```
<cache:cache id="Cache_Data" key="#[attributes.queryParams.id]"
  scope="APPLICATION" docName="Cache" />
```

- **entryTtl:** Time to live (in seconds) for cache entries.
- **entryIdleTimeout:** Time (in seconds) after which idle entries expire.
- **key:** Unique key to identify cached data.
- **scope:** APPLICATION (default), FLOW or SESSION.

## CACHE STRATEGIES

| Strategy      | Description                                               |
|---------------|-----------------------------------------------------------|
| Default (LRU) | Least Recently Used items are removed when cache is full. |
| LFU           | Least Frequently Used items are removed.                  |
| FIFO          | First In First Out (oldest items are removed first).      |
| Custom        | Custom eviction policy (advanced use).                    |

## EVICION POLICIES

| Policy           | Description                                             |
|------------------|---------------------------------------------------------|
| maxEntries       | Maximum number of entries allowed in cache.             |
| entryTtl         | Maximum time (in seconds) an entry can live.            |
| entryIdleTimeout | Time (in seconds) after which an idle entry is removed. |
| maxMemory        | Maximum memory (in bytes) cache can use.                |

## BEST PRACTICES

- ✓ Cache only data that does not change frequently.
- ✓ Use appropriate scope (Application/Flow/Session).
- ✓ Set proper TTL and idle timeout values.
- ✓ Avoid caching large objects.
- ✓ Monitor cache hit/miss ratio regularly.
- ✓ Invalidate or refresh cache on data changes.

## INTERVIEW TIPS

- Q. What is caching in MuleSoft?
- Q. What are cache scopes in MuleSoft?
- Q. What is the difference between Cache Hit and Miss?
- Q. How do you configure caching in MuleSoft?
- Q. How do TTL and Idle Timeout work?
- Q. When should we not use caching?

## REAL-WORLD EXAMPLE

An e-commerce application fetches product details from the database. Instead of querying the database for every request, product details are cached for 5 minutes.

Result:  
Response time improved by 70% and DB load reduced by 60%.



## KEY TAKEAWAY

Caching helps you serve data faster by storing and reusing it. Use it wisely with the right scope, expiration policies, and cache strategies to get the best performance and scalability.



Cache smartly today, serve faster tomorrow!

MODULE  
10

## 10.6 OBJECT STORE



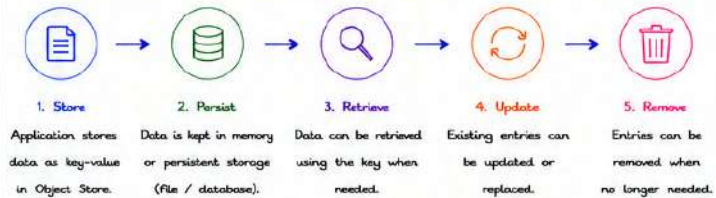
Object Store in MuleSoft is a **key-value** based storage mechanism used to store, retrieve, and manage data across events, flows, and even restarts. It is **persistent** (depending on the type), thread-safe, and ideal for **caching**, **tracking**, **state management**, and more.

## WHAT IS OBJECT STORE?

- Stores data as key-value pairs.
- Survives between events and application restarts (depending on configuration).
- Thread-safe and concurrent access supported.
- Data is stored in memory or persistent storage.
- Useful for caching, tracking IDs, storing intermediate results, etc.



## HOW OBJECT STORE WORKS



## COMMON USE CASES

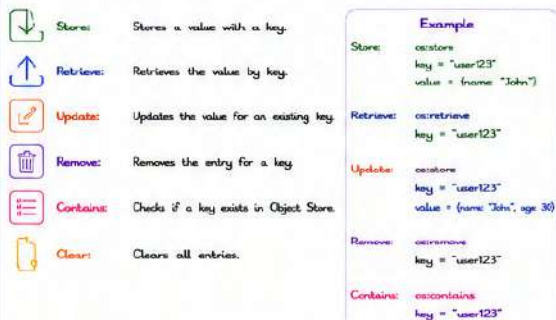
- ✓ Caching reference data (e.g., country codes, product list).
- ✓ Tracking processed message IDs to avoid duplicates.
- ✓ Storing user sessions or authentication tokens.
- ✓ Maintaining counters and sequence numbers.
- ✓ State management in long-running processes.
- ✓ Storing intermediate data between flows.



## OBJECT STORE TYPES IN MULESOFT

| Type       | Description                                    | Persistence    | Use Case                                   |
|------------|------------------------------------------------|----------------|--------------------------------------------|
| In-Memory  | Stores data only in memory.                    | Non-persistent | Fast access, temporary data.               |
| Persistent | Stores data on local file system.              | Persistent     | Data survives restarts, local storage.     |
| Database   | Stores data in an external database.           | Persistent     | Clustered apps, large data, shared access. |
| Custom     | Custom implementation (e.g., Redis, S3, etc.). | Depends        | Advanced use cases, external systems.      |

## KEY OPERATIONS



## OBJECT STORE CONFIGURATION (EXAMPLE)

```
<os:object-store persistent="true"
entryTtl="0" docname="Persistent Object Store"/>
```

- **persistent:** true → data is stored on file system.
- **entryTtl:** time (in seconds) after which an entry expires.
- **maxEntries:** maximum number of entries allowed.
- **maxEntrySize:** maximum size (in bytes) of each entry.
- **overflowStrategy:** what happens when maxEntries is reached.
- **docname:** name of the Object Store.

| Property                | Description                                                                       | Default       |
|-------------------------|-----------------------------------------------------------------------------------|---------------|
| <b>persistent</b>       | Enable persistence to local file system                                           | false         |
| <b>entryTtl</b>         | Time (sec) after which entry expires                                              | 0 (no expiry) |
| <b>maxEntries</b>       | Maximum number of entries                                                         | 1000          |
| <b>maxEntrySize</b>     | Maximum size (bytes) per entry                                                    | 100 KB        |
| <b>overflowStrategy</b> | Strategy when limit is reached (NONE, REMOVE_OLDEST, REMOVE_SOFT, REMOVE_HARDEST) | NONE          |

## OBJECT STORE SCOPE IN MULE APPLICATION

| Scope       | Visibility                    | Lifetime                    | Use Case                              |
|-------------|-------------------------------|-----------------------------|---------------------------------------|
| Application | All flows in the application  | Till application is running | Global data, reference data, caching. |
| Flow        | Only within the same flow     | Till flow completes         | Flow-level state management.          |
| Session     | Across events in same session | Till session expires        | User session data, tracking.          |

## BEST PRACTICES

- ✓ Use Object Store for data that needs to be shared across events or flows.
- ✓ Choose the right type based on performance and persistence needs.
- ✓ Set proper TTL to avoid outdated data.
- ✓ Monitor size and clean old entries regularly.
- ✓ Avoid storing large objects; store references if possible.
- ✓ Use Database Object Store for clustered deployments.



## INTERVIEW TIPS

- What is Object Store in MuleSoft?
- What are the different types of Object Store?
- What are the operations supported by Object Store?
- What is the difference between In-Memory and Persistent Object Store?
- When would you use Database Object Store?
- How do you configure TTL in Object Store?
- What are the use cases of Object Store?

## REAL-WORLD EXAMPLE

An order processing application needs to track processed order IDs to avoid duplicate processing. The flow stores each orderId in Object Store after successful processing. On next request, it checks if the orderId exists. If yes → skip processing. If no → process and store it.



## KEY TAKEAWAY

Object Store is a powerful key-value storage in MuleSoft that helps in caching, tracking, state management and sharing data across events and flows. Choose the right type and configure it well for better performance, scalability and reliability.



Use Object Store smartly to store, retrieve and manage data efficiently in your Mule applications!



**MODULE  
10**

# 10.7 PERFORMANCE TUNING



Performance Tuning in MuleSoft is the process of optimizing your applications to achieve higher throughput, lower latency, efficient resource utilization, and better scalability. It involves selecting the right patterns, configurations, and best practices.

## WHAT IS PERFORMANCE TUNING?

- Improves speed and responsiveness of applications.
- Reduces CPU, memory and network consumption.
- Increases throughput and scalability.
- Enhances user experience.
- Helps handle more load with existing infrastructure.



## PERFORMANCE TUNING APPROACH



## KEY AREAS TO TUNE IN MULE APPLICATIONS

- Streaming:** Avoid loading large payloads fully in memory.
- Batch Processing:** Process data in batches to reduce overhead.
- Memory Usage:** Optimize heap size and avoid unnecessary objects.
- Connection Pooling:** Reuse connections to external systems.
- Caching:** Store frequently used data to reduce external calls.
- Object Store:** Store intermediate data, IDs, and state information.
- Asynchronous Processing:** Use async scope and queues where possible.
- Error Handling:** Handle errors gracefully to avoid retries and timeouts.

## PERFORMANCE TUNING TECHNIQUES

| Technique                   | How it Helps                                       |
|-----------------------------|----------------------------------------------------|
| Use Streaming               | Reduces memory usage and improves throughput.      |
| Use Non-Blocking Operations | Frees threads while waiting for I/O.               |
| Enable Connection Pooling   | Reuses connections, reduces latency and overhead.  |
| Use Batch Processing        | Reduces number of transactions and round trips.    |
| Implement Caching           | Reduces calls to external systems and DB.          |
| Use Object Store            | Stores intermediate results and state.             |
| Tune JVM Settings           | Optimizes heap, GC, and thread usage.              |
| Optimize Flows              | Remove unnecessary components and transformations. |

## JVM TUNING TIPS

- Set initial and maximum heap size properly.  
`-Xms512m -Xmx2048m`
- Use G1GC (recommended for most cases).  
`-XX:+UseG1GC`
- Tune GC pause time.  
`-XX:MaxGCPauseMillis=200`
- Enable GC logging and monitor.  
`-Xloggc:file=gclog,time,uptime,level,logs`

## THREAD POOL TUNING

| Property                      | Description                                             |
|-------------------------------|---------------------------------------------------------|
| <code>maxThreadsActive</code> | Maximum number of active threads.                       |
| <code>maxThreadsIdle</code>   | Maximum number of idle threads.                         |
| <code>threadTTL</code>        | Time (in seconds) after which idle threads are removed. |
| <code>maxQueueSize</code>     | Maximum number of requests in the queue.                |
| <code>rejectedPolicy</code>   | Action when queue is full (ABORT / CALLER_RUNS).        |

## MONITORING & PROFILING TOOLS

- Anypoint Monitoring:** Real-time metrics, alerts and dashboards.
- JMX / VisualVM:** Monitor memory, threads, CPU and heap dumps.
- Mule Profiler:** Identify flow bottlenecks and execution time.
- GC Logs:** Analyze garbage collection behavior and pauses.
- APM Tools (New Relic, Dynatrace, etc.):** End-to-end performance monitoring.

## FLOW OPTIMIZATION BEST PRACTICES

- ✓ Avoid unnecessary DataWeave transformations.
- ✓ Use Set Variables instead of complex transformations.
- ✓ Filter data early in the flow.
- ✓ Use Choice router efficiently.
- ✓ Avoid excessive logging in production.
- ✓ Use Async scope for parallel processing.



## PERFORMANCE TUNING CHECKLIST

- ✓ Enable streaming for large payloads.
- ✓ Use batch processing for bulk data.
- ✓ Configure connection pooling for all external calls.
- ✓ Implement caching for frequently used data.
- ✓ Use Object Store for state and tracking.
- ✓ Tune JVM (heap size, GC, threads).
- ✓ Monitor and analyze metrics regularly.
- ✓ Load test and identify bottlenecks.

## COMMON BOTTLENECKS & FIXES

| Bottleneck         | Symptoms                        | Possible Fix                                            |
|--------------------|---------------------------------|---------------------------------------------------------|
| High Memory Usage  | OutOfMemoryError, slow response | Use streaming, increase heap, optimize objects          |
| Long Response Time | High latency                    | Enable caching, connection pooling, async processing    |
| Low Throughput     | Requests taking too long        | Batch processing, tune thread pools, optimize flows     |
| DB / API Overload  | Timeouts, failures              | Use caching, pooling, limit concurrency                 |
| GC Pauses          | Stops-the-world delays          | Tune GC settings, increase heap, reduce object creation |

## INTERVIEW TIPS

- Q. What is performance tuning in MuleSoft?
- Q. How does streaming improve performance?
- Q. Why do we use connection pooling?
- Q. Difference between caching and Object Store?
- Q. How does batch processing help performance?
- Q. What tools do you use for performance tuning?
- Q. How would you identify a bottleneck?

## REAL-WORLD EXAMPLE

An e-commerce application was slow during peak hours. After performance tuning:

- ✓ Enabled streaming for order data (reduced memory usage).
- ✓ Implemented caching for product catalog (reduced DB calls by 80%).
- ✓ Enabled connection pooling for payment gateway.
- ✓ Used batch processing for order confirmation emails.
- ✓ Tuned JVM and thread pools.

Result: Response time improved by 65% and throughput increased by 3x.

## KEY TAKEAWAY

Performance Tuning is a continuous process of measuring, analysing, optimizing and monitoring. By applying the right techniques and configurations, you can build Mule applications that are fast, reliable, scalable, and cost-effective.



Measure. Analyze. Optimize. Test. Monitor. Repeat.

This cycle is the key to a high performance and scalable Mule application!



# Thank You!

For reading and learning with us.

We hope this **MuleSoft Anypoint Platform Components Guide – Volume 2** has helped you deepen your knowledge and enhance your integration expertise.



Keep practicing, keep building, and keep innovating.  
You're now one step closer to becoming  
a **MuleSoft integration expert!**



**Thank you for being a part of this journey!**

Your curiosity and commitment drive your growth and inspire continuous learning.



**Volume 3 is coming soon! ⇐**

More components, more insights, and more real-world scenarios to take your integration expertise to the next level.

— **Stay tuned!** —